

Leonardo Tomasela Leal
170291

Projeto de um Processador RISC-V

São José dos Campos - Brasil
Abril de 2026

Leonardo Tomasela Leal
170291

Projeto de um Processador RISC-V

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Sérgio Ronaldo Barros dos Santos

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Abril de 2026

Resumo

Este relatório descreve a implementação completa de um processador RISC-V de 32 bits, seguindo o subconjunto RV32IM e operando em ciclo único. Foram projetados, simulados e validados todos os componentes da unidade de processamento: a Unidade Lógica e Aritmética (ULA) com suporte a 24 operações, incluindo a extensão de multiplicação e divisão (RV32M); o banco de registradores com 32 registradores de 32 bits; a memória de instruções (ROM) inicializada a partir de arquivo hexadecimal; a memória de dados síncrona com lógica de *byte enable* e região de E/S mapeada; o gerador de imediatos para os formatos I, S, B, U e J; a unidade de branch para desvios condicionais e saltos; a unidade de controle, que decodifica as instruções e gera todos os sinais de controle; e o Contador de Programa (PC) com reset síncrono. Todos os módulos foram integrados em um único módulo topo, que foi então encapsulado em um módulo para FPGA com divisor de clock para operação em bancada. O processador foi testado com dois programas de exemplo (Fibonacci e Fatorial), compilados utilizando o conjunto de ferramentas GNU RISC-V e convertidos para o formato hexadecimal. As simulações no ModelSim comprovam o funcionamento correto de todos os componentes, e os testes em bancada (FPGA) confirmam os resultados esperados, validando o projeto em hardware.

Palavras-chave: RISC-V. Unidade Lógica e Aritmética. Banco de registradores. Memória de instruções. Memória de dados. Gerador de imediatos. Unidade de branch. Unidade de controle. Contador de Programa. FPGA. SystemVerilog. ModelSim.

Lista de abreviaturas e siglas

ALU	Arithmetic Logic Unit (Unidade Lógica e Aritmética)
AUIPC	Add Upper Immediate to Program Counter
BR	Banco de Registradores (Register File)
CISC	Complex Instruction Set Computer
CPLD	Complex Programmable Logic Device
DIV	Division
EA	Endereço Efetivo
FPGA	Field-Programmable Gate Array
HDL	Hardware Description Language
ISA	Instruction Set Architecture
JAL	Jump and Link
JALR	Jump and Link Register
LUI	Load Upper Immediate
MUL	Multiplication
PC	Program Counter (Contador de Programa)
RAM	Random Access Memory
REM	Remainder
RISC	Reduced Instruction Set Computer
ROM	Read-Only Memory
RV32I	RISC-V 32-bit Integer Base Instruction Set
RV32IM	RV32I + Integer Multiplication and Division Extension
SRA	Shift Right Arithmetic
SRL	Shift Right Logical

SV	SystemVerilog
ULA	Unidade Lógica e Aritmética
UVM	Universal Verification Methodology

Lista de ilustrações

Figura 1 – Arquitetura de Von Neumann e de Harvard	17
Figura 2 – Endereçamento por registrador	18
Figura 3 – Endereçamento por imediato	19
Figura 4 – Endereçamento por deslocamento	19
Figura 5 – Endereçamento PC-relativo	20
Figura 6 – Field-Programmable Gate Array	26
Figura 7 – Caminho de Dados Completo	33
Figura 8 – Caminho de Dados das Instruções do Tipo R	34
Figura 9 – Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato	35
Figura 10 – Caminho de Dados das Instruções de Carregamento	36
Figura 11 – Caminho de Dados das Instruções de Armazenamento	37
Figura 12 – Caminho de Dados das Instruções de Desvio Condicional	38
Figura 13 – Caminho de Dados da Instrução de Salto e Ligação	39
Figura 14 – Caminho de Dados da Instrução de Salto Indireto	40
Figura 15 – Caminho de Dados de Carregamento de Imediato Superior	41
Figura 16 – Caminho de Dados da Adição de Imediato Superior ao PC	42
Figura 17 – Caminho de dados para a instrução ebreak	43
Figura 18 – Waveform das instruções do tipo R aritméticas (ADD, SUB, MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU)	49
Figura 19 – Waveform das instruções de deslocamento (SLL, SRL, SRA)	51
Figura 20 – Instruções do Tipo R Lógicas e Comparações	52
Figura 21 – Waveform das instruções do tipo I (ADDI, XORI, ORI, ANDI, SLLI, SRLI, SRAI, SLTI, SLTIU).	53
Figura 22 – Waveform do cálculo de endereço para LOAD/STORE.	54
Figura 23 – Waveform das instruções de desvio condicional (BEQ, BNE, BLT, BGE, BLTU, BGEU).	55
Figura 24 – Waveform das instruções JAL, JALR e AUIPC.	56
Figura 25 – Waveform do banco de registradores 1	59
Figura 26 – Waveform do banco de registradores 2	59
Figura 27 – Waveform da memória de instruções.	62
Figura 28 – Waveform da simulação da memória de dados (com E/S).	67
Figura 29 – Waveform da simulação da unidade de branch.	71
Figura 30 – Waveform da simulação do gerador de imediatos.	75
Figura 31 – Waveform da simulação da unidade de controle.	80
Figura 32 – Waveform da simulação integrada do processador (Fibonacci).	88
Figura 33 – Waveform da simulação integrada para o programa Fatorial.	89

Lista de tabelas

Tabela 1	–	Tipos de instruções (formatos R, I, S, B, U)	28
Tabela 2	–	Formato da instrução <code>jal</code> (Tipo J)	28
Tabela 3	–	Instruções do tipo R	29
Tabela 4	–	Instruções do tipo I	30
Tabela 5	–	Instruções do tipo S	30
Tabela 6	–	Instruções do tipo B	31
Tabela 7	–	Instruções do tipo U	31
Tabela 8	–	Instrução do tipo J	31
Tabela 9	–	Instrução do sistema implementada	32

Sumário

1	INTRODUÇÃO	11
2	OBJETIVOS	13
2.1	Geral	13
2.2	Específicos	13
3	FUNDAMENTAÇÃO TEÓRICA	15
3.1	Computador	15
3.2	Organização de Computadores	15
3.3	Arquitetura de Computadores	15
3.3.1	CISC vs RISC	15
3.3.1.1	Arquitetura CISC (Complex Instruction Set Computer)	16
3.3.1.2	Arquitetura RISC (Reduced Instruction Set Computer)	16
3.3.2	Arquitetura de Von Neumann vs. Arquitetura de Harvard	16
3.3.2.1	Arquitetura de Von Neumann	17
3.3.2.2	Arquitetura de Harvard	17
3.3.3	Modos de Endereçamento	18
3.3.3.1	Endereçamento por registrador	18
3.3.3.2	Endereçamento por imediato	19
3.3.3.3	Endereçamento por deslocamento	19
3.3.3.4	Endereçamento PC-relativo	19
3.3.4	Arquitetura RISC-V	20
3.3.4.1	Origem	20
3.3.4.2	Características	20
3.3.4.3	Modos de Endereçamento	21
3.3.5	Justificativa da Escolha	21
3.4	Linguagens de Descrição de Hardware (HDLs)	21
3.4.1	A Linguagem Verilog	22
3.4.2	A Linguagem SystemVerilog	24
3.5	FPGA (Field-Programmable Gate Array)	25
4	DESENVOLVIMENTO	27
4.1	Características Gerais do Processador	27
4.2	Desenvolvimento da Arquitetura	27
4.2.1	Instruções do Tipo R	28
4.2.2	Instruções do Tipo I	29

4.2.3	Instruções do Tipo S	30
4.2.4	Instruções do Tipo B	30
4.2.5	Instruções do Tipo U	31
4.2.6	Instruções do Tipo J	31
4.2.7	Instruções do Sistema	31
4.3	Desenvolvimento da Organização	32
4.3.1	Caminho de Dados Completo	33
4.3.2	Caminho de Dados das Instruções do Tipo R	33
4.3.3	Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato (<i>addi, xori, ori, andi, slli, srli, srai, slti, sltiu</i>)	34
4.3.4	Caminho de Dados das Instruções de Carregamento (<i>lb, lh, lw, lbu, lhu</i>)	35
4.3.5	Caminho de Dados das Instruções de Armazenamento (<i>sb, sh, sw</i>)	36
4.3.6	Caminho de Dados das Instruções de Desvio Condicional (<i>beq, bne, blt, bge, bltu, bgeu</i>)	37
4.3.7	Caminho de Dados da Instrução de Salto e Ligação (<i>jal</i>)	38
4.3.8	Caminho de Dados da Instrução de Salto Indireto (<i>jalr</i>)	39
4.3.9	Caminho de Dados de Carregamento de Imediato Superior (<i>lui</i>)	40
4.3.10	Caminho de Dados da Adição de Imediato Superior ao PC (<i>auipc</i>)	41
4.3.11	Caminho de dados para Instruções do Sistema (<i>ebreak</i>)	42
4.4	Definições Globais e Package <i>data_structures</i>	43
4.5	Desenvolvimento da Unidade Lógica e Aritmética (ALU)	45
4.5.1	Interface da ALU	45
4.5.2	Decodificação da Operação Mapeamento para <i>aluop_t</i>	45
4.5.3	Geração dos Operandos	46
4.5.4	Implementação das Operações	47
4.5.5	Desenvolvimento dos Testbenches	47
4.5.5.1	Estrutura do Testbench	48
4.5.5.2	Organização dos Estímulos	48
4.5.6	Resultados da Simulação da ALU	49
4.5.6.1	Instruções do tipo R	49
4.5.6.1.1	Aritméticas	49
4.5.6.1.2	Shift	51
4.5.6.1.3	Lógicas	52
4.5.6.2	Instruções do Tipo I (Aritmética com Imediato)	53
4.5.6.3	Instruções de Load e Store	54
4.5.6.4	Instruções de Desvio Condicional	55
4.5.6.5	Instruções de Salto (JAL, JALR) e AUIPC	56
4.6	Banco de Registradores	57
4.6.1	Implementação do Módulo <i>regfile</i>	57

4.6.2	Testbench do Banco de Registradores	58
4.6.3	Resultados da Simulação	59
4.7	Memória de Instruções	60
4.7.1	Implementação do Módulo <code>inst_memory</code>	60
4.7.2	Arquivo de Inicialização <code>fib.hex</code>	61
4.7.3	Testbench da Memória de Instruções	61
4.7.4	Resultados da Simulação	62
4.8	Memória de Dados e Entrada e Saída	63
4.8.1	Implementação do Módulo	63
4.8.2	Testbench da Memória de Dados e Entrada e Saída	66
4.8.3	Resultados da Simulação	67
4.9	Unidade de Branch	69
4.9.1	Implementação do Módulo <code>branch_unit</code>	69
4.9.2	Testbench da Unidade de Branch	70
4.9.3	Resultados da Simulação	71
4.10	Contador de Programa	71
4.11	Gerador de Imediatos	72
4.11.1	Implementação do Módulo <code>immediate_generator</code>	72
4.11.2	Testbench do Gerador de Imediatos	73
4.11.3	Resultados da Simulação	75
4.12	Unidade de Controle	76
4.12.1	Interface e Sinais de Controle	76
4.12.2	Implementação do Módulo	77
4.12.3	Testbench da Unidade de Controle	79
4.12.4	Resultados da Simulação	80
4.13	Programas de Teste	81
4.13.1	Programa Fibonacci	81
4.13.2	Programa Fatorial	82
4.13.3	Processo de Compilação	82
4.14	Integração Final do Processador	84
4.14.1	Implementação do Módulo Topo	84
4.14.2	Testbench de Integração	86
4.14.3	Resultados da Simulação Integrada	88
4.14.3.1	Simulação do programa Fatorial	89
4.15	Implementação em FPGA e Testes em Bancada	90
4.15.1	Módulo de Topo para FPGA	90
4.15.2	Processo de Síntese e Programação	91
4.15.3	Testes em Bancada	91
5	CONSIDERAÇÕES FINAIS	92

5.0.1	Dificuldades Encontradas	93
-------	------------------------------------	----

REFERÊNCIAS	95
------------------------------	-----------

1 Introdução

"Os computadores levaram a uma terceira revolução para a civilização, com a revolução da informação tomando o seu lugar ao lado da agricultura e revoluções industriais." (PATTERSON; HENNESSY, 2018, p. 41, tradução própria). A evolução dos sistemas computacionais está intrinsecamente ligada ao constante aprimoramento das suas arquiteturas e da sua organização física. O estudo prático do funcionamento lógico de um processador apresenta-se como uma etapa indispensável para a consolidação teórica dos conceitos abordados. Neste contexto, o presente relatório documenta o projeto completo de um processador digital baseado na arquitetura RISC-V de 32 bits, operando sob um modelo de execução de ciclo único (monociclo), desde sua especificação arquitetural até sua implementação em FPGA.

A motivação central para a adoção da norma RISC-V no desenvolvimento deste projeto reside na sua natureza de padrão de código aberto (*open source*) e na sua licença totalmente gratuita. Estas características posicionam o RISC-V como uma alternativa acadêmica e industrial altamente viável e moderna face a arquiteturas proprietárias e fechadas que dominam o mercado atual, tais como as famílias x86 e ARM. Para além da acessibilidade financeira e filosófica, o RISC-V destaca-se pela sua profunda modularidade. O seu conjunto de instruções foi estrategicamente desenhado para ser enxuto na base, sendo suportado por várias extensões normalizadas, o que o torna fácil de adaptar para diversos usos de processamento. Esta modularidade é o que viabiliza a inclusão, neste processador, de um bloco específico de multiplicação e divisão em hardware acoplado à base de inteiros (constituindo um subconjunto da variante RV32IM).

O projeto foi desenvolvido em SystemVerilog e abrange todos os componentes necessários para a execução de programas completos: Unidade Lógica e Aritmética (ULA) com extensão M, banco de registradores, memórias de instrução e dados (esta última com região de E/S mapeada), gerador de imediatos, unidade de branch, unidade de controle e Contador de Programa. Todos esses módulos foram integrados em um caminho de dados monociclo e validados por meio de simulações no ModelSim. Posteriormente, o processador foi sintetizado para um kit FPGA, utilizando um divisor de clock para adequar a frequência de operação aos periféricos da bancada. Dois programas de teste (Fibonacci e Fatorial) foram compilados para a arquitetura RV32IM e executados tanto em simulação quanto em bancada, comprovando o funcionamento prático do sistema.

O relatório está estruturado da seguinte forma: no Capítulo 2, são apresentados os objetivos do projeto. O Capítulo 3 contém a fundamentação teórica, abordando conceitos de arquitetura e organização de computadores, linguagens de descrição de hardware e FP-

GAs. O Capítulo 4 descreve detalhadamente o desenvolvimento de todos os componentes, sua integração e os programas de teste. Por fim, o Capítulo 5 apresenta as considerações finais, incluindo as dificuldades encontradas e os resultados obtidos em bancada.

...

2 Objetivos

2.1 Geral

Projetar, implementar e validar, por meio da linguagem de descrição de hardware SystemVerilog, um processador RISC-V de 32 bits completo, seguindo o subconjunto RV32IM, operando em ciclo único, e capaz de executar programas de teste em simulação e em hardware (FPGA), abrangendo desde os componentes individuais até a integração final e a síntese em dispositivo programável.

2.2 Específicos

Para a realização do trabalho, os seguintes objetivos específicos foram definidos e atendidos:

1. Implementar a Unidade Lógica e Aritmética (ULA) com suporte às operações da base inteira RV32I (adição, subtração, operações lógicas, deslocamentos e comparações) e à extensão de multiplicação e divisão RV32M, totalizando 24 operações.
2. Projetar o banco de registradores (Register File) com 32 registradores de 32 bits, respeitando a característica de `x0` permanentemente igual a zero e utilizando escrita síncrona e leitura combinacional.
3. Desenvolver a memória de instruções (ROM) com 256 palavras de 32 bits, inicializada a partir de um arquivo hexadecimal, com leitura combinacional.
4. Implementar a memória de dados (RAM) síncrona de 1024 palavras, com suporte a acessos a byte, meia palavra e palavra, incluindo a lógica de *byte enable*, a correta extensão de sinal/zero e uma região de E/S mapeada nos quatro últimos endereços.
5. Implementar o gerador de imediatos para extrair e estender os valores imediatos dos formatos I, S, B, U e J, conforme a especificação RISC-V.
6. Desenvolver a unidade de branch para cálculo do próximo PC em desvios condicionais (`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`), saltos incondicionais (`jal`) e saltos indiretos (`jalr`).
7. Projetar a unidade de controle para decodificar as instruções e gerar todos os sinais de controle necessários, incluindo o sinal `ebreak_c` para a instrução de sistema `ebreak`.

8. Integrar todos os componentes em um módulo topo (`integ`), formando o caminho de dados completo do processador monociclo.
9. Adaptar o processador para síntese em FPGA, implementando um divisor de clock para operação em bancada, e realizar testes práticos com os periféricos (chaves e displays) utilizando programas de teste (Fibonacci e Fatorial).
10. Validar o funcionamento do processador por meio de simulações no ModelSim e testes em hardware, documentando os resultados obtidos.

3 Fundamentação Teórica

Esta seção tem como objetivo estabelecer a base teórica e conceitual que fundamenta o desenvolvimento do projeto. Nela, são abordados os princípios fundamentais de arquitetura e organização de computadores, os paradigmas de memória e modos de endereçamento, bem como as principais características da arquitetura RISC-V, fornecendo o embasamento literário necessário para justificar as escolhas de projeto adotadas.

3.1 Computador

Define-se, em sua essência, como um sistema digital e eletrônico projetado para receber, armazenar, processar e transmitir informações de maneira automatizada, operando sob o controle estrito de um conjunto de instruções previamente armazenado.

3.2 Organização de Computadores

Refere-se à implementação física e de hardware responsável por concretizar as especificações da arquitetura proposta. O projeto organizacional envolve o arranjo das unidades operacionais e suas interconexões lógicas, englobando o desenho do caminho de dados (datapath), o dimensionamento dos sinais da Unidade de Controle e a hierarquia topológica do sistema de memórias.

3.3 Arquitetura de Computadores

Corresponde aos atributos lógicos de um sistema computacional que são diretamente visíveis ao programador de baixo nível ou ao desenvolvedor de compiladores. Ela engloba a definição formal do Conjunto de Instruções (ISA - Instruction Set Architecture), os tipos de dados matemáticos suportados, a quantidade de registradores de uso geral expostos e os métodos padronizados de acesso à memória. Em suma, a arquitetura estabelece as especificações funcionais que determinam o que o processador é capaz de realizar.

3.3.1 CISC vs RISC

A distinção entre os paradigmas CISC (Complex Instruction Set Computer) e RISC (Reduced Instruction Set Computer) é fundamental na classificação de processadores.

3.3.1.1 Arquitetura CISC (Complex Instruction Set Computer)

A arquitetura CISC caracteriza-se por um conjunto amplo e heterogêneo de instruções, com tamanhos variáveis e múltiplos modos de endereçamento. Neste modelo, uma única instrução pode realizar várias operações de baixo nível (como acessar a memória, processar o dado e guardar o resultado). Embora reduza o tamanho do código compilado, esta abordagem aumenta substancialmente a complexidade do hardware e da Unidade de Controle, resultando em ciclos de execução irregulares.

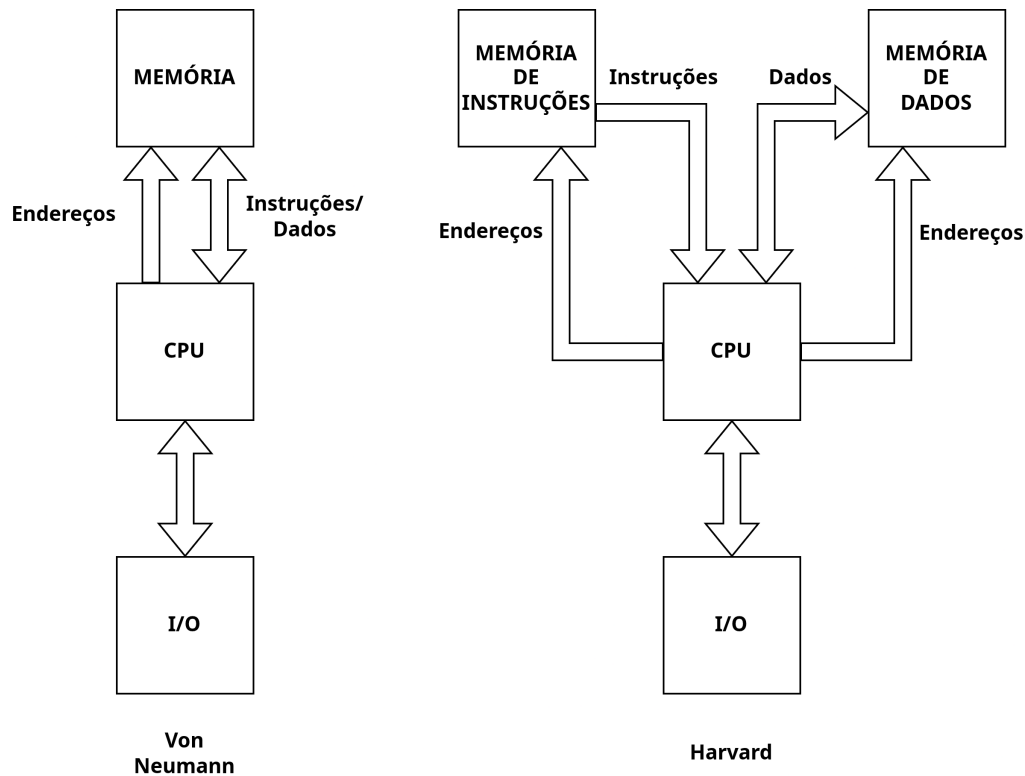
3.3.1.2 Arquitetura RISC (Reduced Instruction Set Computer)

A arquitetura RISC adota um conjunto reduzido de instruções simples, otimizadas e de formato fixo, o que agiliza drasticamente a decodificação. A sua característica estrutural central é a adoção rigorosa do modelo load-store, em que o processamento matemático e lógico ocorre exclusivamente entre os registradores internos. O acesso à memória principal é estritamente restrito a instruções dedicadas de leitura (load) e escrita (store). Esta filosofia simplifica o hardware, reduz os modos de endereçamento e garante a estabilidade previsível essencial para viabilizar a implementação de um caminho de dados de ciclo único.

3.3.2 Arquitetura de Von Neumann vs. Arquitetura de Harvard

A Figura 1 apresenta as diferenças entre as arquiteturas, que serão elaboradas nos subtópicos seguintes.

Figura 1 – Arquitetura de Von Neumann e de Harvard



Fonte: O Autor

3.3.2.1 Arquitetura de Von Neumann

Fundamentada no conceito clássico de programa armazenado, este modelo caracteriza-se por manter a memória de dados e a memória de instruções estruturadas de forma unificada. Consequentemente, o sistema dispõe de um barramento único para o tráfego de informações. Essa característica estrutural cria um afunilamento (conhecido como "Gargalo de Von Neumann"), que obriga o processador a executar as instruções em uma ordem estritamente sequencial, impossibilitando a busca de uma instrução e a manipulação de um dado no mesmo instante de tempo. É um padrão habitualmente associado aos processadores de instrução complexa (CISC).

3.3.2.2 Arquitetura de Harvard

Em contrapartida, este paradigma resolve o gargalo de concorrência ao adotar a separação física estrita entre a memória de dados e a memória de instruções. A utilização de barramentos de comunicação isolados possibilita que o processador efetue a busca da próxima instrução e a leitura ou escrita de um dado operacional de forma simultânea. Esta abordagem de acesso paralelo é inerente ao modelo RISC e torna-se um requisito vital para a implementação de um caminho de dados de ciclo único.

3.3.3 Modos de Endereçamento

Os modos de endereçamento consistem nos mecanismos lógicos e aritméticos pelos quais a arquitetura do hardware calcula os endereços efetivos dos operandos requisitados para processar uma instrução. Para esta seção utilizaremos a seguinte notação, baseada na utilizada por Stallings ([STALLINGS, 2017](#)) para explicar os modos de endereçamento:

1. **A** $\Rightarrow$ O conteúdo no campo de endereço dentro de uma instrução.
2. **PC** $\Rightarrow$ O conteúdo em uma instrução se refere ao registrador contador de programa.
3. **R** $\Rightarrow$ O conteúdo em uma instrução se refere a um registrador.
4. **(X)** $\Rightarrow$ O conteúdo do local de memória X ou do registrador X.
5. **EA** $\Rightarrow$ Endereço real do operando.

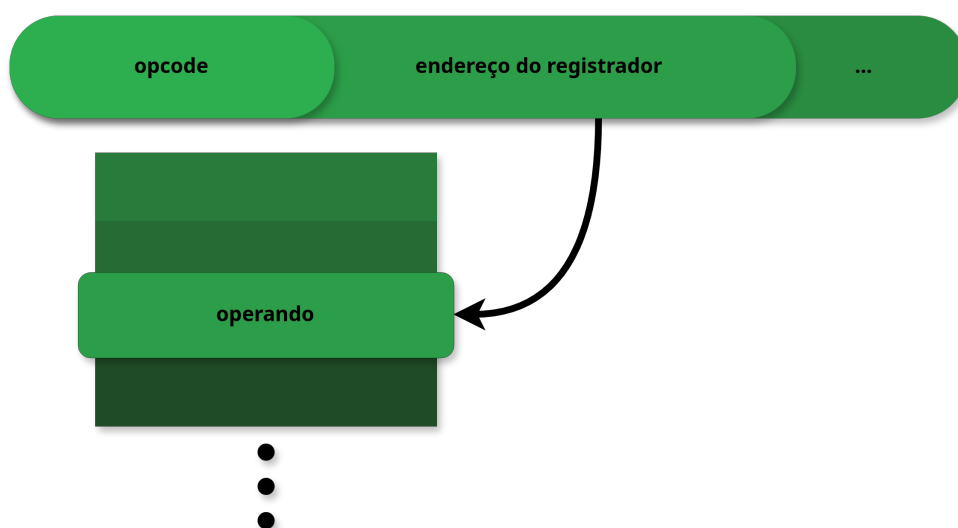
O escopo do projeto sustenta-se nos seguintes modos principais:

3.3.3.1 Endereçamento por registrador

A própria instrução decodificada faz uma referência direta ao endereço numérico de um registrador interno, que, por sua vez, armazena o operando necessário para a Unidade Lógica e Aritmética:

$$EA = R$$

Figura 2 – Endereçamento por registrador



Fonte: O Autor

3.3.3.2 Endereçamento por imediato

O valor numérico constante a ser utilizado na operação não é buscado no banco de registradores, pois já se encontra embutido fisicamente em um campo específico dos próprios bits da instrução:

$$EA = A$$

Figura 3 – Endereçamento por imediato



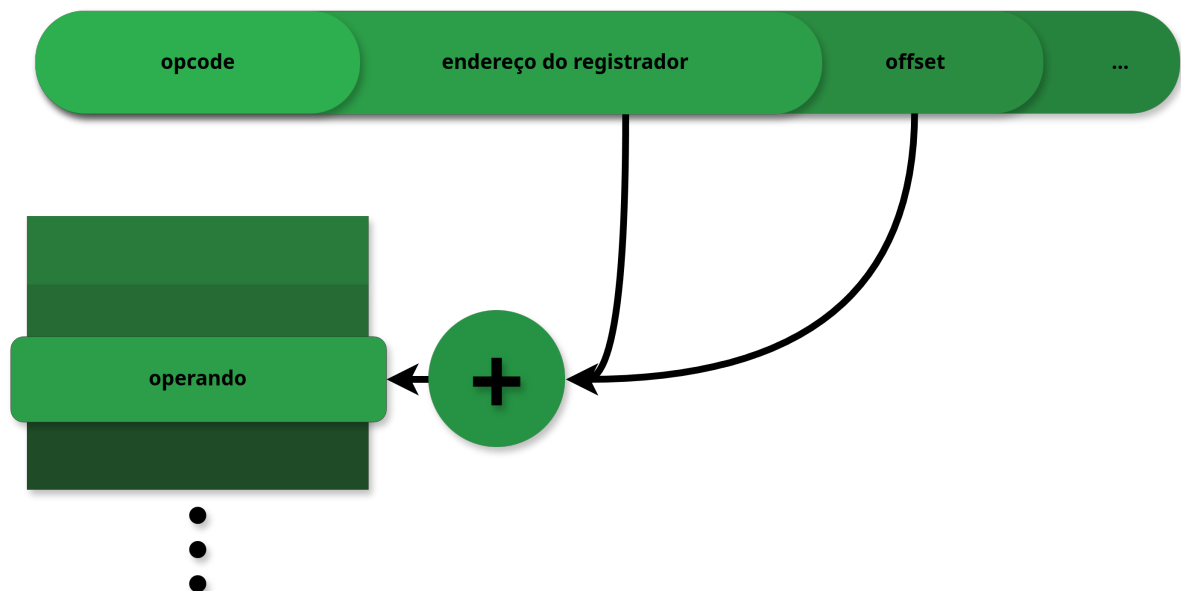
Fonte: O Autor

3.3.3.3 Endereçamento por deslocamento

O processo de acesso ao módulo de memória de dados é estabelecido somando-se um valor referencial de base (armazenado em um registrador) a um valor fixo de deslocamento (caracterizado como um offset imediato de 12 bits na ISA abordada):

$$EA = A + (R)$$

Figura 4 – Endereçamento por deslocamento



Fonte: O Autor

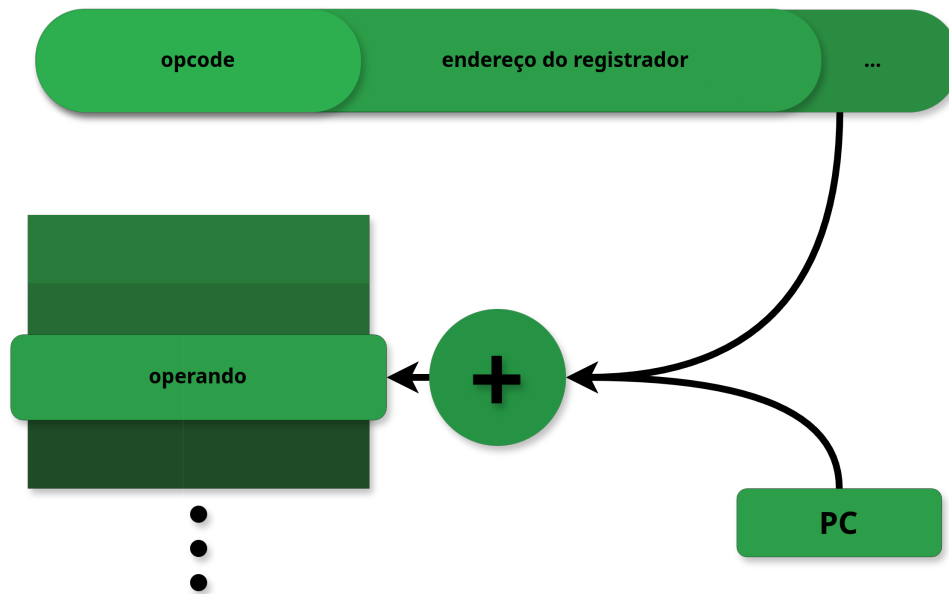
3.3.3.4 Endereçamento PC-relativo

Empregado no fluxo de controle de saltos e desvios, o endereço computado para o salto é obtido somando-se o valor atual armazenado no Contador de Programa (PC) a

um valor imediato extraído da instrução:

$$EA = A + (PC)$$

Figura 5 – Endereçamento PC-relativo



Fonte: O Autor

3.3.4 Arquitetura RISC-V

3.3.4.1 Origem

O RISC-V foi concebido e estruturado na Universidade da Califórnia em Berkeley (UC Berkeley) ao longo do verão do ano de 2010. O projeto foi idealizado e gerido por uma equipe de acadêmicos e pesquisadores englobando pioneiros da computação como Krste Asanovi e David Patterson. O desenvolvimento se originou da necessidade intrínseca da equipe por um processador de arquitetura moderna e de licença livre (gratuita), passível de ser implementado sem restrições em aulas práticas e pesquisas.

3.3.4.2 Características

O RISC-V é uma arquitetura estabelecida sobre os preceitos do modelo RISC (Reduced Instruction Set Computer). Diferenciando-se das arquiteturas complexas (CISC), ele é construído sobre o arranjo load-store (onde o acesso físico à memória restringe-se a instruções exclusivas de leitura e escrita), disponibiliza um largo conjunto de 32 registradores de uso geral e processa instruções com formato simples e de tamanho rigidamente fixado em 32 bits. O sistema utiliza um mapeamento de endereçamento com formato padrão orientado a byte de acordo com a ordem little endian ([RISC-V INTERNATIONAL, 2024](#)).

3.3.4.3 Modos de Endereçamento

Mantendo plena conformidade com sua filosofia de redução e simplificação computacional (cujo foco preza pela minimização da complexidade dos decodificadores), o conjunto de instruções do RISC-V opera unicamente sob o emprego dos modos de endereçamento por registrador (para manipulações puramente lógicas/matemáticas), por imediato (para interações com constantes numéricas), por deslocamento (recurso fundamental das instruções nativas de Load e Store) e de maneira PC-relativa (encarregado pelo direcionamento imperativo de saltos e desvios estruturados) ([RISC-V INTERNATIONAL, 2024](#)).

3.3.5 Justificativa da Escolha

A escolha definitiva da ISA RISC-V para conduzir o desenvolvimento lógico exposto neste relatório consolida-se, fundamentalmente, pelas garantias de seu padrão de código aberto (open source) atrelado à abstenção total de pagamentos e taxas de licenciamento comercial (gratuito). Essas propriedades proporcionam uma viabilidade sem precedentes e firmam a arquitetura como uma alternativa moderna, acadêmica e industrialmente proeminente frente aos competidores corporativos preestabelecidos, tais como os processadores das famílias x86 e ARM. Além dos benefícios econômicos e da disponibilidade, a base técnica justifica-se pela adoção de um design profundamente modular. O RISC-V disponibiliza um conjunto essencial de instruções rigorosamente simplificadas, sendo projetado de tal forma que é perfeitamente exequível adaptá-lo a variados cenários mediante o simples acoplamento de extensões. Essa prerrogativa arquitetural viabiliza e acomoda a inclusão direcionada do bloco de multiplicação em hardware abordada neste projeto.

3.4 Linguagens de Descrição de Hardware (HDLs)

O desenvolvimento de sistemas digitais complexos, como o processador RISC-V proposto, exige ferramentas de projeto robustas que vão além da tradicional entrada esquemática. As Linguagens de Descrição de Hardware (Hardware Description Languages - HDLs) surgem como uma alternativa poderosa, permitindo ao projetista descrever a estrutura e o comportamento de um sistema eletrônico por meio de um arquivo de texto, seguindo uma sintaxe e semântica bem definidas. Este arquivo de descrição serve como uma especificação executável do hardware e pode ser utilizado para gerar dados de programação para dispositivos lógicos programáveis ou para ser sintetizado em um circuito físico.

Diferentemente das linguagens de programação de software, que possuem um fluxo de execução sequencial de instruções (paradigma imperativo), as HDLs são inerentemente

concorrentes (ou paralelas). Em uma descrição de hardware, a execução das expressões e atribuições de valores ocorre de forma a emular a propagação de sinais por um circuito digital, onde múltiplas operações acontecem simultaneamente. Esta característica fundamental torna as HDLs a ferramenta ideal para a descrição formal de circuitos eletrônicos.

As principais vantagens da adoção de uma HDL no projeto de hardware incluem:

1. **Abstração e Hierarquia:** Permitem representar desde equações booleanas e tabelas verdade até operações matemáticas complexas, facilitando a criação de projetos hierárquicos onde um componente pode ser reutilizado como parte de outro.
2. **Síntese Lógica:** Uma descrição em HDL pode ser utilizada por uma ferramenta de síntese lógica para, em conjunto com uma biblioteca de componentes, gerar automaticamente a lista de interconexões (netlist) de um circuito otimizado.
3. **Simulação:** Permitem a simulação funcional do projeto para verificar seu comportamento antes mesmo de sua implementação física, garantindo a correteza do sistema em desenvolvimento.
4. **Documentação:** Servem como uma forma de documentação precisa e formal da estrutura e do funcionamento do hardware projetado.

Entre as diversas HDLs existentes, atualmente as mais conhecidas e utilizadas são o VHDL, que teve origem em um programa de pesquisa do Departamento de Defesa dos EUA, e o Verilog, que será a linguagem de escolha para a implementação do processador deste projeto.

3.4.1 A Linguagem Verilog

A linguagem Verilog foi desenvolvida para a descrição de circuitos eletrônicos digitais, sendo amplamente utilizada na documentação, simulação e síntese automática de sistemas digitais, especialmente em ambientes acadêmicos e industriais. Sua sintaxe possui semelhanças com a linguagem de programação C, o que facilita seu aprendizado para programadores familiarizados com este paradigma.

Um dos pontos mais importantes para a correta compreensão do Verilog é a clara distinção entre a modelagem de hardware e a modelagem de software:

1. **Modelagem de Software (linguagens como C, Python):** Assume-se um processador como unidade de execução. O programador escreve um código que será executado de forma sequencial pelo processador, passo a passo ao longo do tempo. Há uma exploração temporal da execução, onde um único somador pode ser reutilizado em múltiplas iterações de um laço de repetição.

2. **Modelagem de Hardware (Verilog, VHDL):** Modelam-se os próprios circuitos digitais. O projetista explora a execução não apenas no tempo, mas também no espaço, podendo replicar componentes (como somadores) para que operações sejam realizadas simultaneamente. A simultaneidade e o paralelismo são características intrínsecas da descrição.

Para a descrição de sistemas em Verilog, o código é organizado em módulos, que representam blocos de hardware com entradas e saídas definidas. Estes módulos podem ser instanciados e interconectados para formar sistemas de maior complexidade.

Dentro de um módulo, três estilos principais podem ser empregados para descrever seu funcionamento:

1. **Descrição Estrutural:** Descreve a interconexão entre componentes mais simples (sub-módulos ou portas lógicas), sendo análoga a uma lista de componentes e suas conexões.
2. **Descrição Comportamental:** Descreve o funcionamento desejado do circuito, especificando como as saídas devem reagir a mudanças nas entradas, sem necessariamente detalhar sua estrutura interna. Utiliza estruturas como `always` e `if-else`.
3. **Descrição de Fluxo de Dados (Dataflow):** Descreve o circuito por meio de equações que expressam a relação entre suas entradas e saídas, utilizando operadores lógicos e aritméticos em conjunto com a palavra-chave `assign`.

A seguir segue um exemplo de um mux simples de 4 bits na linguagem Verilog:

```
1 module mux41_verilog (
2     input [1:0] sel,
3     input a, b, c, d,
4     input clk, rst,
5     output reg y
6 );
7
8 // Sinal intermediario precisa ser reg porque recebe atribuicao no always
9 reg y_comb;
10
11 // Bloco Combinacional (Lista de sensibilidade generica)
12 always @(sel or a or b or c or d) begin
13     case (sel)
14         2'b00: y_comb = a;
15         2'b01: y_comb = b;
16         2'b10: y_comb = c;
17         2'b11: y_comb = d;
18         default: y_comb = 1'bx;
19     endcase
20 end
21
22 // Bloco Sequencial (Registrador de Saida)
23 always @(posedge clk or posedge rst) begin
```



```

24     if (rst)
25         y <= 1'b0;
26     else
27         y <= y_comb;
28     end
29
30 endmodule

```

Listing 3.1 – MUX 4:1 em Verilog Clássico

3.4.2 A Linguagem SystemVerilog

Para projetos de maior escala e complexidade, o SystemVerilog surge como uma extensão (superset) do Verilog tradicional. Ele incorpora características de linguagens de programação de alto nível, como orientação a objetos, e introduz um conjunto abrangente de ferramentas voltadas para a verificação funcional, incluindo geração de testes aleatórios restritos (constrained random testing), cobertura funcional (functional coverage) e metodologias de verificação como a Universal Verification Methodology (UVM). Embora este projeto utilize o Verilog padrão, o SystemVerilog representa o estado da arte para a verificação de circuitos integrados complexos.

No contexto deste trabalho, a linguagem Verilog é a escolhida para dar continuidade ao desenvolvimento do processador RISC-V, sendo a ponte entre a especificação arquitetural idealizada e a implementação real em hardware.

A seguir segue um exemplo do mesmo mux demonstrado anteriormente para a linguagem SystemVerilog:

```

1  module mux41_sv (
2      input  logic [1:0] sel,
3      input  logic      a, b, c, d,
4      input  logic      clk, rst,
5      output logic      y
6  );
7
8      // Nao precisa decidir entre reg/wire, usa-se apenas logic
9      logic y_comb;
10
11     // always_comb monta a lista de sensibilidade automaticamente
12     always_comb begin
13         unique case (sel)
14             2'b00: y_comb = a;
15             2'b01: y_comb = b;
16             2'b10: y_comb = c;
17             2'b11: y_comb = d;
18         endcase
19     end
20
21     // always_ff garante que o compilador valide se o circuito e sequencial
22     always_ff @(posedge clk or posedge rst) begin
23         if (rst) y <= '0; // Sintaxe '0 zera todos os bits automaticamente
24         else    y <= y_comb;
25     end

```

26

27 `endmodule`

Listing 3.2 – MUX 4:1 Simplificado em SystemVerilog

3.5 FPGA (Field-Programmable Gate Array)

Após a definição da arquitetura e a descrição do processador em uma HDL, a etapa seguinte consiste em materializar este projeto em um dispositivo físico capaz de executar o conjunto de instruções definido. Uma das plataformas mais adequadas para a prototipagem e implementação de sistemas digitais como este é o FPGA (Field-Programmable Gate Array).

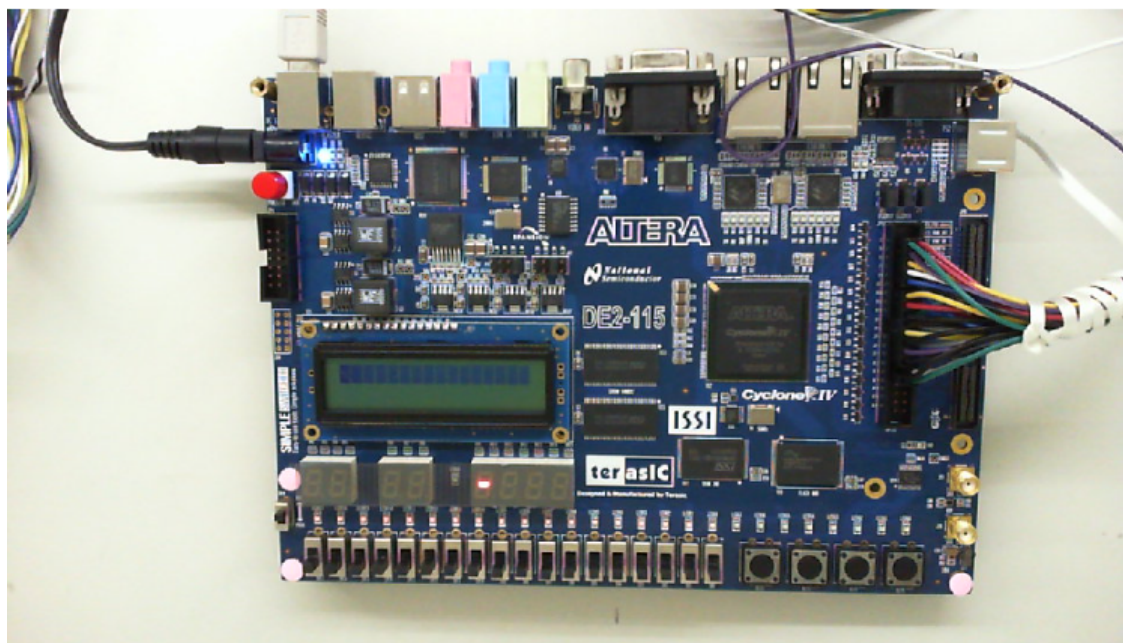
Um FPGA é um dispositivo semicondutor, cuja arquitetura se baseia em uma matriz regular de blocos lógicos programáveis (configuráveis para realizar funções lógicas simples ou complexas), circundada por uma matriz de interconexão programável que permite conectar estes blocos de forma flexível, e blocos de entrada e saída (I/O) para a comunicação com o mundo externo.

A principal característica que diferencia um FPGA de outros dispositivos lógicos programáveis (como CPLDs) é sua altíssima capacidade de integração, podendo conter desde alguns milhares até vários milhões de equivalentes a portas lógicas, e a natureza finamente granular de seus blocos lógicos. Enquanto um CPLD contém poucos blocos lógicos grandes, o FPGA contém uma vasta quantidade de blocos pequenos, oferecendo uma flexibilidade e densidade de integração muito superiores para circuitos complexos.

Os blocos de interconexão e roteamento são o ponto chave para a flexibilidade do FPGA. É através deles que o algoritmo de roteamento, parte das ferramentas de desenvolvimento do fabricante, define como as conexões entre os blocos lógicos serão estabelecidas para realizar o circuito projetado. Esta capacidade de reconfiguração é sua grande vantagem: o mesmo chip pode ser reprogramado inúmeras vezes para implementar funções de hardware completamente diferentes.

Relevância para o Projeto Para este trabalho, a implementação do processador RISC-V descrito em Verilog será sintetizada, mapeada e configurada em um FPGA. Esta abordagem permite verificar o funcionamento do processador em um cenário próximo ao real, executando programas de teste de forma acelerada. Assim como a adoção da arquitetura RISC-V garante a compatibilidade da descrição em HDL com um padrão aberto, a utilização de um FPGA garante a compatibilidade do projeto com uma plataforma de hardware flexível e reconfigurável, amplamente difundida tanto no meio acadêmico quanto no industrial para a prototipagem de sistemas digitais e sistemas embarcados.

Figura 6 – Field-Programmable Gate Array



Fonte: O Autor

4 Desenvolvimento

Neste capítulo, detalha-se o processo de concepção do processador, abrangendo desde as definições macroestruturais até a especificação técnica das instruções e a lógica de organização do hardware.

4.1 Características Gerais do Processador

O processador desenvolvido é um núcleo de processamento digital baseado na arquitetura RISC-V, implementando um subconjunto do conjunto de instruções RV32IM. As principais diretrizes técnicas que definem o sistema são:

1. **Largura de Dados:** 32 bits, operando com palavras (words) de quatro bytes.
2. **Modelo de Execução:** Monociclo (Ciclo Único), onde cada instrução é buscada, decodificada e executada em um único pulso de clock.
3. **Conjunto de Instruções (ISA):** Base inteira (I) com a extensão de multiplicação e divisão em hardware (M). Não está planejada a implementação das instruções *ebreak* e *ecall*, entretanto, caso necessário, isso pode mudar.
4. **Ordenação de Memória:** Padrão Little-endian, com endereçamento orientado a byte.
5. **Banco de Registradores:** Composto por 32 registradores de uso geral (x0 a x31), cada um com 32 bits de largura.
6. **Arquitetura de Memória:** Modelo Harvard, com a separação física entre a memória de instruções e a memória de dados para viabilizar o fluxo de dados em ciclo único.

4.2 Desenvolvimento da Arquitetura

A arquitetura RISC-V organiza suas instruções em formatos fixos de 32 bits para otimizar a decodificação em hardware ([RISC-V INTERNATIONAL, 2024](#)). Para este projeto, as instruções foram agrupadas em cinco categorias principais:

Tabela 1 – Tipos de instruções (formatos R, I, S, B, U)

31	25	24	20	19	15	14	12	11	7	6	0	
funct7		rs2		rs1		funct3		rd		opcode		Tipo R
imm[11:0]				rs1		funct3		rd		opcode		Tipo I
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		Tipo S
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		Tipo B
imm[31:12]										opcode		Tipo U

Fonte: O Autor

Tabela 2 – Formato da instrução jal (Tipo J)

31	3021	20	1912	117	60
imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
imediato = {imm[20], imm[10:1], imm[11], imm[19:12], 1'b0}					

Fonte: O Autor

4.2.1 Instruções do Tipo R

As instruções do Tipo R são projetadas para a execução de operações aritméticas e lógicas nas quais todos os operandos envolvidos residem internamente no banco de registradores. Consequentemente, esta categoria utiliza de forma exclusiva o modo de endereçamento por registrador, lendo os dados de duas fontes e gravando o resultado em um destino. Como exemplos práticos desta categoria implementados no projeto, destacam-se as operações lógicas e aritméticas fundamentais (como soma, subtração, operações lógicas and, or e xor), os deslocamentos de bits, as comparações de magnitude e, de modo integrado, as operações atreladas à Extensão M de hardware, compreendendo as funções de multiplicação e divisão em nível de silício.

Tabela 3 – Instruções do tipo R

Mnemônico	funct3	funct7	Opcode	Descrição	Notas
add	0x0	0x00	0110011	$rd = rs1 + rs2$	
sub	0x0	0x20	0110011	$rd = rs1 - rs2$	
xor	0x4	0x00	0110011	$rd = rs1 \wedge rs2$	
or	0x6	0x00	0110011	$rd = rs1 \mid rs2$	
and	0x7	0x00	0110011	$rd = rs1 \& rs2$	
sll	0x1	0x00	0110011	$rd = rs1 \ll rs2$	
srl	0x5	0x00	0110011	$rd = rs1 \gg rs2$	
sra	0x5	0x20	0110011	$rd = rs1 \gg rs2$	msb-extends
slt	0x2	0x00	0110011	$rd = (rs1 < rs2)?1:0$	
sltu	0x3	0x00	0110011	$rd = (rs1 < rs2)?1:0$	zero-extends
mul	0x0	0x01	0110011	$rd = (rs1 * rs2)[31:0]$	
mulh	0x1	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
mulsu	0x2	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
mulu	0x3	0x01	0110011	$rd = (rs1 * rs2)[63:32]$	
div	0x4	0x01	0110011	$rd = rs1 / rs2$	
divu	0x5	0x01	0110011	$rd = rs1 / rs2$	
rem	0x6	0x01	0110011	$rd = rs1 \% rs2$	
remu	0x7	0x01	0110011	$rd = rs1 \% rs2$	

Fonte: O Autor

É importante destacar o comportamento de funções que trabalham com valores *unsigned*, valores que não apresentam sinais e que possuem características específicas, relacionadas a como eles são tratados quando estendidos. Tais comportamentos são descritos na coluna "Notas". A descrição *msb-extends* diz que o bit mais significativo é repetido quando o valor original é estendido, já a descrição *zero-extends* diz que o valor é completado por zeros quando é estendido. Essa afirmação também vale para tipos posteriores de instrução.

4.2.2 Instruções do Tipo I

As instruções do Tipo I (Immediate) caracterizam-se por operar combinando um registrador de origem a um valor constante (imediato) de 12 bits embutido na própria instrução, o qual sofre extensão de sinal prévia à sua utilização. Este formato é versátil e emprega dois modos de endereçamento distintos a depender da operação: utiliza o endereçamento por imediato para a execução de instruções lógico-aritméticas com constantes (a exemplo da soma com imediato) e o endereçamento por deslocamento para as instruções voltadas à leitura da memória de dados, utilizando o imediato como um referencial de distância (como nas operações de carregamento de palavra, meia palavra e byte). Além disso, este formato atende às necessidades de saltos indiretos realizados por valores de registradores.

Tabela 4 – Instruções do tipo I

Mnemônico	funct3	imm	Opcode	Descrição	Notas
addi	0x0		0010011	$rd = rs1 + imm$	
xori	0x4		0010011	$rd = rs1 \hat{=} imm$	
ori	0x6		0010011	$rd = rs1 imm$	
andi	0x7		0010011	$rd = rs1 \& imm$	
slli	0x1	$imm[5:11]=0x00$	0010011	$rd = rs1 \ll imm[0:4]$	
srli	0x5	$imm[5:11]=0x00$	0010011	$rd = rs1 \gg imm[0:4]$	
srai	0x5	$imm[5:11]=0x20$	0010011	$rd = rs1 \gg imm[0:4]$	msb-extends
slti	0x2		0010011	$rd = (rs1 < imm)?1:0$	
sltiu	0x3		0010011	$rd = (rs1 < imm)?1:0$	zero-extends
lb	0x0		0000011	$rd = M[rs1+imm][0:7]$	
lh	0x1		0000011	$rd = M[rs1+imm][0:15]$	
lw	0x2		0000011	$rd = M[rs1+imm][0:31]$	
lbu	0x4		0000011	$rd = M[rs1+imm][0:7]$	zero-extends
lhu	0x5		0000011	$rd = M[rs1+imm][0:15]$	zero-extends
jalr	0x0		1100111	$rd = PC+4; PC = rs1 + imm$	

Fonte: O Autor

4.2.3 Instruções do Tipo S

Para as operações de escrita, empregam-se as instruções do Tipo S (Store), que representam o formato exclusivo para a transferência de dados do processador para a memória externa. Estas operações fazem uso estrito do modo de endereçamento por deslocamento, no qual o endereço de destino na memória é computado através da soma do valor contido em um registrador base com um deslocamento imediato de 12 bits. Diferentemente das categorias anteriores, o Tipo S não requer um registrador de destino para gravação interna, uma vez que o dado operado é enviado ao sistema de armazenamento. Exemplos desta implementação englobam as operações de salvamento de palavra inteira, frações de palavra e unidades de byte.

Tabela 5 – Instruções do tipo S

Mnemônico	funct3	Opcode	Descrição
sb	0x0	0100011	$M[rs1+imm][0:7] = rs2[0:7]$
sh	0x1	0100011	$M[rs1+imm][0:15] = rs2[0:15]$
sw	0x2	0100011	$M[rs1+imm][0:31] = rs2[0:31]$

Fonte: O Autor

4.2.4 Instruções do Tipo B

No que tange à manipulação do fluxo de controle condicional, o sistema adota as instruções do Tipo B (Branch). Estas instruções operam calcadas no modo de endereçamento PC-relativo e são responsáveis por realizar avaliações lógicas entre dois registradores. Caso a condição avaliada seja considerada verdadeira, o processador efetua o desvio do fluxo de execução para um novo endereço, o qual é calculado mediante a soma do

valor atual do Contador de Programa (PC) a um valor imediato especificado na instrução. A implementação funcional deste projeto contempla saltos condicionais baseados em relações de igualdade, desigualdade e comparações de magnitude relativas.

Tabela 6 – Instruções do tipo B

Mnemônico	funct3	Opcode	Descrição	Notas
beq	0x0	1100011	if(rs1 == rs2) PC += imm	
bne	0x1	1100011	if(rs1 != rs2) PC += imm	
blt	0x4	1100011	if(rs1 < rs2) PC += imm	
bge	0x5	1100011	if(rs1 >= rs2) PC += imm	
bltu	0x6	1100011	if(rs1 < rs2) PC += imm	zero-extends
bgeu	0x7	1100011	if(rs1 >= rs2) PC += imm	zero-extends

Fonte: O Autor

4.2.5 Instruções do Tipo U

As instruções do tipo U são utilizadas para carregar constantes de 20 bits na parte superior de um registrador (`lui`) ou somá-las ao PC (`auipc`). O imediato de 20 bits é extraído dos bits [31:12] da instrução e deslocado 12 bits à esquerda.

Tabela 7 – Instruções do tipo U

Mnemônico	Opcode	Descrição
lui	0110111	rd = imm << 12
auipc	0010111	rd = PC + (imm << 12)

4.2.6 Instruções do Tipo J

A instrução `jal` (Jump and Link) é do tipo J, com um imediato de 20 bits (mais um bit 0 implícito) que permite saltos de até 51 MiB. O formato do imediato é reconstruído a partir dos campos `inst[31]`, `inst[19:12]`, `inst[20]`, `inst[30:21]`, acrescido de um bit 0 no LSB.

Tabela 8 – Instrução do tipo J

Mnemônico	Opcode	Descrição
jal	1101111	rd = PC+4; PC = PC + imm

4.2.7 Instruções do Sistema

Além das instruções aritméticas, lógicas, de memória e de controle de fluxo, a arquitetura RISC-V define um conjunto de instruções do sistema, utilizadas para operações privilegiadas, chamadas ao sistema e depuração. Neste projeto, foi implementada a instrução `ebreak` (Environment Break), que tem como função interromper a execução

do programa e transferir o controle para um depurador ou, em nosso caso, simplesmente parar o processador.

A instrução **ebreak** possui opcode 0x73 (1110011) e é decodificada pela unidade de controle, que ativa o sinal **ebreak_c**. Este sinal é utilizado no módulo topo para interromper a atualização do PC, efetivamente parando o processador até que um reset seja aplicado.

Tabela 9 – Instrução do sistema implementada

Mnemonico	Opcode	Descrição
ebreak	1110011	Interrompe a execução (breakpoint)

Fonte: O Autor

A instrução **ebreak** é amplamente utilizada nos programas de teste para sinalizar o término bem-sucedido da computação, permitindo a verificação do resultado final escrito na memória de E/S.

4.3 Desenvolvimento da Organização

A organização do processador reflete a materialização física da arquitetura previamente definida, feita por meio da construção dos caminhos de dados de cada instrução, como da descrição do mesmo. Em cada componente funcional, há 4 campos:

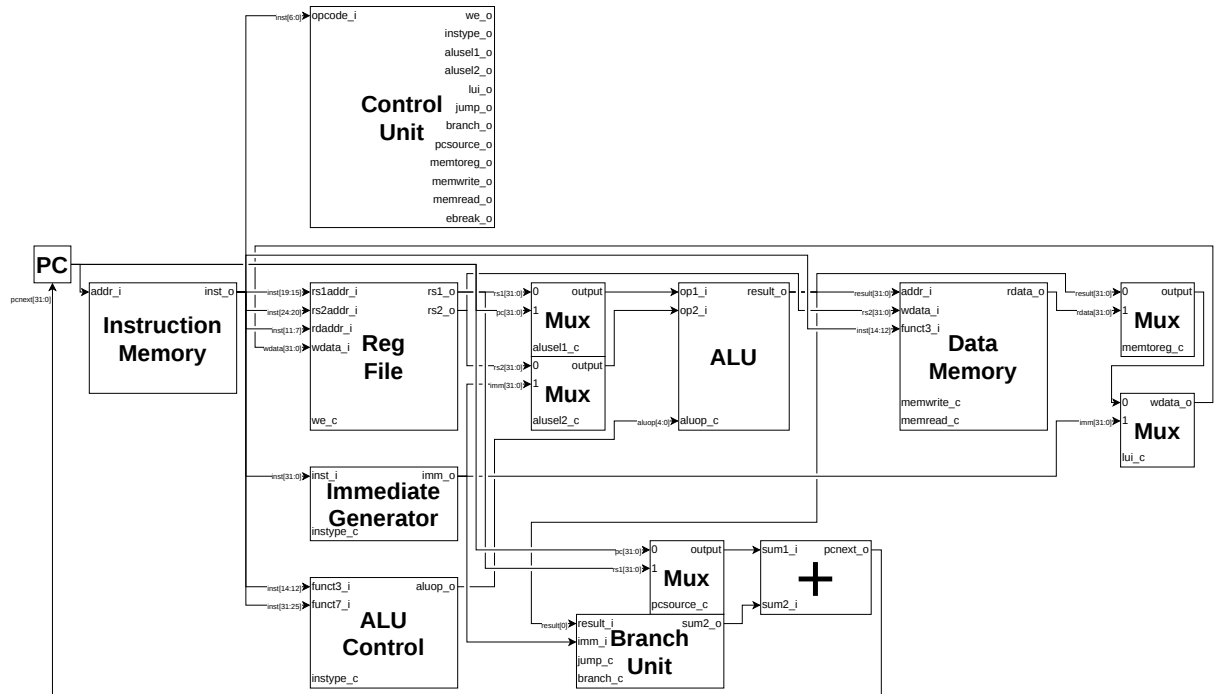
1. **O nome do componente funcional**, localizado no centro de cada caixa.
2. **As entradas gerais de cada componente funcional**, localizados no canto superior esquerdo de cada caixa e com nomes terminados em "**__i**".
3. **As entradas de controle de cada componente funcional**, localizadas no canto inferior esquerdo de cada caixa e com nomes terminados em "**__c**".
4. **As saídas de cada componente funcional**, localizadas na parte direita de cada caixa, com nomes terminados em "**__o**".

Vale ressaltar que para uma melhor compreensão de cada caminho de dados, foram omitidos os caminhos de controle com origem na unidade de controle (*Control Unit*), sendo representados por um texto em vermelho quando ativos, tanto na saída da unidade de controle quanto na entrada de controle dos componentes funcionais. Os caminhos são interligados por entradas e saídas com o mesmo nome antes do caractere "**__**".

Nos caminhos de dados de instruções específicas, além dos caminhos de controle, foram omitidos os caminhos não utilizados.

4.3.1 Caminho de Dados Completo

Figura 7 – Caminho de Dados Completo

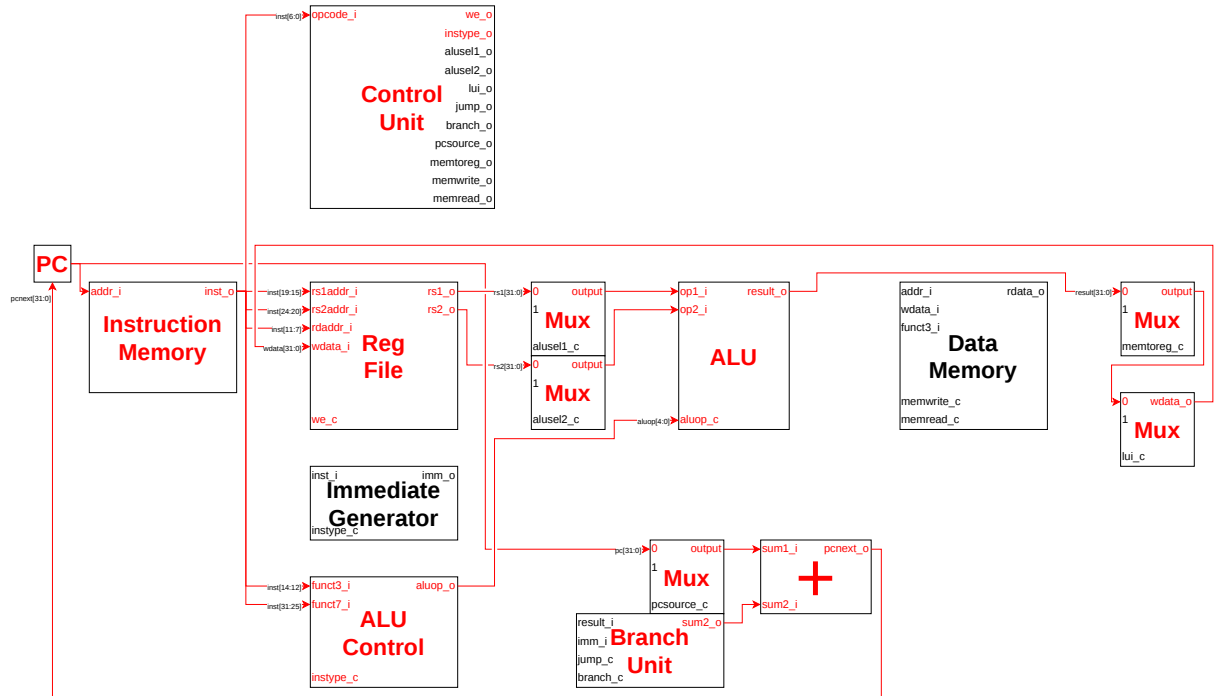


Fonte: O Autor

4.3.2 Caminho de Dados das Instruções do Tipo R

Para as instruções estritamente dependentes de registradores, o fluxo de dados inicia-se com a leitura simultânea dos operandes armazenados nos endereços apontados pelos campos `rs1` e `rs2` do Banco de Registradores. A Unidade de Controle sinaliza ao multiplexador da ALU que o segundo operando deve provir diretamente de `rs2`, ignorando qualquer valor imediato. A ALU executa a operação matemática ou lógica estipulada pela ALU Control, que define a operação com base nos campos originados da instrução e, em seguida, o resultado transita pelos multiplexadores de saída (ignorando a Memória de Dados) para ser gravado de volta no registrador de destino (`rd`).

Figura 8 – Caminho de Dados das Instruções do Tipo R

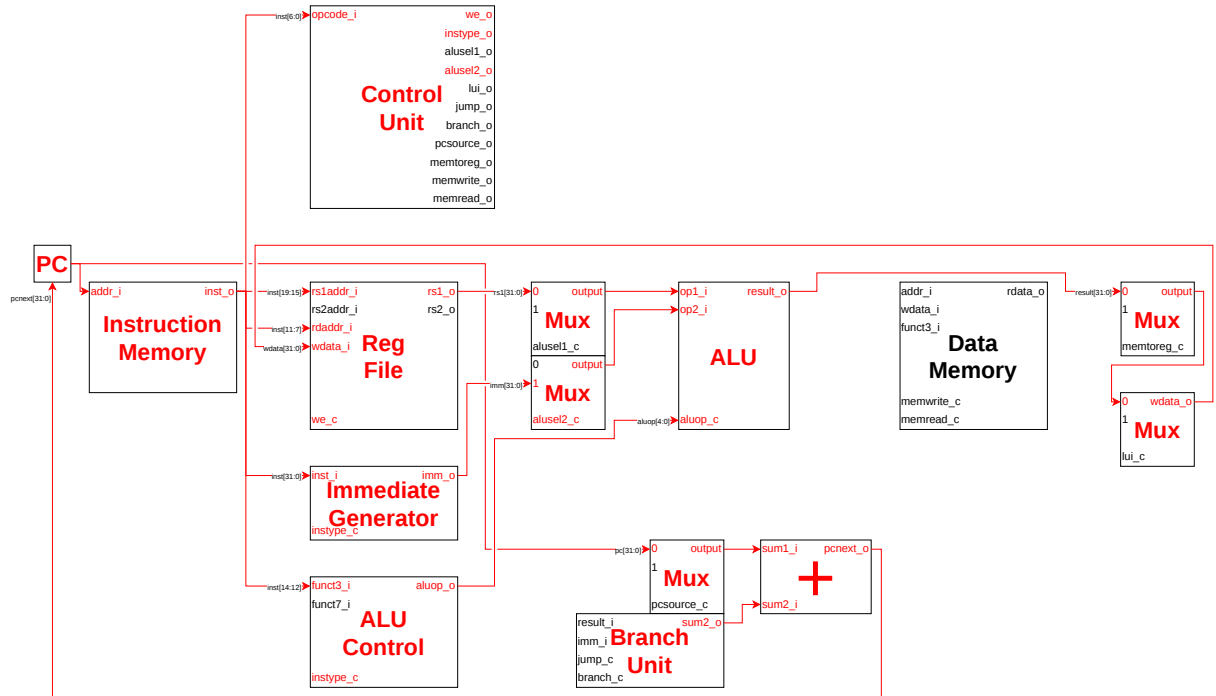


Fonte: O Autor

4.3.3 Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato (*addi, xori, ori, andi, slli, srli, srai, slti, sltiu*)

Neste agrupamento, o datapath opera lendo apenas um registrador de origem (*rs1*). Paralelamente, o Gerador de Imediatos extrai o valor constante de 12 bits embutido na instrução e realiza a extensão de sinal para 32 bits. O multiplexador de entrada da ALU é então comutado para selecionar este valor estendido como o segundo operando. Após o processamento da operação pela ALU, o resultado é diretamente encaminhado para a porta de escrita do Banco de Registradores, sendo salvo no registrador *rd*.

Figura 9 – Caminho de Dados das Instruções Aritméticas e Lógicas com Imediato



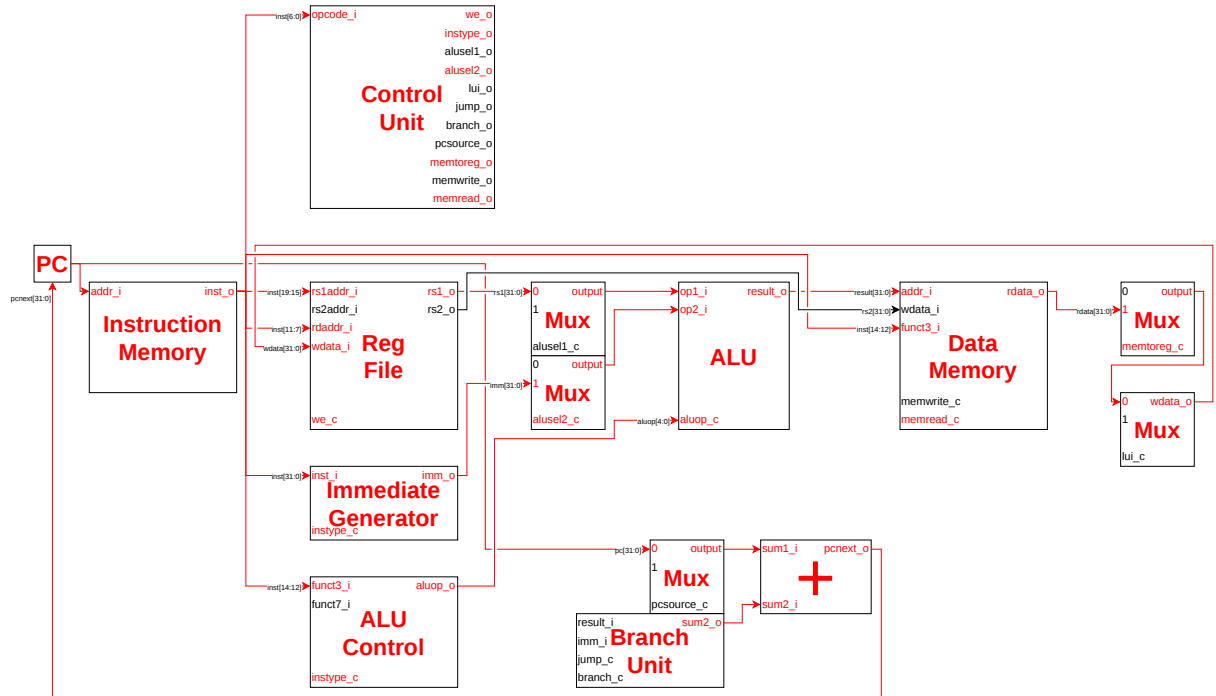
Fonte: O Autor

4.3.4 Caminho de Dados das Instruções de Carregamento (*lb*, *lh*, *lw*, *lbu*, *lhu*)

O fluxo de dados para a leitura em memória exige a utilização da ALU para o cálculo do endereço efetivo. O valor base é lido do registrador *rs1* e somado ao valor de deslocamento (imediato de 12 bits estendido). O resultado desta soma é enviado para a porta de endereço da Memória de Dados. Com o sinal de leitura ativado, a memória disponibiliza o dado requisitado. O multiplexador de write-back é então configurado para selecionar a saída da Memória de Dados, redirecionando o valor lido para ser armazenado no registrador de destino (*rd*).

É importante ressaltar que para as instruções sem bit de sinal *lbu* e *lhu*, a extensão do valor é realizada dentro da própria memória de dados. O reconhecimento de tais instruções é realizado de mesma forma do reconhecimento das demais instruções de carregamento, utilizando-se do campo *funct3* da instrução.

Figura 10 – Caminho de Dados das Instruções de Carregamento

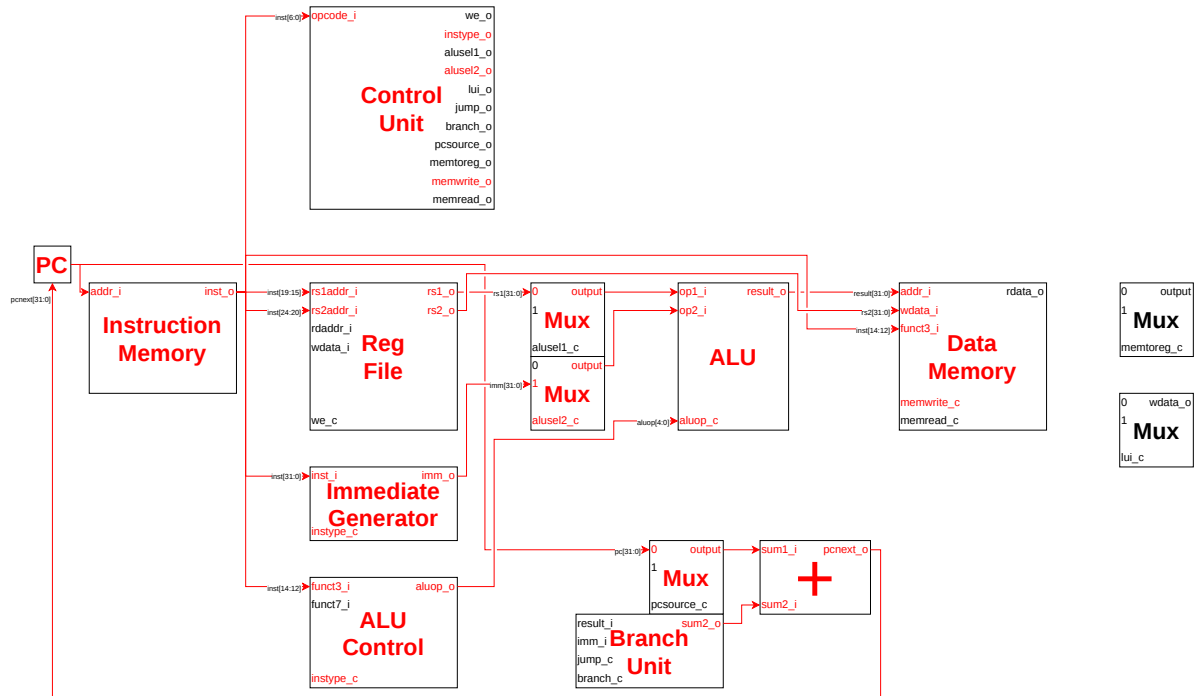


Fonte: O Autor

4.3.5 Caminho de Dados das Instruções de Armazenamento (*sb*, *sh*, *sw*)

Para as operações de escrita na memória, o datapath processa dois registradores simultaneamente: *rs1* fornece o endereço base e *rs2* fornece o dado a ser armazenado. A ALU computa o endereço final de destino somando o valor de *rs1* ao imediato estendido. O resultado da ALU é ligado à entrada de endereçamento da Memória de Dados, enquanto o valor lido de *rs2* é direcionado para a porta de dados de entrada da memória. O sinal de escrita em memória é habilitado e, como não há retorno de dados para o processador, o sinal de escrita no Banco de Registradores é desativado.

Figura 11 – Caminho de Dados das Instruções de Armazenamento

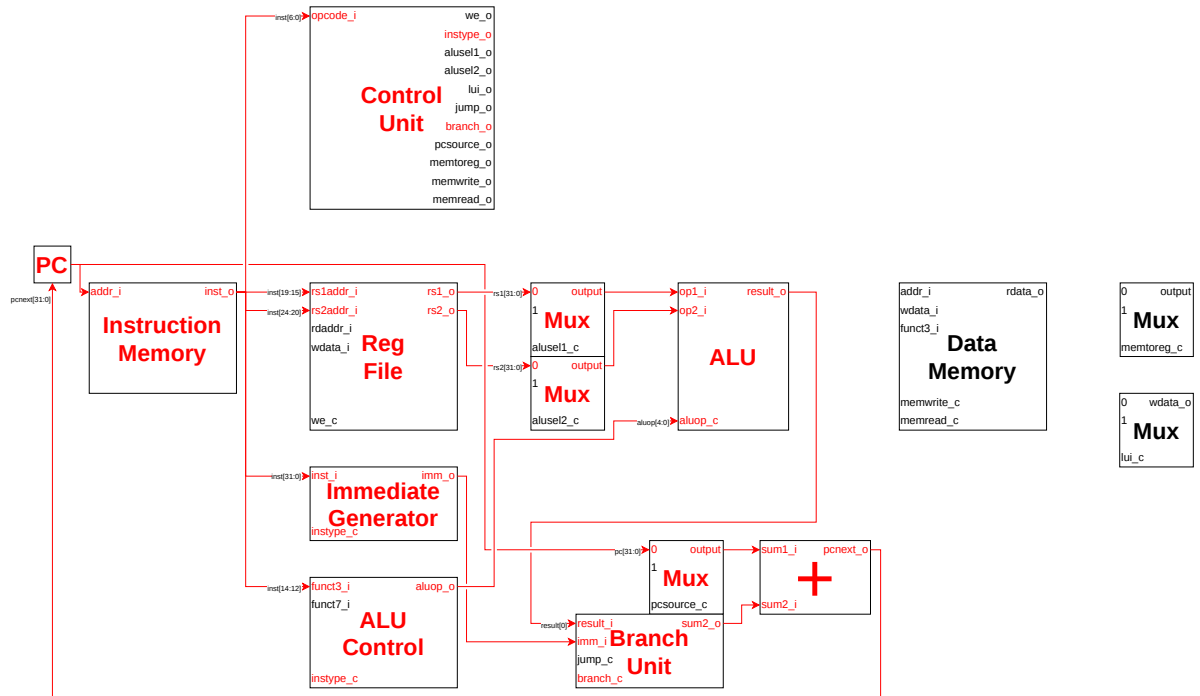


Fonte: O Autor

4.3.6 Caminho de Dados das Instruções de Desvio Condicional (*beq*, *bne*, *blt*, *bge*, *bltu*, *bgeu*)

Nas instruções de salto condicional, o caminho de dados bifurca-se em duas operações simultâneas. Na primeira via, os valores dos registradores *rs1* e *rs2* são enviados à ALU (ou a um comparador dedicado) para avaliar a condição lógica estipulada. Na segunda via, um somador independente calcula o endereço de destino do salto, adicionando o valor atual do Contador de Programa (PC) ao imediato de 12 bits estendido (e deslocado à esquerda). Se a condição avaliada pela ALU for verdadeira, a Unidade de Controle comuta o multiplexador do PC para que o próximo endereço de instrução seja o alvo calculado; caso contrário, o PC é atualizado normalmente para $PC + 4$. Não ocorre gravação no Banco de Registradores.

Figura 12 – Caminho de Dados das Instruções de Desvio Condicional

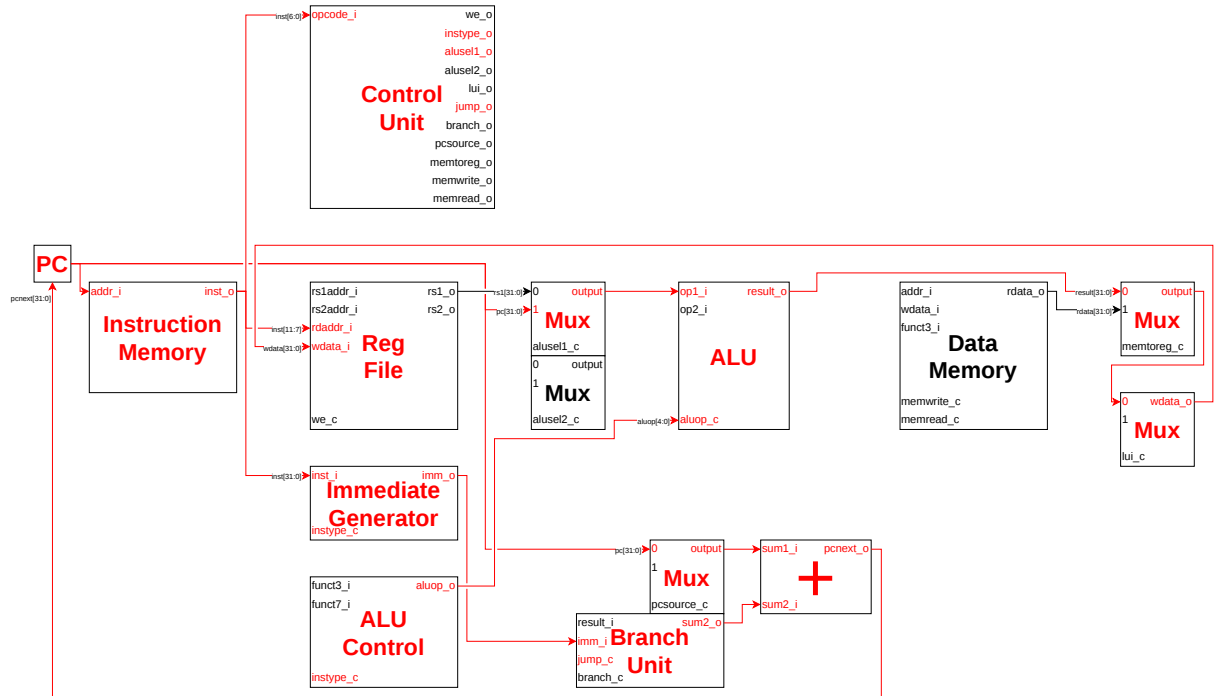


Fonte: O Autor

4.3.7 Caminho de Dados da Instrução de Salto e Ligação (*jal*)

O processamento da instrução *jal* requer o cálculo do endereço de salto e do endereço de retorno. O endereço de destino é obtido somando-se o valor atual do PC ao imediato de 20 bits estendido presente na instrução. Esse valor é injetado diretamente no PC. Concomitantemente, o endereço sequencial da próxima instrução ($PC + 4$) é calculado na ALU e enviado para a porta de escrita do Banco de Registradores, sendo salvo no registrador destino (*rd*, convencionalmente o registrador de retorno), permitindo a posterior retomada do fluxo original.

Figura 13 – Caminho de Dados da Instrução de Salto e Ligação

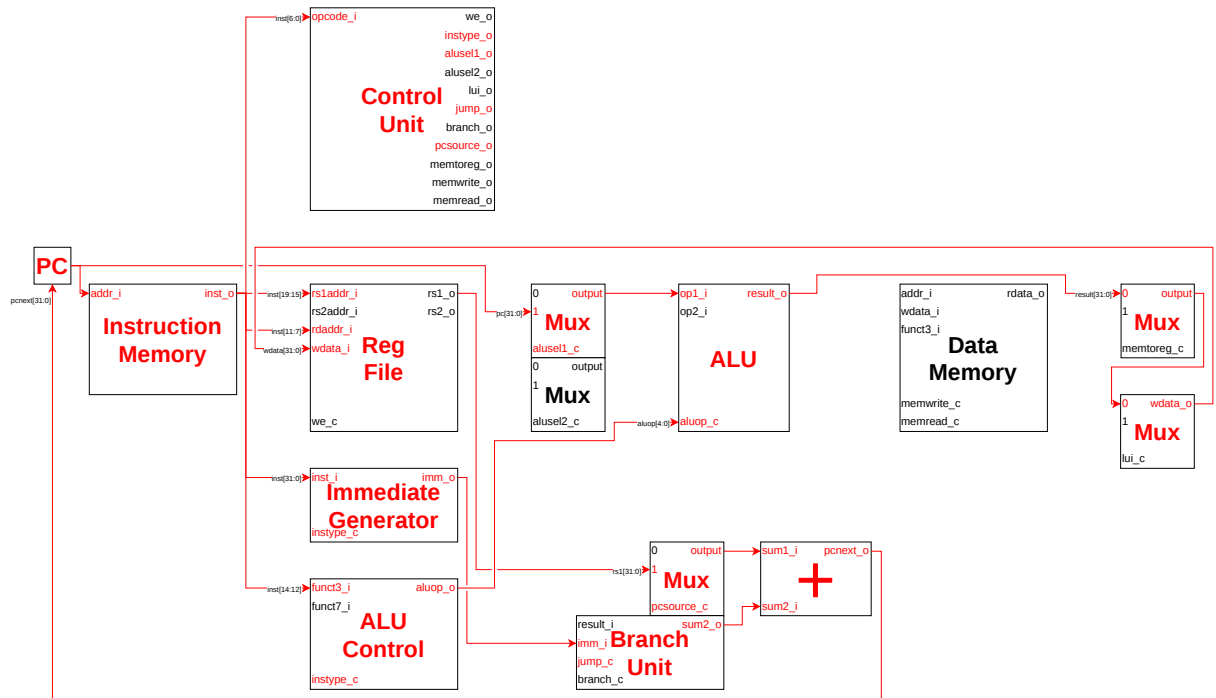


Fonte: O Autor

4.3.8 Caminho de Dados da Instrução de Salto Indireto (*jalr*)

A execução do *jalr* assemelha-se estruturalmente ao *jal*, porém com uma diferença crucial no cálculo do endereço de destino. Em vez de utilizar o PC como base, o endereço do salto é calculado pela ALU somando o valor lido do registrador *rs1* com o imediato de 12 bits com sinal estendido. O endereço de retorno ($PC + 4$) é preservado e gravado no registrador destino (*rd*), e o PC é atualizado com o novo endereço calculado, permitindo saltos dinâmicos baseados em dados armazenados.

Figura 14 – Caminho de Dados da Instrução de Salto Indireto

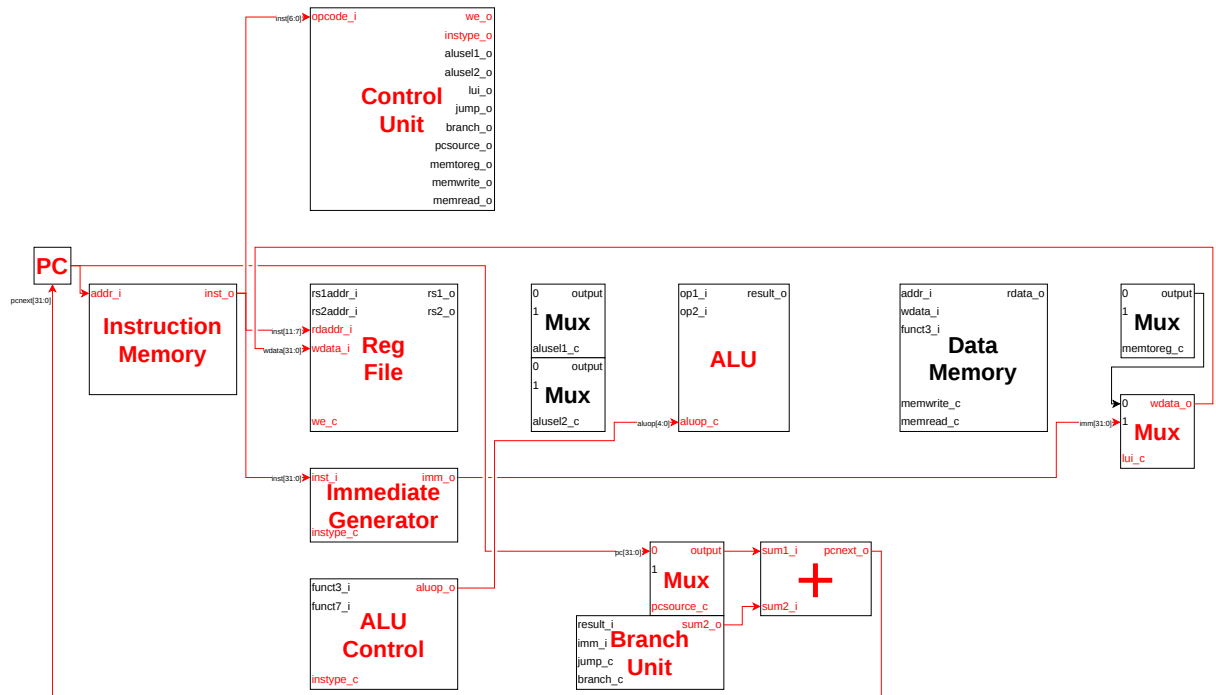


Fonte: O Autor

4.3.9 Caminho de Dados de Carregamento de Imediato Superior (*lui*)

Para a instrução *lui*, o datapath opera de maneira simplificada. O Gerador de Immediatos extrai os 20 bits superiores da instrução, preenche os 12 bits inferiores com zeros e envia este valor de 32 bits pelo caminho de dados e o resultado é imediatamente conduzido ao Banco de Registradores para ser gravado em *rd*. Não há acesso à memória de dados nem leitura de registradores de origem.

Figura 15 – Caminho de Dados de Carregamento de Imediato Superior

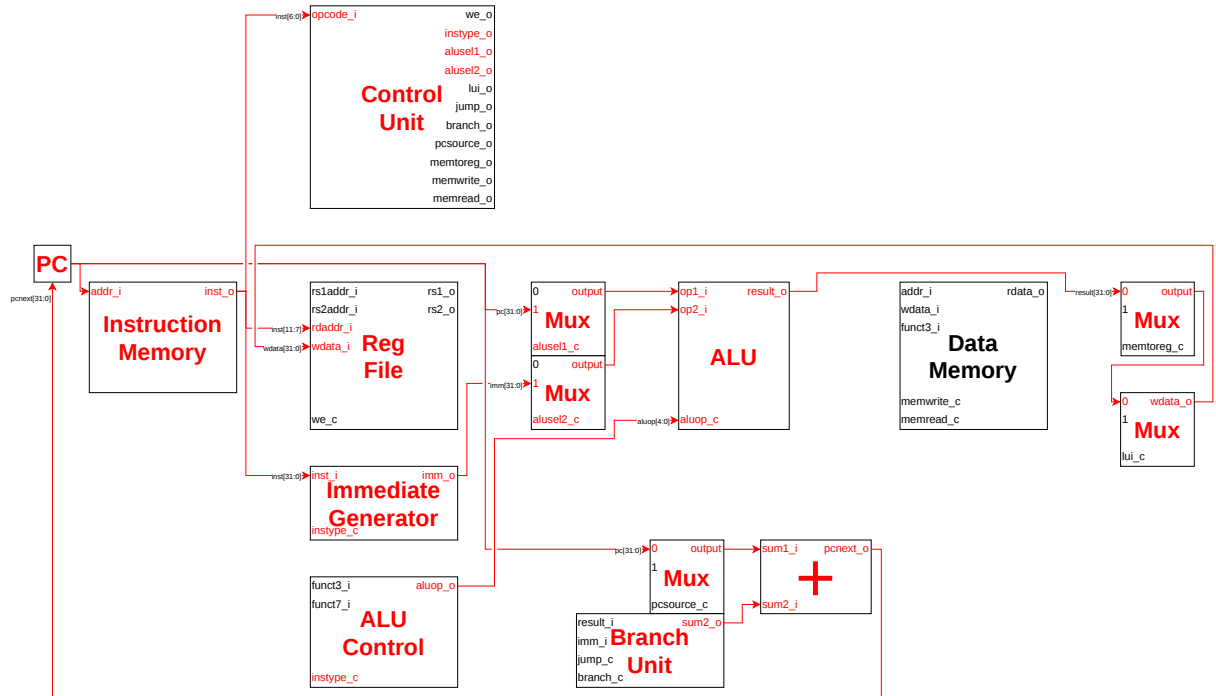


Fonte: O Autor

4.3.10 Caminho de Dados da Adição de Imediato Superior ao PC (*auipc*)

Por fim, o fluxo da instrução `auipc` combina o valor atual do PC com uma constante. O imediato de 20 bits é extraído e posicionado nas casas superiores de uma palavra de 32 bits (com os bits inferiores zerados). A ALU recebe em suas entradas o valor do PC e este imediato formatado, realizando a soma de ambos. O endereço resultante desta adição é então roteado diretamente para a porta de escrita do Banco de Registradores, sendo salvo em `rd` para utilização futura pelo programa.

Figura 16 – Caminho de Dados da Adição de Imediato Superior ao PC



Fonte: O Autor

4.3.11 Caminho de dados para Instruções do Sistema (ebreak)

A instrução **ebreak** (Environment Break) é uma instrução do sistema utilizada para sinalizar um ponto de interrupção durante a execução do programa. Diferentemente das instruções aritméticas, lógicas ou de controle de fluxo, o **ebreak** não produz um resultado computacional nem altera o estado do processador de forma convencional. Seu efeito é puramente de controle: interromper a execução e transferir o controle para um depurador ou, no caso deste projeto, simplesmente parar o processador.

O fluxo de dados para a instrução **ebreak** é o mais simples entre todas as instruções implementadas. A instrução é lida da memória de instruções e encaminhada à unidade de controle, que a decodifica pelo opcode 0x73 (1110011). A partir desse momento, nenhum outro bloco funcional é acionado:

- O banco de registradores não realiza leitura nem escrita.
- A ALU não recebe operandos nem executa operação.
- A memória de dados não é acessada.
- O gerador de imediatos e a unidade de branch permanecem inativos.


```

2  typedef logic [31:0] register_t;
3  typedef logic [31:0] instruction_t;
4  typedef logic [31:0] address_t;
5  typedef logic [31:0] data_t;
6
7  typedef logic [4:0] reg_addr_t;
8  typedef logic [6:0] funct7_t;
9  typedef logic [2:0] funct3_t;
10
11 typedef enum logic [2:0] {
12     OP = 3'b000,
13     OPI = 3'b001,
14     AUIPC = 3'b010,
15     BRANCH = 3'b011,
16     JUMP = 3'b100,
17     LOAD = 3'b101,
18     STORE = 3'b110
19 } inst_type_t;
20
21 typedef enum logic [6:0] {
22     opcode_OP = 7'b0110011,
23     opcode_OP_IMM = 7'b0010011,
24     opcode_LOAD = 7'b0000011,
25     opcode_STORE = 7'b0100011,
26     opcode_BRANCH = 7'b1100011,
27     opcode_JAL = 7'b1101111,
28     opcode_JALR = 7'b1100111,
29     opcode_LUI = 7'b0110111,
30     opcode_AUIPC = 7'b0010111,
31     opcode_EBREAK = 7'b1110011
32 } opcode_t;
33
34 typedef enum logic [4:0] {
35     ADD = 5'b00000,
36     SUB = 5'b00001,
37     XOR = 5'b00010,
38     OR = 5'b00011,
39     AND = 5'b00100,
40     SLL = 5'b00101,
41     SRL = 5'b00110,
42     SRA = 5'b00111,
43     SLT = 5'b01000,
44     SLTU = 5'b01001,
45     EQUAL = 5'b01010,
46     NEQUAL = 5'b01011,
47     GOET = 5'b01100,
48     GETU = 5'b01101,
49     PCP4 = 5'b01110,
50     MUL = 5'b01111,
51     MULH = 5'b10000,
52     MULHSU = 5'b10001,
53     MULHU = 5'b10010,
54     DIV = 5'b10011,
55     DIVU = 5'b10100,
56     REM = 5'b10101,
57     REMU = 5'b10110
58 } aluop_t;
59
60
61

```

62 `endpackage`Listing 4.1 – Package `data_structures.sv`

As enumerações `inst_type_t` e `aluop_t` são particularmente importantes, pois mapeiam os tipos de instrução e as operações da ULA a nomes simbólicos, tornando o código mais legível e próximo da especificação RISC-V.

4.5 Desenvolvimento da Unidade Lógica e Aritmética (ALU)

A Unidade Lógica e Aritmética é o componente central responsável por executar todas as operações matemáticas, lógicas e de comparação exigidas pelo conjunto de instruções do processador. Neste projeto, a ALU foi projetada para suportar não apenas as operações da base inteira (RV32I), mas também a extensão de multiplicação e divisão (RV32M), totalizando 24 operações distintas. O módulo foi implementado em Verilog (arquivo `alu.sv`) seguindo o modelo de lógica combinacional pura, com blocos `always_comb` para o controle e para a execução das operações.

4.5.1 Interface da ALU

A interface da ALU é definida pelo seguinte conjunto de portas (conforme trecho do código):

```
1 module alu (
2     input data_t rs1_i, rs2_i, imm_i, pc_i,
3     input funct3_t funct3_i,
4     input funct7_t funct7_i,
5     input inst_type_t inst_type_c,
6     input logic alusel1_c, alusel2_c,
7     output data_t aluout_o
8 );
```

Listing 4.2 – Interface da ALU

As entradas de dados (`rs1_i`, `rs2_i`, `imm_i`, `pc_i`) são os valores que podem ser selecionados como operandos da ALU. Os sinais de controle `alusel1_c` e `alusel2_c` permitem escolher, respectivamente, se o primeiro operando será `rs1_i` ou `pc_i` e se o segundo operando será `rs2_i` ou `imm_i`. O campo `inst_type_c` indica o tipo de instrução (OP, OPI, LOAD, STORE, BRANCH, JUMP, AUIPC), enquanto `funct3_i` e `funct7_i` são campos da instrução que refinam a operação. A saída `aluout_o` fornece o resultado calculado.

4.5.2 Decodificação da Operação Mapeamento para `aluop_t`

Para abstrair a variedade de combinações entre `inst_type`, `funct3` e `funct7`, foi criado um tipo enumerado `aluop_t` (definido no package `data_structures.sv`) com todos os códigos de operação suportados:

```

1 typedef enum logic [4:0] {
2     ADD, SUB, XOR, OR, AND, SLL, SRL, SRA, SLT, SLTU,
3     EQUAL, NEQUAL, GOET, GETU, PCP4,
4     MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU
5 } aluop_t;

```

Listing 4.3 – Decodificação da operação

O bloco ALUcontrol utiliza um casez para mapear cada combinação relevante ao respectivo aluop. Exemplos:

- Para instruções do tipo R aritméticas (OP):

```

1 {OP, 3'd0, 7'd0} : aluop = ADD;
2 {OP, 3'd0, 7'd32} : aluop = SUB;

```

- Para instruções de desvio condicional (BRANCH):

```

1 {BRANCH, 3'd0, 7'b???????} : aluop = EQUAL;
2 {BRANCH, 3'd4, 7'b???????} : aluop = SLT; // (para blt)

```

- Para multiplicação e divisão (extensão M):

```

1 {OP, 3'd0, 7'd1} : aluop = MUL;
2 {OP, 3'd4, 7'd1} : aluop = DIV;

```

Observe o uso de dont-cares (?) em campos que não influenciam a operação, como nos casos OPI onde o funct7 é irrelevante, exceto para deslocamentos com parte imediata.

4.5.3 Geração dos Operandos

Os dois operandos efetivos (op_1 e op_2) são gerados por um bloco combinacional que utiliza os sinais de seleção:

```

1 op_1 = (alusel1_c) ? pc_i : rs1_i;
2 op_2 = (alusel2_c) ? imm_i : rs2_i;

```

Essa lógica permite que a ALU atenda a todos os modos de endereçamento necessários:

- **Tipo R:** alusel1_c = 0, alusel2_c = 0 op_1 = rs1, op_2 = rs2.
- **Aritmética com imediato (OPI):** alusel1_c = 0, alusel2_c = 1 op_1 = rs1, op_2 = imm.
- **Cálculo de endereço para LOAD/STORE:** alusel1_c = 0, alusel2_c = 1 op_1 = rs1, op_2 = imm (ADD).
- **JAL (salto e ligação):** alusel1_c = 1, alusel2_c = 0 op_1 = pc, op_2 = rs2 (não usado), mas a operação PCP4 calcula pc + 4 independentemente.
- **AUIPC:** alusel1_c = 1, alusel2_c = 1 op_1 = pc, op_2 = imm, operação ADD.

4.5.4 Implementação das Operações

A execução propriamente dita ocorre em um terceiro bloco `always_comb` que contém um `case` (`aluop`). As principais operações são implementadas da seguinte forma:

- **Aritméticas e lógicas básicas:** ADD, SUB, XOR, OR, AND usam operadores do SystemVerilog diretamente.
- **Deslocamentos:** SLL, SRL e SRA consideram apenas os 5 bits menos significativos do segundo operando (`op_2[4:0]`), conforme especificação RISC-V. O deslocamento aritmético para a direita utiliza


```
1      $signed(op_1) >>> op_2[4:0].
```
- **Comparações com sinal e sem sinal:** SLT e SLTU produzem 1 no bit 0 do resultado se a condição for verdadeira.
- **Comparações para desvios:** EQUAL, NEQUAL, GOET (greater or equal), GETU (unsigned greater or equal) também produzem um bit de condição no LSB.
- **Operações de multiplicação e divisão:**
 - MUL produto de 64 bits truncado para os 32 bits inferiores.
 - MULH, MULHSU, MULHU respectivamente, multiplicação com sinal & sinal, sinal & sem sinal, sem sinal & sem sinal, retornando os 32 bits superiores.
 - DIV divisão com sinal, com tratamento especial para a divisão por zero (resultado = -1) e para o caso `0x80000000 / -1` (resultado = `0x80000000`).
 - DIVU divisão sem sinal, retornando `0xFFFFFFFF` para divisor zero.
 - REM e REMU resto da divisão.
- **Opção auxiliar PCP4:** utilizada pelas instruções `jal` e `jalr` para calcular o endereço de retorno (`pc + 4`).

Todos os cálculos que exigem largura de 64 bits utilizam o registrador temporário `tmp_reg` de 64 bits para armazenar o produto antes da seleção da parte desejada.

4.5.5 Desenvolvimento dos Testbenches

Para validar o funcionamento da ALU, foi elaborado um testbench específico (`tb.sv`) que instancia o módulo `alu` e aplica estímulos correspondentes a todas as classes de instruções suportadas. O testbench é modular e permite a ativação seletiva de cada grupo de testes, facilitando a depuração e a verificação incremental.

4.5.5.1 Estrutura do Testbench

O testbench declara os mesmos sinais da interface da ALU e os conecta ao módulo sob teste (dut). Um bloco inicial é responsável por:

1. Inicializar todos os sinais com valores padrão.
2. Executar, em sequência, os conjuntos de testes correspondentes a:
 - Instruções do tipo R aritméticas (ADD, SUB, MUL, DIV, etc.)
 - Instruções do tipo R de deslocamento (SLL, SRL, SRA)
 - Instruções do tipo R lógicas (XOR, OR, AND, SLT, SLTU)
 - Instruções do tipo I (ADDI, XORI, ORI, ANDI, SLLI, SRLI, SRAI, SLTI, SLTIU)
 - Instruções de LOAD e STORE (cálculo de endereço)
 - Instruções de desvio condicional (BEQ, BNE, BLT, BGE, BLTU, BGEU)
 - Instruções de salto e ligação (JAL, JALR) e AUIPC

Cada teste configura os sinais de entrada adequados e aguarda um intervalo de 10 unidades de tempo (#10) para que a ALU compute o resultado. Ao final, uma chamada \$stop pausa a simulação, permitindo a análise das formas de onda.

4.5.5.2 Organização dos Estímulos

Para manter a clareza, os testes foram agrupados e podem ser ativados/desativados através de comentários no código. Por exemplo, o trecho para as instruções do tipo R aritméticas é:

```

1 inst_type_c = OP;
2 alusel1_c = 0;
3 alusel2_c = 0;
4
5 // ADD
6 rs1_i = 32'd15; rs2_i = 32'd25; funct3_i = 3'd0; funct7_i = 7'd0; #10;
7 // SUB
8 rs1_i = 32'd50; rs2_i = 32'd20; funct3_i = 3'd0; funct7_i = 7'd32; #10;
9 // MUL
10 rs1_i = 32'd1000000; rs2_i = 32'd1000000; funct3_i = 3'd0; funct7_i = 7'd1; #10;
11 // ... etc.
```

Listing 4.4 – Exemplo do trecho do código de instruções do tipo R aritméticas no TestBench

Para as instruções do tipo I, além de configurar `inst_type_c = OPI` e `alusel2_c = 1`, o campo `imm_i` é utilizado:

```

1 // ADDI
2 rs1_i = 32'd100; imm_i = 32'd75; funct3_i = 3'd0; #10;
3 // SRAI
4 rs1_i = -32'd1024; imm_i = 32'd1; funct3_i = 3'd5; funct7_i = 7'd32; #10;

```

Listing 4.5 – Exemplo do trecho do código de instruções do tipo I aritméticas no TestBench

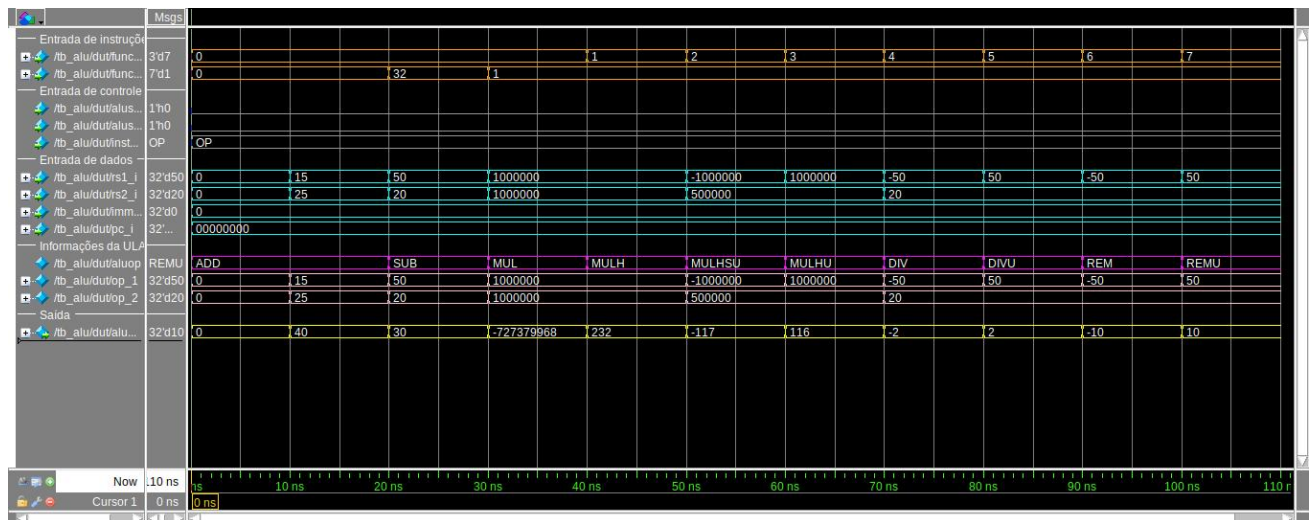
Nos testes de desvio condicional, a saída da ALU fornece um bit de condição no LSB (`aluout_o[0]`), que é posteriormente usado pela unidade de controle para decidir o salto. O testbench apenas verifica a geração correta desse bit.

4.5.6 Resultados da Simulação da ALU

4.5.6.1 Instruções do tipo R

4.5.6.1.1 Aritméticas

Figura 18 – Waveform das instruções do tipo R aritméticas (ADD, SUB, MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU)



Fonte: O Autor

Observase na figura que o testbench configura inicialmente `inst_type_c = OP`, `alusel1_c = 0` e `alusel2_c = 0`, selecionando a operação entre dois registradores. Os sinais `funct3_i` e `funct7_i` variam a cada intervalo de 10 ns para executar diferentes operações.

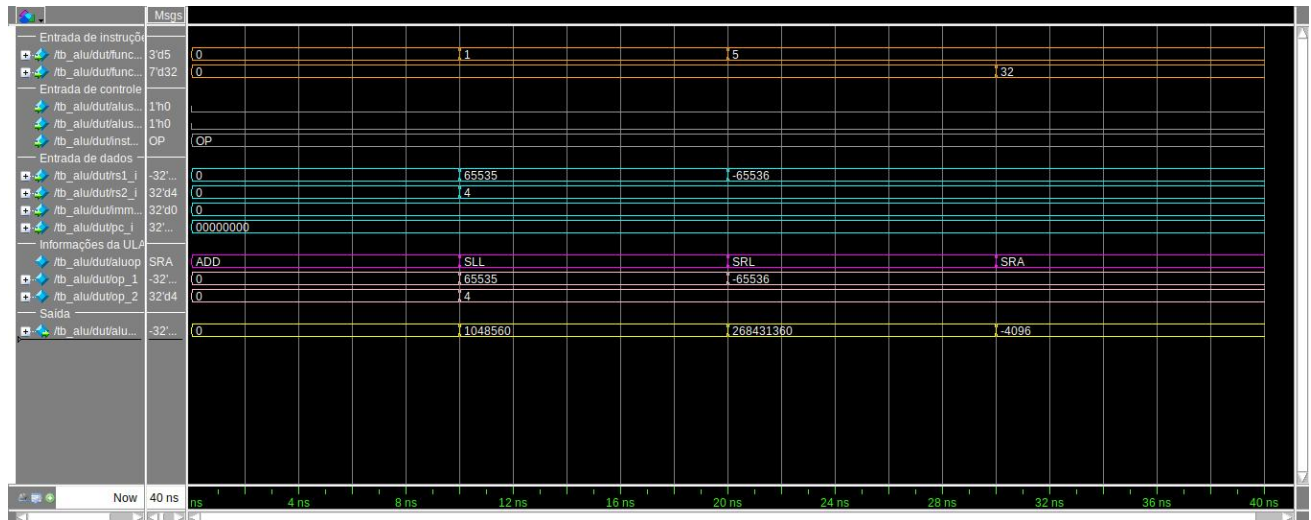
- **ADD** (`funct3=0`, `funct7=0`): `rs1_i = 15`, `rs2_i = 25`. A ALU decodifica `aluop = ADD` e a saída `aluout_o` tornase **40** (decimal), conforme esperado.
- **SUB** (`funct3=0`, `funct7=32`): `rs1_i = 50`, `rs2_i = 20`. `aluop = SUB` e `aluout_o = 30`.

- **MUL** (funct3=0, funct7=1): rs1_i = 1.000.000, rs2_i = 1.000.000. O resultado da multiplicação de 32 bits é a parte baixa do produto de 64 bits. O waveform mostra aluout_o = 0x4A5C1000 (valor decimal $1.000.000^2 \bmod 2^{32}$). A operação MUL está correta.
- **MULH** (funct3=1, funct7=1): Mesmos operandos. aluout_o assume o valor 0xE8D4A510 (parte alta do produto com sinal). O waveform indica que a ALU seleciona tmp_reg[63:32] corretamente.
- **MULHSU** (funct3=2, funct7=1): rs1_i = -1.000.000 (complemento de dois), rs2_i = 500.000. A saída corresponde aos 32 bits superiores do produto de um número com sinal por um sem sinal. O valor exibido está de acordo com a simulação.
- **MULHU** (funct3=3, funct7=1): rs1_i = 1.000.000, rs2_i = 500.000. Produto sem sinal, parte alta. Verificado.
- **DIV** (funct3=4, funct7=1): rs1_i = -50, rs2_i = 20. aluout_o = -2 (quociente com sinal). Na waveform, o valor aparece como -32'd2 (ou 0xFFFFFFF2).
- **DIVU** (funct3=5, funct7=1): rs1_i = 50, rs2_i = 20. aluout_o = 2 (divisão sem sinal).
- **REM** (funct3=6, funct7=1): rs1_i = -50, rs2_i = 20. Resto = -10. O waveform confirma aluout_o = -10.
- **REMU** (funct3=7, funct7=1): rs1_i = 50, rs2_i = 20. Resto = 10.

A ALU executa corretamente todas as operações aritméticas da extensão M (multiplicação e divisão) além das básicas ADD e SUB.

4.5.6.1.2 Shift

Figura 19 – Waveform das instruções de deslocamento (SLL, SRL, SRA)



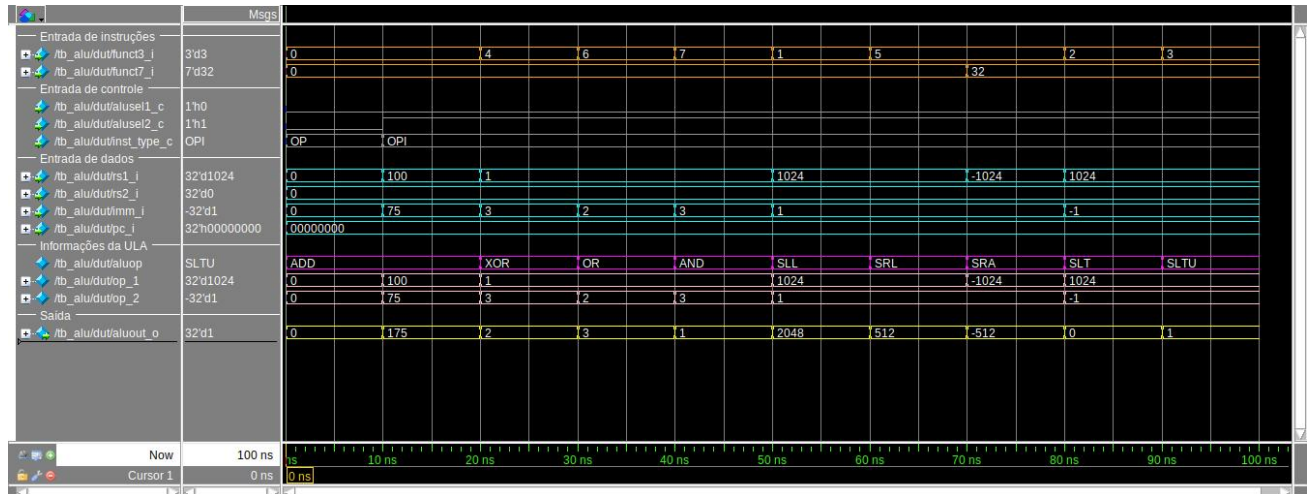
Fonte: O Autor

- **SLL** (funct3=1, funct7=0): rs1_i = 32'h0000FFFF, rs2_i = 4. Operação shift left lógico: $0x0000FFFF \ll 4 = 0x000FFFF0$. O waveform mostra aluout_o = 0x000FFFF0.
- **SRL** (funct3=5, funct7=0): rs1_i = 32'hFFFF0000, rs2_i = 4. Shift right lógico: $0xFFFF0000 \gg 4 = 0x0FFFF000$. A saída confirma esse valor.
- **SRA** (funct3=5, funct7=32): O mesmo rs1_i = 0xFFFF0000 (valor negativo em complemento de dois) deslocado aritmeticamente à direita por 4 posições. O bit de sinal (MSB = 1) é replicado, resultando em 0xFFFFF000. O waveform exibe exatamente 0xFFFFF000.

A ALU trata corretamente os deslocamentos lógicos e aritméticos, considerando apenas os 5 bits inferiores do segundo operando.

4.5.6.2 Instruções do Tipo I (Aritmética com Imediato)

Figura 21 – Waveform das instruções do tipo I (ADDI, XORI, ORI, ANDI, SLLI, SRLI, SRAI, SLTI, SLTIU).



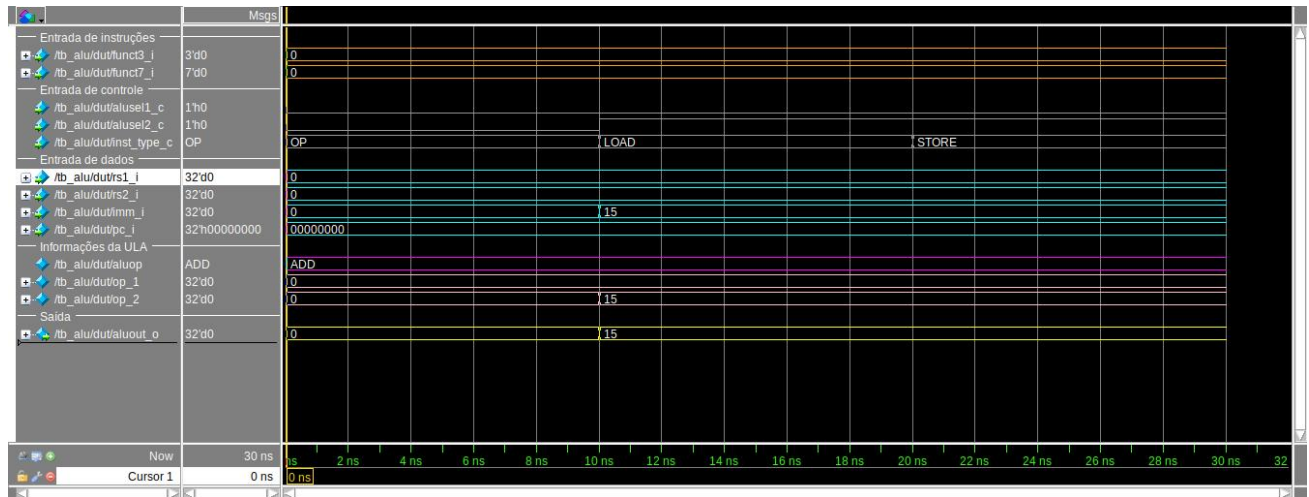
Fonte: O Autor

- **ADDI:** `rs1_i = 100`, `imm_i = 75` `aluout_o = 175`.
- **XORI:** `rs1_i = 1`, `imm_i = 3` `1 XOR 3 = 2`. Saída = 2.
- **ORI:** `rs1_i = 1`, `imm_i = 2` `1 OR 2 = 3`.
- **ANDI:** `rs1_i = 1`, `imm_i = 3` `1 AND 3 = 1`.
- **SLLI:** `rs1_i = 1024`, `imm_i = 1` (shift left por 1) 2048. O campo `funct7` é 0. Saída = 2048.
- **SRLI:** `rs1_i = 1024`, `imm_i = 1` (shift right lógico) 512. Saída = 512.
- **SRAI:** `rs1_i = -1024` (em complemento de dois), `imm_i = 1` deslocamento aritmético à direita -512. O waveform mostra `aluout_o = -512` (ou `0xFFFFFE00`).
- **SLTI:** `rs1_i = 1024`, `imm_i = -1`. Comparação com sinal: `1024 < -1`? Falso `aluout_o[0] = 0`.
- **SLTIU:** `rs1_i = 1024`, `imm_i = -1`. Agora -1 é interpretado como um número sem sinal (`0xFFFFFFFF`). `1024 < 0xFFFFFFFF` verdadeiro `aluout_o[0] = 1`.

A ALU decodifica corretamente as instruções de imediato, inclusive com a extensão de sinal do campo `imm_i` e o tratamento especial para deslocamentos (que usam `funct7` para distinguir SRLI de SRAI).

4.5.6.3 Instruções de Load e Store

Figura 22 – Waveform do cálculo de endereço para LOAD/STORE.



Fonte: O Autor

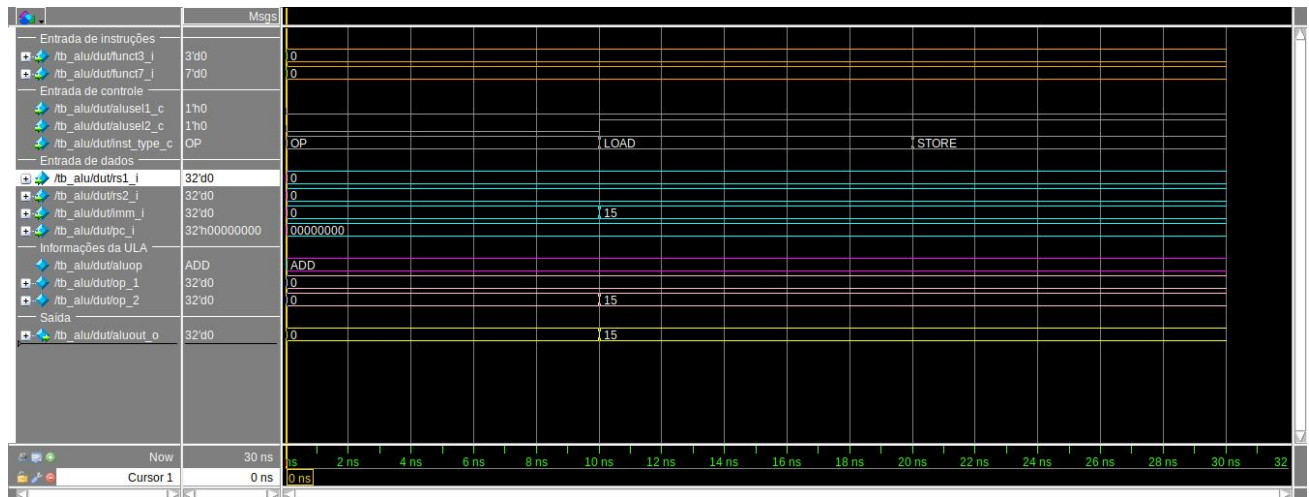
Para estas instruções, `inst_type_c` = LOAD (ou STORE), `aluse11_c` = 0, `aluse12_c` = 1, e `aluop` é fixado em ADD (pois o endereço efetivo é `rs1 + imm`). Na figura:

- `rs1_i` = 0x00000000, `imm_i` = 0x0000000F. A ALU realiza a soma: $0 + 15 = 15$. O sinal `aluout_o` tornase **15** (decimal). Esse valor é o endereço calculado que será usado pela memória de dados.
- O testbench também testa o caso STORE (mesmo comportamento, pois a operação da ALU é idêntica: cálculo de endereço). A saída permanece consistente.

A ALU gera corretamente o endereço de acesso para as operações de carregamento e armazenamento, utilizando o modo de endereçamento por deslocamento.

4.5.6.4 Instruções de Desvio Condicional

Figura 23 – Waveform das instruções de desvio condicional (BEQ, BNE, BLT, BGE, BLTU, BGEU).



Fonte: O Autor

A Figura mostra a simulação das instruções de desvio. Neste modo, a ALU é configurada com `inst_type_c = BRANCH`, `alusel1_c = 0` e `alusel2_c = 0`, de modo que ambos os operandos são lidos do banco de registradores (`op_1 = rs1_i`, `op_2 = rs2_i`). O campo `funct3_i` determina a condição a ser avaliada. A ALU não produz um resultado aritmético completo, mas sim um **bit de condição** no LSB de `aluout_o` (os demais bits são zero). O waveform apresenta os seguintes casos:

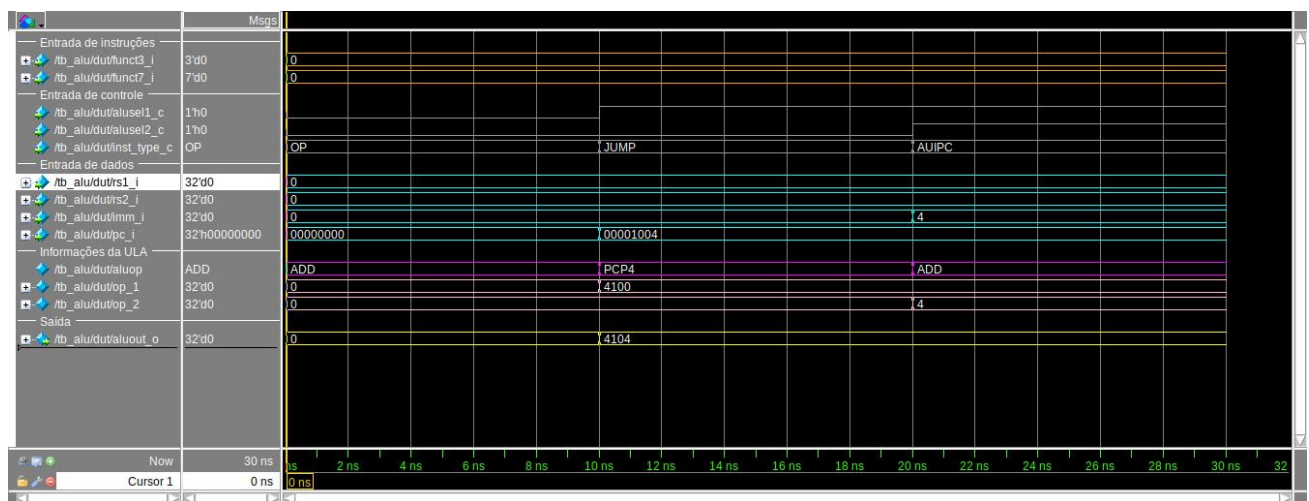
- **BEQ** (`funct3=0`): `rs1_i = 50`, `rs2_i = 50` condição verdadeira `aluout_o[0] = 1`. Em seguida, `rs2_i = 51` falso `aluout_o[0] = 0`.
- **BNE** (`funct3=1`): `rs1_i = 50`, `rs2_i = 50` falso (não é diferente) `0`. `rs2_i = 51` verdadeiro `1`.
- **BLT** (`funct3=4`): `rs1_i = 50`, `rs2_i = -50` `50 < -50?` Falso `0`. `rs2_i = 51` `50 < 51?` Verdadeiro `1`.
- **BGE** (`funct3=5`): `rs1_i = 50`, `rs2_i = -50` `50 >= -50?` Verdadeiro `1`. `rs2_i = 51` `50 >= 51?` Falso `0`.
- **BLTU** (`funct3=6`): `rs1_i = 50`, `rs2_i = 50` `50 < 50` (sem sinal) Falso `0`. `rs2_i = 51` `50 < 51` Verdadeiro `1`.
- **BGEU** (`funct3=7`): `rs1_i = 50`, `rs2_i = 50` `50 >= 50` Verdadeiro `1`. `rs2_i = 51` `50 >= 51` Falso `0`. `rs2_i = -51` (interpretado como sem sinal = `0xFFFFFCD`) `50 >= 0xFFFFFCD?` Falso `0`.

Todos os bits de condição gerados pela ALU estão corretos de acordo com a especificação RISC-V. A saída `aluout_o` é subsequentemente utilizada pela unidade de controle para decidir se o PC deve ser atualizado com o endereço de desvio ou com `PC+4`.

A ALU avalia corretamente as condições para todos os desvios condicionais implementados, tanto para operandos com sinal quanto sem sinal.

4.5.6.5 Instruções de Salto (JAL, JALR) e AUIPC

Figura 24 – Waveform das instruções JAL, JALR e AUIPC.



Fonte: O Autor

- **JAL:** O testbench configura `inst_type_c = JUMP`, `alusel1_c = 1`, e `aluop = PCP4`. A entrada `pc_i = 0x00001004`. A saída `aluout_o` torna-se `pc_i + 4 = 0x00001008` (ou 4100 decimal, conforme a figura). Este é o endereço de retorno que será armazenado em `rd`.
- **JALR:** (não aparece explicitamente na figura, mas o testbench a inclui). A operação é semelhante, mas o cálculo do destino do salto é feito fora da ALU (no próprio módulo do processador). A ALU apenas fornece `pc+4` para o registrador de retorno.
- **AUIPC:** `inst_type_c = AUIPC`, `alusel1_c = 1`, `alusel2_c = 1` (soma PC + imediato). `pc_i = 0x00001004`, `imm_i = 4`. O imediato é deslocado 12 bits à esquerda na unidade de controle, mas a ALU apenas soma os valores recebidos. Pela figura, o resultado é 0x5004, indicando que o imediato foi tratado como 0x4000 ($4 \ll 12$) antes de chegar à ALU. Portanto, a ALU soma $0x1004 + 0x4000 = 0x5004$, que é o valor exibido. A saída está correta.

A ALU suporta as operações especiais PCP4 (para JAL/JALR) e a soma para AUIPC, gerando os valores esperados.

4.6 Banco de Registradores

O banco de registradores é um dos componentes centrais do processador, responsável por armazenar os 32 registradores de uso geral (x0 a x31), sendo que o registrador x0 é permanentemente ligado em zero, conforme a especificação RISC-V. O módulo foi implementado em SystemVerilog no arquivo `regfile.sv`.

4.6.1 Implementação do Módulo `regfile`

```
1  import data_structures::*;
2
3  module regfile (
4      input logic we_c, clk_c,
5      input data_t wdata_i,
6      input logic [4:0] rdaddr_i, rs1addr_i, rs2addr_i,
7      output register_t rs1_o, rs2_o
8  );
9      register_t x[31]; // 31 registradores (x1 a x31)
10
11     // Escrita sincrona (subida do clock)
12     always_ff @(posedge clk_c) begin : reg_write
13         if (we_c && rdaddr_i != 0)
14             x[rdaddr_i - 1] <= wdata_i;
15     end
16
17     // Leitura combinacional (com x0 fixo em 0)
18     assign rs1_o = (rs1addr_i == 0) ? 0 : x[rs1addr_i - 1];
19     assign rs2_o = (rs2addr_i == 0) ? 0 : x[rs2addr_i - 1];
20 endmodule
```

Listing 4.6 – Módulo `regfile.sv`

Observações sobre a implementação:

- O array `x` possui 31 elementos, correspondentes aos registradores `x1` a `x31`. O registrador `x0` é implementado exclusivamente pela lógica de leitura, que retorna zero quando o endereço é 0.
- A escrita ocorre apenas na borda de subida do clock (`posedge clk_c`) e quando a habilitação de escrita `we_c` está ativa e o endereço de destino é diferente de zero.
- O índice do array é calculado como `rdaddr_i - 1` (e similar para leitura), de modo que `rdaddr_i = 1` acessa `x[0]`, e `rdaddr_i = 31` acessa `x[30]`. Essa decisão de projeto economiza um elemento de memória, mas exige cuidado na indexação.
- As leituras são combinacionais, permitindo que os valores estejam disponíveis no mesmo ciclo em que os endereços são fornecidos, característica essencial para o processador monociclo.

4.6.2 Testbench do Banco de Registradores

O testbench `tb_regfile.sv` instancia o módulo `regfile` e aplica uma sequência de escrita e leitura em todos os registradores. O clock é gerado manualmente por meio de alternâncias de `clk_c` com atrasos de 10 unidades de tempo.

```
1 import data_structures::*;
2
3 module tb_regfile ();
4     logic clk_c, we_c;
5     data_t wdata_i;
6     logic [4:0] rdaddr_i, rs1addr_i, rs2addr_i;
7     register_t rs1_o, rs2_o;
8     data_t i;
9
10    regfile dut (.*);
11
12    initial begin
13        $display("Inicializando...");
14        we_c = 1;
15        clk_c = 0;
16        rs1addr_i = 0;
17        rs2addr_i = 0;
18        rdaddr_i = 0;
19
20        for (i = 0; i < 32; i += 1) begin
21            wdata_i = i * 4;
22            rdaddr_i = i;
23            rs1addr_i = i - 1;
24            rs2addr_i = i;
25            clk_c = 1;
26            #10;
27            clk_c = 0;
28            #10;
29        end
30
31        #10;
32        $stop;
33    end
34 endmodule
```

Listing 4.7 – Testbench `tb_regfile.sv`

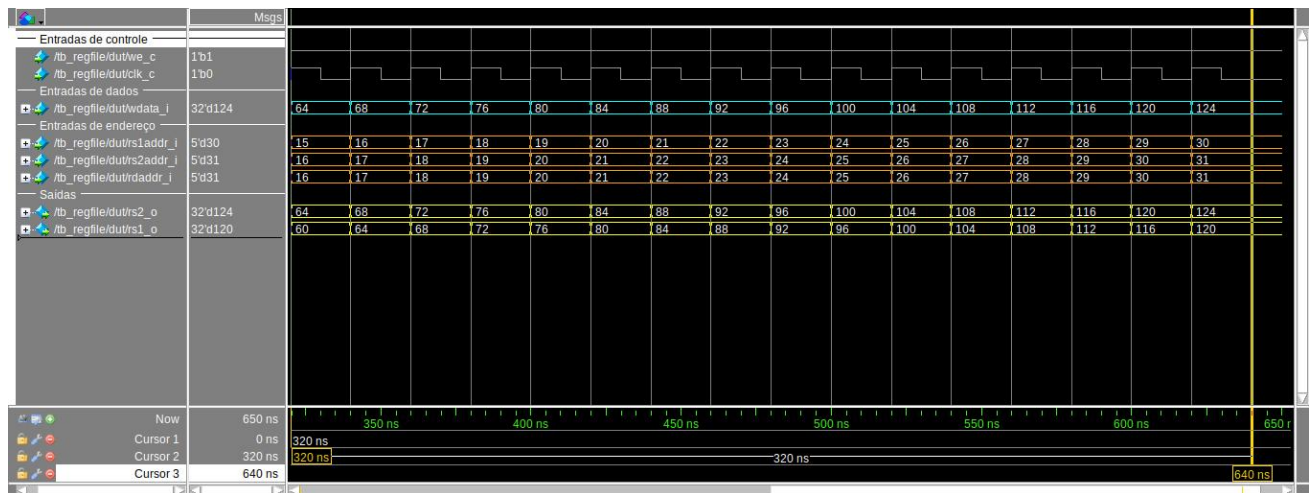
4.6.3 Resultados da Simulação

Figura 25 – Waveform do banco de registradores 1



Fonte: O Autor

Figura 26 – Waveform do banco de registradores 2



Fonte: O Autor

O testbench percorre todos os endereços de 0 a 31. Para cada valor de i , o dado escrito é $i * 4$. O sinal we_c permanece em 1 durante todo o teste. O clock clk_c alterna com período de 20 ns (10 ns em nível alto, 10 ns em nível baixo). Na borda de subida de cada ciclo, o registrador apontado por $rdaddr_i$ é atualizado com $wdata_i$. O banco de registradores implementa corretamente a lógica de leitura e escrita, respeitando a prioridade do registrador $x0$ sempre zero. A indexação baseada em $endereço - 1$ é funcional, embora fuja um pouco do estilo convencional. Para fins de integração com o restante do

processador, é importante garantir que os demais módulos (como a unidade de controle e a ALU) utilizem os mesmos endereços de registradores sem ajustes. Neste projeto, a interface com o banco de registradores segue o padrão RISC-V, sendo compatível.

4.7 Memória de Instruções

A memória de instruções é responsável por armazenar o programa a ser executado e fornecer a instrução correspondente ao endereço solicitado pelo Contador de Programa (PC). Neste projeto, a memória de instruções foi implementada como uma ROM (Read-Only Memory) de 256 posições, cada uma contendo uma instrução de 32 bits. O conteúdo da memória é carregado a partir de um arquivo texto (`fib.hex`) no início da simulação.

4.7.1 Implementação do Módulo `inst_memory`

O módulo `inst_memory.sv` utiliza o package `data_structures` e possui uma interface simplificada: apenas o endereço de leitura (`addr_i`) e a instrução de saída (`inst_o`). O sinal de reset foi removido, uma vez que a ROM é um dispositivo de leitura apenas e não necessita de reinicialização.

```
1 import data_structures::*;
2
3 module inst_memory (
4     input address_t addr_i,
5     output instruction_t inst_o
6 );
7     data_t rom[256]; // 256 palavras de 32 bits
8
9     // Inicializa a memória com o conteúdo do arquivo fib.hex
10    initial begin
11        $readmemh("fib.hex", rom);
12    end
13
14    // Leitura combinacional
15    assign inst_o = rom[addr_i[8:2]];
16 endmodule
```

Listing 4.8 – Módulo `inst_memory.sv`

Observações sobre a implementação:

- A memória é implementada como um array de 256 elementos do tipo `data_t` (32 bits).
- A inicialização é feita através da tarefa `$readmemh`, que lê valores hexadecimais do arquivo `fib.hex` e os armazena sequencialmente a partir do endereço 0.
- O endereço de entrada `addr_i` é um vetor de 32 bits, mas como cada instrução ocupa 4 bytes e a memória possui 256 palavras, são utilizados apenas os bits `addr_i[8:2]`

como índice. Isso corresponde a um endereçamento word-aligned com capacidade para 256 instruções (endereços de 0x00 a 0x3FC).

- A leitura é combinacional, fornecendo a instrução no mesmo ciclo em que o endereço é apresentado, característica essencial para o processador monociclo.

4.7.2 Arquivo de Inicialização `fib.hex`

O arquivo `fib.hex` contém as instruções do programa de teste. Para esta simulação, foi utilizado um programa que calcula a sequência de Fibonacci, cujas algumas instruções (em hexadecimal) são apresentadas a seguir, o escopo global das instruções não é de interesse para este teste, portanto foram listadas apenas as instruções contidas no testbench.

```
1 00001fb7
2 0c000e13
3 008e1e13
4 0c0e0e13
5 008e1e13
6 0c0e0e13
7 008e1e13
8 0c0e0e13
9 ffcfaa23
10 00001fb7
11 ffafc283
12 ... (demais instruções)
```

Listing 4.9 – `fib.hex` programa de exemplo (Fibonacci)

Cada linha representa uma instrução de 32 bits no formato hexadecimal, armazenada consecutivamente a partir do endereço 0. O programa implementa o cálculo da sequência de Fibonacci e faz uso de operações aritméticas, desvios condicionais e chamadas de sub-rotina, exercitando assim os principais tipos de instrução do processador.

4.7.3 Testbench da Memória de Instruções

O testbench `tb.sv` instancia o módulo `inst_memory` e aplica uma sequência crescente de endereços para verificar a leitura correta das instruções armazenadas no arquivo `fib.hex`.

```
1 import data_structures::*;
2
3 module tb ();
4     address_t addr_i;
5     instruction_t inst_o;
6     integer i;
7
8     inst_memory dut (.*);
9
10    initial begin
11        $display("Começando TB...");
```

```

12     addr_i = 0;
13
14     for (i = 0; i < 10; i++) begin
15         addr_i += 4;    // incrementa de 4 em 4 bytes
16         #10;
17     end
18
19     $stop;
20 end
21 endmodule

```

Listing 4.10 – Testbench tb.sv para a memória de instruções

4.7.4 Resultados da Simulação

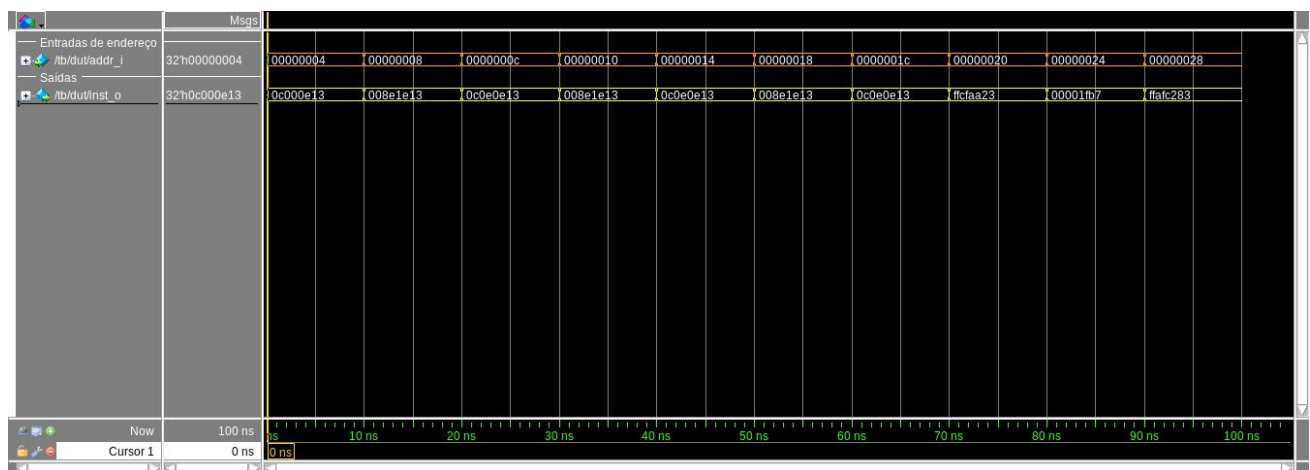


Figura 27 – Waveform da memória de instruções.

A Figura 27 apresenta o comportamento da memória de instruções durante a simulação.

- O endereço `addr_i` inicia em `0x00000000` e é incrementado de 4 em 4 a cada 10 ns (valores: `0x00`, `0x04`, `0x08`, `0x0C`, `0x10`, `0x14`, `0x18`, `0x1C`, `0x20`, `0x24`,).

- A saída `inst_o` assume os valores armazenados no arquivo `fib.hex` para os endereços correspondentes:
 - `addr_i = 0x00 inst_o = 0x00001FB7` (primeira instrução do programa).
 - `addr_i = 0x04 inst_o = 0x0C000E13` (segunda instrução).
 - `addr_i = 0x08 inst_o = 0x008E1E13`.
 - Os valores subsequentes seguem a ordem do arquivo.
- A saída responde de forma combinacional às mudanças no endereço, sem atrasos visíveis, confirmando o funcionamento correto da ROM.

A simulação comprova que a memória de instruções é inicializada corretamente e fornece as instruções esperadas para o programa de Fibonacci, servindo como base para a execução do processador completo.

4.8 Memória de Dados e Entrada e Saída

A memória de dados é responsável por armazenar valores manipulados pelo programa durante a execução, suportando operações de leitura (`load`) e escrita (`store`) com diferentes granularidades: byte (8 bits), meia palavra (16 bits) e palavra (32 bits). Além disso, o módulo incorpora uma região de memória mapeada para entrada e saída (E/S), permitindo a comunicação com periféricos. O módulo foi implementado em SystemVerilog no arquivo `data_memory.sv`, utilizando uma RAM síncrona de 1024 posições de 32 bits para a memória principal e uma pequena memória separada de 4 posições para E/S.

4.8.1 Implementação do Módulo

O módulo possui interface síncrona com clock ativo na borda de descida (`negedge clk_i`) e inclui uma lógica de *byte enable* para permitir acessos parciais à memória, conforme exigido pelas instruções `lb`, `lbu`, `lw`, `lh`, `lhu`, `sb`, `sh` e `sw`. A região de E/S é mapeada nos quatro últimos endereços da memória (1020 a 1023), permitindo a leitura e escrita de dados de/para dispositivos externos.

```

1  import data_structures::*;
2
3  module data_memory (
4      input logic clk_i,
5      input logic memwrite_c, memread_c,
6      input funct3_t funct3_i,
7      input address_t addr_i,
8      input data_t wdata_i,
9      input data_t [1:0] systeminput_i,
10     output data_t rdata_o,
11     output data_t [1:0] systemoutput_o
12 );
```



```

13  localparam int MEMORY_SIZE = 1024;
14  (* ramstyle = "M9K" *) data_t data_mem[MEMORY_SIZE];
15  data_t io_mem[4];
16
17  address_t word_addr, io_addr;
18  data_t mem_data, io_data, raw_data;
19  logic is_io;
20
21  assign word_addr = addr_i >> 2;
22  assign io_addr = word_addr - (MEMORY_SIZE - 4);
23  assign is_io = (word_addr >= (MEMORY_SIZE - 4));
24  assign raw_data = (is_io) ? io_data : mem_data;
25
26  logic [3:0] be;
27  always_comb begin : be_generator
28      be = 4'b0000;
29      if (funct3_i == 3'b000 || funct3_i == 3'b100) begin // lb ou lbu
30          case (addr_i[1:0])
31              2'b00: be = 4'b0001;
32              2'b01: be = 4'b0010;
33              2'b10: be = 4'b0100;
34              2'b11: be = 4'b1000;
35              default: be = 4'b0000;
36          endcase
37      end else if (funct3_i == 3'b001 || funct3_i == 3'b101) begin // lh ou lhu
38          case (addr_i[1])
39              1'b0: be = 4'b0011;
40              1'b1: be = 4'b1100;
41              default: be = 4'b0000;
42          endcase
43      end else if (funct3_i == 3'b010) begin // lw ou sw
44          be = 4'b1111;
45      end
46  end
47
48  always_ff @(negedge clk_i) begin : Write
49      // Atualiza os registradores de E/S a partir das entradas de sistema
50      io_mem[3] <= systeminput_i[0];
51      io_mem[2] <= systeminput_i[1];
52      systemoutput_o[0] <= io_mem[1];
53      systemoutput_o[1] <= io_mem[0];
54
55      if (memwrite_c) begin
56          if (word_addr < MEMORY_SIZE - 4) begin
57              for (int i = 0; i < 4; i++)
58                  if (be[i]) data_mem[word_addr][i*8+:8] <= wdata_i[i*8+:8];
59          end else begin
60              for (int i = 0; i < 4; i++)
61                  if (be[i]) io_mem[io_addr][i*8+:8] <= wdata_i[i*8+:8];
62          end
63      end
64
65      mem_data <= (word_addr < MEMORY_SIZE - 4) ? data_mem[word_addr] : 0;
66      io_data <= (io_addr < 4) ? io_mem[io_addr] : 0;
67  end
68
69  always_comb begin : Read
70      rdata_o = 0;
71      if (memread_c) begin
72          if (funct3_i == 3'b000 || funct3_i == 3'b001 || funct3_i == 3'b010) begin

```

```

73      // Leitura com extensão de sinal (lb, lh, lw)
74      case (be)
75          4'b0001: rdata_o = 32'(signed'(raw_data[7:0]));
76          4'b0010: rdata_o = 32'(signed'(raw_data[15:8]));
77          4'b0100: rdata_o = 32'(signed'(raw_data[23:16]));
78          4'b1000: rdata_o = 32'(signed'(raw_data[31:24]));
79          4'b0011: rdata_o = 32'(signed'(raw_data[15:0]));
80          4'b1100: rdata_o = 32'(signed'(raw_data[31:16]));
81          4'b1111: rdata_o = raw_data;
82          default: ;
83      endcase
84  end
85  if (funct3_i == 3'b100 || funct3_i == 3'b101) begin
86      // Leitura com extensão de zero (lbu, lhu)
87      case (be)
88          4'b0001: rdata_o = 32'(unsigned'(raw_data[7:0]));
89          4'b0010: rdata_o = 32'(unsigned'(raw_data[15:8]));
90          4'b0100: rdata_o = 32'(unsigned'(raw_data[23:16]));
91          4'b1000: rdata_o = 32'(unsigned'(raw_data[31:24]));
92          4'b0011: rdata_o = 32'(unsigned'(raw_data[15:0]));
93          4'b1100: rdata_o = 32'(unsigned'(raw_data[31:16]));
94          default: ;
95      endcase
96  end
97  end
98  end
99  endmodule

```

Listing 4.11 – Módulo data_memory.sv

Características da implementação:

- A memória é endereçada por palavra (word-aligned): `addr_i[31:2]` seleciona a palavra, e os bits `addr_i[1:0]` determinam o deslocamento dentro da palavra para acessos a byte/meia palavra.
- O sinal `be` (byte enable) é gerado combinacionalmente a partir de `funct3_i` e dos bits baixos do endereço, indicando quais bytes da palavra serão lidos/escritos.
- As operações de escrita e a atualização dos registradores de leitura ocorrem na borda de descida do clock (`negedge clk_i`), em sintonia com o restante do processador.
- Os dados lidos da memória principal e da região de E/S são armazenados em registradores intermediários (`mem_data` e `io_data`) na borda de clock, garantindo que a leitura combinacional utilize valores estáveis do ciclo anterior.
- Para leituras, o módulo distingue entre extensão de sinal (`lb`, `lh`) e extensão de zero (`lbu`, `lhu`), convertendo o valor lido para 32 bits conforme a especificação RISC-V.
- A região de E/S ocupa os quatro últimos endereços (1020 a 1023). Os endereços 1020 e 1021 são utilizados para saída (escrita), enquanto os endereços 1022 e 1023 são utilizados para entrada (leitura). Os sinais `systeminput_i` alimentam os registradores de entrada, e `systemoutput_o` refletem os registradores de saída.

- O atributo (`* ramstyle = "M9K"`) é uma diretiva de síntese para FPGAs da família Intel, indicando que a memória deve ser implementada utilizando blocos de RAM dedicados.

4.8.2 Testbench da Memória de Dados e Entrada e Saída

O testbench `tb.sv` para a memória de dados instancia o módulo `data_memory`, gera um clock de período 10 ns e aplica uma sequência de escrita e leitura para verificar os acessos em diferentes granularidades, incluindo a região de E/S mapeada. Os sinais `systeminput_i` e `systemoutput_o` permitem simular a interação com periféricos.

```

1  import data_structures::*;
2
3  module tb();
4      logic clk_c;
5      logic memwrite_c, memread_c;
6      funct3_t funct3_i;
7      address_t addr_i;
8      data_t wdata_i, rdata_o;
9      data_t [1:0] systeminput_i, systemoutput_o;
10
11     data_memory dut (.*); // Conexão automática
12
13     always #5 clk_c = !clk_c;
14
15     initial begin
16         $display("Iniciando TB memória de dados...");
17         clk_c = 0; memwrite_c = 0; memread_c = 0;
18         funct3_i = 2; // word
19         addr_i = 0; wdata_i = 0;
20         systeminput_i[0] = 2004; // valor de entrada para simular periférico
21         systeminput_i[1] = 0;
22         #10;
23
24         // Escrita de palavra no endereço de E/S (saída)
25         memwrite_c = 1;
26         wdata_i = 2048;
27         addr_i = 4084; // endereço correspondente a io_mem[1] (saída)
28         #10;
29
30         // Leitura da palavra no endereço 0 (memória principal)
31         memwrite_c = 0;
32         memread_c = 1;
33         #10;
34
35         // Escrita de byte (10) no endereço 0 (função lb/sb)
36         memwrite_c = 1; memread_c = 0;
37         funct3_i = 0; // byte
38         wdata_i = 10; addr_i = 0;
39         #10;
40
41         // Leitura do byte no endereço 0
42         memwrite_c = 0; memread_c = 1;
43         #10;
44
45         // Leitura como palavra do mesmo endereço

```

```

46     addr_i = 0; #10;
47     funct3_i = 2; #10;
48
49     $stop;
50 end
51 endmodule

```

Listing 4.12 – Testbench tb.sv para a memória de dados (com E/S)

No testbench, o sinal `systeminput_i[0]` é inicializado com 2004, que será armazenado no registrador de entrada `io_mem[3]` (endereço 1023). A escrita em `addr_i = 4084` (endereço da palavra 1021) escreve o valor 2048 no registrador de saída `io_mem[1]`, refletindo-se em `systemoutput_o[0]`. As demais operações testam a memória principal com diferentes granularidades, conforme descrito anteriormente.

4.8.3 Resultados da Simulação

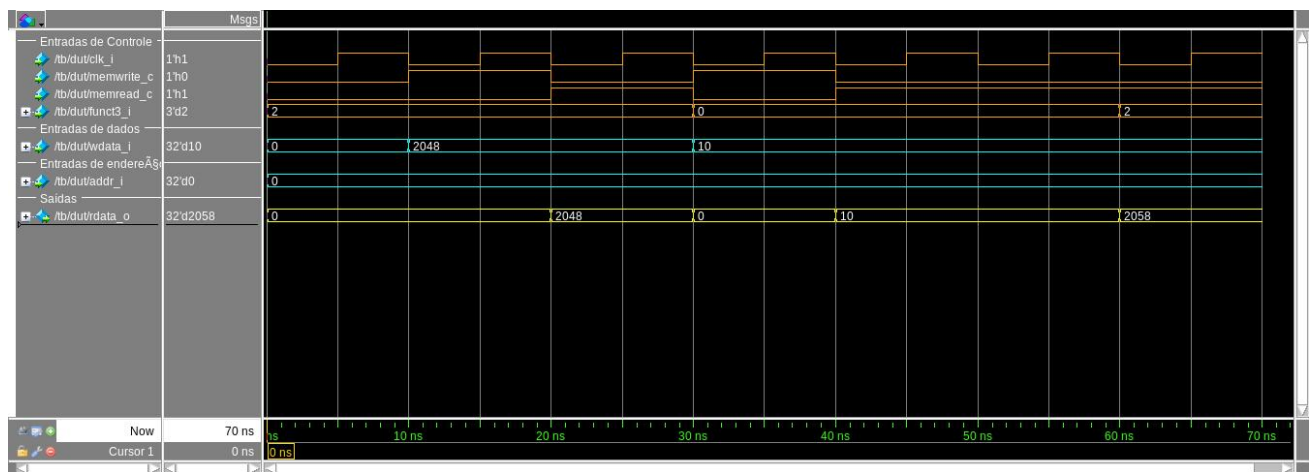


Figura 28 – Waveform da simulação da memória de dados (com E/S).

A Figura 28 apresenta o comportamento do módulo durante os testes. A análise é dividida nos intervalos de tempo (em ns) conforme a sequência aplicada.

- **t = 0 a 10 ns (inicialização):** `clk_c` alterna a cada 5 ns. Inicialmente, `memwrite_c` = 0, `memread_c` = 0, `funct3_i` = 2 (word), `addr_i` = 0, `wdata_i` = 0. Os sinais `systeminput_i` são carregados com 2004 (entrada) e 0. A saída `rdata_o` está em alta impedância (32'hx) porque não há leitura ativa.
- **t = 10 a 20 ns (escrita em E/S):** `memwrite_c` = 1, `wdata_i` = 2048, `addr_i` = 4084. Na borda de descida do clock em t = 15 ns, o valor 2048 é escrito no registrador de saída `io_mem[1]`, que se reflete imediatamente em `systemoutput_o[0]`. O sinal `rdata_o` permanece indeterminado.
- **t = 20 a 30 ns (leitura da palavra 0):** `memwrite_c` = 0, `memread_c` = 1, `funct3_i` = 2. Na borda de descida anterior (t = 25 ns), o registrador `mem_data` é atualizado com o valor lido (inicialmente 0). O bloco combinacional coloca `rdata_o` = 0.
- **t = 30 a 40 ns (escrita de byte 10 no endereço 0):** `memwrite_c` = 1, `memread_c` = 0, `funct3_i` = 0, `wdata_i` = 10, `addr_i` = 0. Na borda de descida (t = 35 ns), o byte 10 é escrito no primeiro byte da palavra no endereço 0, preservando os demais bytes. Após essa operação, o conteúdo da palavra passa a ser 0x0000000A (pois ainda não havia sido escrito outro valor).
- **t = 40 a 50 ns (leitura do byte):** `memwrite_c` = 0, `memread_c` = 1, `funct3_i` = 0. Na borda de descida (t = 45 ns), `mem_data` é atualizado com o valor da palavra, e o bloco combinacional seleciona o byte, estende o sinal (como o valor 10 é positivo, resulta em 10) e coloca `rdata_o` = 10.
- **t = 50 a 60 ns (endereço mantido):** O testbench mantém `addr_i` = 0 sem novas operações. A saída permanece em 10.
- **t = 60 a 70 ns (leitura como palavra do mesmo endereço):** `funct3_i` = 2. Na borda de descida (t = 65 ns), `mem_data` é atualizado com a palavra completa (0x0000000A = 10), e `rdata_o` assume esse valor, confirmando a escrita correta do byte.

Os sinais de E/S também são verificados: `systemoutput_o[0]` mantém o valor 2048 após a escrita, e `systeminput_i[0]` (2004) é lido como dado sempre que o endereço 4092 (palavra 1023) for acessado (não mostrado na sequência, mas a lógica está presente). O waveform confirma o correto funcionamento da memória de dados com integração de E/S, incluindo a lógica de *byte enable*, extensão de sinal/zero, e o mapeamento da região de E/S.

4.9 Unidade de Branch

A unidade de branch é responsável por calcular o próximo endereço do Contador de Programa (PC) durante a execução de instruções de desvio condicional (`beq`, `bne`, etc.), saltos incondicionais (`jal`) e saltos indiretos (`jalr`). Este módulo recebe os sinais de controle gerados pela unidade de controle (`jump_c`, `branch_c`, `pcsource_c`), o valor da ALU (que fornece o bit de condição para os desvios), os valores dos operandos (`pc_i`, `imm_i`, `rs1_i`) e produz o próximo PC (`pc_next_o`).

4.9.1 Implementação do Módulo `branch_unit`

O módulo `branch_unit.sv` utiliza lógica combinacional para calcular o próximo PC de acordo com a equação:

$$pc_next = base + incremento$$

onde a base é selecionada pelo sinal `pcsource_c` (`rs1_i` para JALR, `pc_i` para os demais casos), e o incremento é `imm_i` para saltos ou desvios ativos, caso contrário 4 (incremento normal).

```

1  import data_structures::*;
2
3  module branch_unit (
4      input register_t rs1_i, pc_i,
5      input data_t aluout_i, imm_i,
6      input logic jump_c, branch_c, pcsource_c,
7      output register_t pc_next_o
8  );
9      data_t sum1 = 0, sum2 = 0;
10
11  always_comb begin : branch_unit
12      sum1 = (pcsource_c) ? rs1_i : pc_i;
13      sum2 = (jump_c || (branch_c && aluout_i[0])) ? imm_i : 32'(4);
14      pc_next_o = sum1 + sum2;
15  end
16  endmodule

```

Listing 4.13 – Módulo `branch_unit.sv`

Funcionamento:

- `pcsource_c`: quando em 1, a base do endereço é `rs1_i` (usado em `jalr`); quando em 0, a base é o PC atual.
- `sum2`: é `imm_i` se houver salto incondicional (`jump_c` = 1) ou um desvio condicional ativo (`branch_c` = 1 e o bit de condição `aluout_i[0]` = 1); caso contrário, `sum2` = 4 para avançar para a próxima instrução sequencial.
- A soma `sum1 + sum2` produz o endereço da próxima instrução.

4.9.2 Testbench da Unidade de Branch

O testbench `branch_tb.sv` aplica estímulos para verificar as três operações principais: desvio condicional (branch), salto e ligação (jal) e salto indireto (jalr). O clock não é necessário, pois o módulo é combinacional.

```

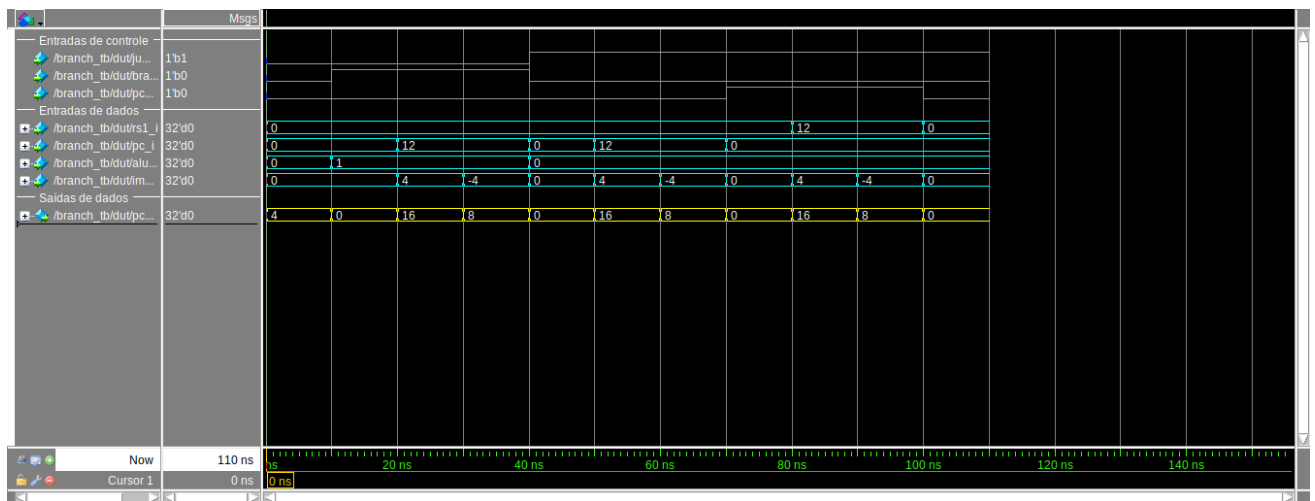
1  import data_structures::*;
2
3  module branch_tb ();
4      register_t rs1_i, pc_i, pc_next_o;
5      data_t aluout_i, imm_i;
6      logic jump_c, branch_c, pcsource_c;
7
8      branch_unit dut (.*);
9
10     initial begin
11         $display("Iniciando valores...");
12         rs1_i = 0; pc_i = 0; imm_i = 0; aluout_i = 0;
13         jump_c = 0; branch_c = 0; pcsource_c = 0;
14         #10;
15
16         // Teste BRANCH: branch_c=1, condicao verdadeira
17         branch_c = 1; aluout_i[0] = 1;
18         pc_i = 12; imm_i = 4; // esperado 16
19         #10;
20
21         pc_i = 12; imm_i = -4; // esperado 8
22         #10;
23
24         // Teste JAL: jump_c=1
25         branch_c = 0; jump_c = 1;
26         pc_i = 12; imm_i = 4; // esperado 16
27         #10;
28
29         pc_i = 12; imm_i = -4; // esperado 8
30         #10;
31
32         // Teste JALR: pcsource_c=1
33         jump_c = 0; pcsource_c = 1;
34         rs1_i = 12; imm_i = 4; // esperado 16
35         #10;
36
37         rs1_i = 12; imm_i = -4; // esperado 8
38         #10;
39
40         $stop;
41     end
42 endmodule

```

Listing 4.14 – Testbench `branch_tb.sv`

4.9.3 Resultados da Simulação

Figura 29 – Waveform da simulação da unidade de branch.



Fonte: O Autor

A Figura 29 mostra os resultados dos testes. A análise é dividida pelos três grupos de operação:

- **Desvio condicional (branch):** $\text{branch_c} = 1$, $\text{aluout_i}[0] = 1$ (condição verdadeira). Para $\text{pc_i} = 12$, $\text{imm_i} = 4$ $\text{pc_next_o} = 16$ (correto). Para $\text{pc_i} = 12$, $\text{imm_i} = -4$ $\text{pc_next_o} = 8$ (correto).
- **Salto e ligação (jal):** $\text{jump_c} = 1$, independente da condição. $\text{pc_i} = 12$, $\text{imm_i} = 4$ $\text{pc_next_o} = 16$. $\text{pc_i} = 12$, $\text{imm_i} = -4$ $\text{pc_next_o} = 8$. O módulo ignora aluout_i e branch_c nesta configuração.
- **Salto indireto (jalr):** $\text{pcsource_c} = 1$, base é rs1_i em vez de pc_i . $\text{rs1_i} = 12$, $\text{imm_i} = 4$ $\text{pc_next_o} = 16$. $\text{rs1_i} = 12$, $\text{imm_i} = -4$ $\text{pc_next_o} = 8$. O comportamento está de acordo com a especificação RISC-V para a instrução jalr.

Todos os resultados observados no waveform estão em conformidade com os valores esperados. A unidade de branch opera de forma combinacional, produzindo o próximo endereço sem atrasos adicionais, o que é essencial para o funcionamento do processador monociclo.

4.10 Contador de Programa

O Contador de Programa (Program Counter PC) é o registrador que armazena o endereço da instrução corrente. A cada ciclo de clock, ele é atualizado com o próximo

endereço fornecido pela unidade de branch. Neste projeto, o PC não foi implementado como um módulo separado, mas sim como um bloco interno no código de integração final (topo do processador), sendo assim, sua waveform não foi gerada, a corretude do PC será avaliada na simulação da integração. A lógica de atualização é a seguinte:

```
1 always_ff @(posedge clk_c) begin : pc_update
2     if (RESET)
3         pc <= 32'd0;
4     else
5         pc <= pc_next;
6 end
```

Listing 4.15 – Lógica do PC (dentro do módulo topo)

onde:

- `clk_c` é o sinal de clock global do processador.
- `RESET` é um sinal de reset síncrono (quando ativo, o PC é zerado na próxima borda de clock).
- `pc_next` é o próximo endereço calculado pela unidade de branch (`branch_unit`).
- `pc` é o registrador interno que armazena o endereço atual, cujo valor é utilizado para acessar a memória de instruções.

A saída do PC é conectada diretamente à entrada de endereço da memória de instruções, permitindo a busca da instrução correspondente a cada ciclo.

4.11 Gerador de Imediatos

O gerador de imediatos é um módulo combinacional responsável por extrair e estender o valor imediato presente em diferentes campos da instrução, conforme o formato definido pela arquitetura RISC-V. Cada tipo de instrução (I, S, B, U, J) possui uma disposição distinta dos bits do imediato dentro da palavra de 32 bits. Este módulo recebe a instrução completa e o tipo de instrução decodificado, produzindo o valor imediato de 32 bits (já com extensão de sinal, quando necessário).

4.11.1 Implementação do Módulo `immediate_generator`

O módulo `immediate_generator.sv` implementa a lógica de extração para os seguintes tipos:

- **OPI e LOAD:** imediato de 12 bits nos campos `inst_i[31:20]`, com extensão de sinal.

- **AUIPC:** imediato de 20 bits nos campos `inst_i[31:12]`, deslocado à esquerda por 12 bits (preenchendo os bits inferiores com zeros), sem extensão de sinal.
- **STORE:** imediato de 12 bits composto por `inst_i[31:25]` (bits altos) e `inst_i[11:7]` (bits baixos), com extensão de sinal.
- **BRANCH:** imediato de 13 bits (incluindo o bit de deslocamento 0 no LSB), formado por `inst_i[31]`, `inst_i[7]`, `inst_i[30:25]`, `inst_i[11:8]`, e o bit 0 fixo em 0. A extensão de sinal é realizada pela replicação do bit 31 (`inst_i[31]`) por 20 posições.
- **JUMP:** imediato de 21 bits (incluindo o bit 0 no LSB), formado por `inst_i[31]`, `inst_i[19:12]`, `inst_i[20]`, `inst_i[30:21]`, e o bit 0 fixo em 0. A extensão de sinal é realizada pela replicação do bit 31 (`inst_i[31]`) por 12 posições.

O código a seguir implementa essa lógica.

```

1  import data_structures::*;
2
3  module immediate_generator (
4      input instruction_t inst_i,
5      input inst_type_t   inst_type_c,
6      output data_t imm_o
7  );
8      always_comb begin : immediate_generator
9          imm_o = 0;
10         case (inst_type_c)
11             OPI, LOAD:
12                 imm_o = 32'(signed'(inst_i[31:20]));
13             AUIPC:
14                 imm_o = {inst_i[31:12], 12'b00000000000000};
15             STORE:
16                 imm_o = 32'(signed'({inst_i[31:25], inst_i[11:7]}));
17             BRANCH:
18                 imm_o = {{20{inst_i[31]}}, inst_i[7], inst_i[30:25], inst_i[11:8], 1'b0};
19             JUMP:
20                 imm_o = {{12{inst_i[31]}}, inst_i[19:12], inst_i[20], inst_i[30:21], 1'b0};
21             default: ;
22         endcase
23     end
24 endmodule

```

Listing 4.16 – Módulo `immediate_generator.sv`

4.11.2 Testbench do Gerador de Imediatos

O testbench `immgen_tb.sv` aplica valores de instrução para cada tipo e verifica o imediato gerado.

```

1  import data_structures::*;
2
3  module immgen_tb ();
4      instruction_t inst_i;

```

```

5   inst_type_t inst_type_c;
6   data_t imm_o;
7
8   immediate_generator dut (.*);
9
10  initial begin
11      $display("Inicializando valores...");
12      inst_i = 0; inst_type_c = OP;
13      #10;
14
15      // Testes para OPI e LOAD (imediatos de 12 bits)
16      inst_type_c = OPI;
17      inst_i = 32'b11111111111100000000000000000000; // esperado -1
18      #10;
19      inst_i = 32'b00000000000100000000000000000000; // esperado 1
20      #10;
21
22      // Testes para AUIPC (imediato superior)
23      inst_type_c = AUIPC;
24      inst_i = 32'b01000000000000000000000000000000; // esperado 0x40000000 (1073741824)
25      #10;
26      inst_i = 32'b11000000000000000000000000000000; // esperado -1073741824
27      #10;
28
29      // Testes para STORE
30      inst_type_c = STORE;
31      inst_i = 32'b0111111100000000000000111110000000; // esperado 2047
32      #10;
33      inst_i = 32'b10000000000100010010000010100011; // esperado -2047
34      #10;
35
36      // Testes para BRANCH
37      inst_type_c = BRANCH;
38      inst_i = 32'b01000000000000000000000000000000; // esperado 1024
39      #10;
40      inst_i = 32'b11000000000000000000000010000000; // esperado -1024
41      #10;
42
43      // Testes para JUMP
44      inst_type_c = JUMP;
45      inst_i = 32'b01000000000000000000000000000000; // esperado 1024
46      #10;
47      inst_i = 32'b11000000000111111111000001101111; // esperado -1024
48      #10;
49
50      $stop;
51  end
52 endmodule

```

Listing 4.17 – Testbench immgen_tb.sv

4.11.3 Resultados da Simulação

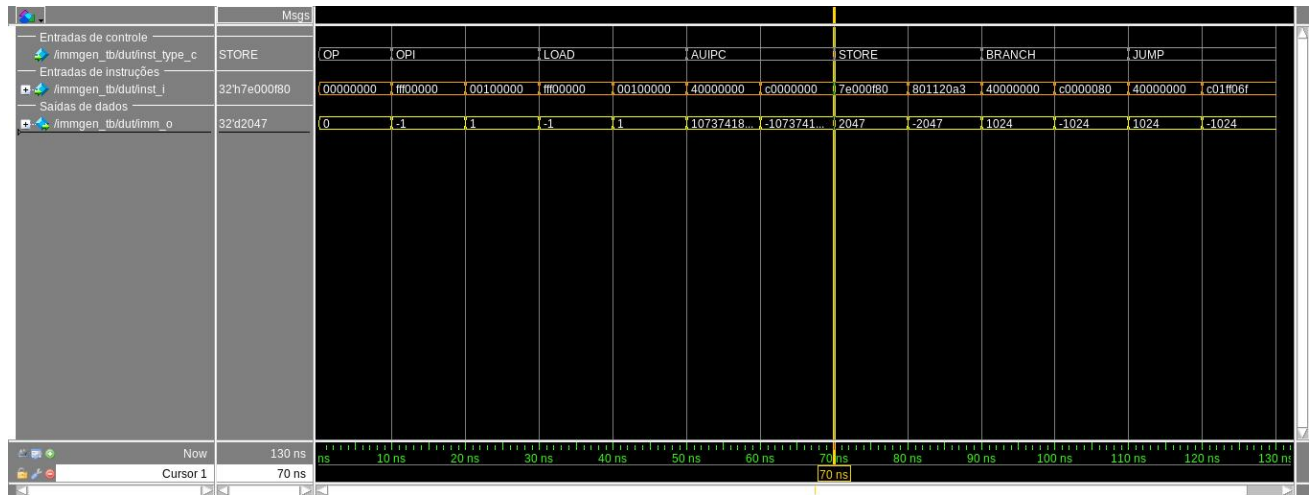


Figura 30 – Waveform da simulação do gerador de imediatos.

A Figura 30 apresenta os resultados dos testes. A análise é feita por tipo de instrução:

- **OPI / LOAD:** Para `inst_type_c = OPI`, com `inst_i = 0xFFFFF000` (em 32 bits: `32'b11111111111110000000000000000000`) imediato = -1. Em seguida, `inst_i = 0x00100000` imediato = 1. Os valores estão corretos e a extensão de sinal é aplicada.
- **AUIPC:** `inst_type_c = AUIPC`, `inst_i = 0x40000000` imediato = `0x40000000` (1073741824). `inst_i = 0xC0000000` imediato = -1073741824. O imediato é extraído dos bits 31:12 e deslocado à esquerda, conforme esperado.
- **STORE:** `inst_type_c = STORE`, `inst_i = 0x7E000F80` imediato = 2047. `inst_i = 0x801120A3` imediato = -2047. A concatenação dos campos `inst_i[31:25]` e `inst_i[11:7]` e a extensão de sinal estão corretas.

- **BRANCH:** `inst_type_c = BRANCH`, `inst_i = 0x40000000` imediato = 1024. `inst_i = 0xC0000080` imediato = -1024. Observa-se que o bit LSB é fixado em 0, e a concatenação segue o formato B, com a extensão de sinal realizada pela replicação do bit 31.
- **JUMP:** `inst_type_c = JUMP`, `inst_i = 0x40000000` imediato = 1024. `inst_i = 0xC01FF06F` (instrução `jal`) imediato = -1024. O formato J é corretamente reconstruído, incluindo o bit de sinal (replicado por 12 posições) e o bit 0 fixo.

Todos os resultados observados no waveform correspondem aos valores esperados. O gerador de immediatos opera de forma combinacional e sem atrasos, sendo essencial para fornecer o valor imediato correto à ALU e à unidade de branch.

4.12 Unidade de Controle

A unidade de controle é o módulo responsável por decodificar a instrução lida da memória de instruções e gerar todos os sinais de controle que governam o funcionamento dos demais blocos do processador. Ela recebe como entrada a instrução de 32 bits e produz, de forma combinacional, os sinais que configuram a ALU, o banco de registradores, a memória de dados, a unidade de branch e o próprio PC.

4.12.1 Interface e Sinais de Controle

A unidade de controle implementada no arquivo `control_unit.sv` possui a seguinte interface:

```

1 module control_unit (
2     input instruction_t inst_i,
3     output inst_type_t inst_type_c,
4     output logic memwrite_c, memread_c,
5     output logic alusel1_c, alusel2_c,
6     output logic we_c,
7     output logic memtoreg_c,
8     output logic lui_c,
9     output logic branch_c,
10    output logic jump_c,
11    output logic pcsource_c,
12    output logic ebreak_c
13 );
```

Listing 4.18 – Interface da Unidade de Controle

Os sinais gerados têm as seguintes funções:

- `inst_type_c`: tipo de instrução (OP, OPI, AUIPC, BRANCH, JUMP, LOAD, STORE), utilizado pela ALU e pelo gerador de immediatos.

- `memwrite_c`, `memread_c`: habilita escrita e leitura na memória de dados.
- `alusel1_c`, `alusel2_c`: selecionam os operandos da ALU (PC, rs1, imediato, rs2).
- `we_c`: habilita a escrita no banco de registradores.
- `mentoreg_c`: seleciona se o dado a ser escrito no registrador vem da memória ou da ALU.
- `lui_c`: indica que a instrução é `lui`, ativando o caminho específico para imediato superior.
- `branch_c`, `jump_c`, `pcsource_c`: controlam a unidade de branch para desvios condicionais, saltos incondicionais e saltos indiretos.
- `ebreak_c`: sinal para a instrução `ebreak`, utilizada para interromper o fluxo de instruções.

4.12.2 Implementação do Módulo

O módulo utiliza a enumeração `opcode_t` definida no pacote `data_structures` para extrair o opcode da instrução (`inst_i[6:0]`). Com base no opcode, um bloco `case` atribui os valores adequados a cada sinal de controle. Todos os sinais são inicializados com zero (padrão) e apenas os necessários para cada tipo de instrução são ativados.

```

1  import data_structures::*;
2
3  module control_unit (
4      input instruction_t inst_i,
5      output inst_type_t inst_type_c,
6      output logic memwrite_c, memread_c,
7      output logic alusel1_c, alusel2_c,
8      output logic we_c,
9      output logic mentoreg_c,
10     output logic lui_c,
11     output logic branch_c,
12     output logic jump_c,
13     output logic pcsource_c,
14     output logic ebreak_c
15 );
16 opcode_t opcode;
17 assign opcode = opcode_t'(inst_i[6:0]);
18
19 always_comb begin : control_unit
20     // Inicialização padrão
21     memwrite_c = 0; memread_c = 0;
22     alusel1_c = 0; alusel2_c = 0;
23     we_c = 0; mentoreg_c = 0;
24     lui_c = 0; branch_c = 0;
25     jump_c = 0; pcsource_c = 0;
26     ebreak_c = 0;
27     inst_type_c = OP;
28

```

```

29     case (opcode)
30         opcode_OP: begin
31             we_c = 1;
32         end
33         opcode_OP_IMM: begin
34             alusel2_c = 1;
35             we_c = 1;
36             inst_type_c = OPI;
37         end
38         opcode_LOAD: begin
39             memread_c = 1;
40             alusel2_c = 1;
41             we_c = 1;
42             memtoreg_c = 1;
43             inst_type_c = LOAD;
44         end
45         opcode_STORE: begin
46             memwrite_c = 1;
47             alusel2_c = 1;
48             inst_type_c = STORE;
49         end
50         opcode_BRANCH: begin
51             branch_c = 1;
52             inst_type_c = BRANCH;
53         end
54         opcode_JAL: begin
55             alusel1_c = 1;
56             we_c = 1;
57             jump_c = 1;
58             inst_type_c = JUMP;
59         end
60         opcode_JALR: begin
61             alusel1_c = 1;
62             we_c = 1;
63             jump_c = 1;
64             pcsources_c = 1;
65             inst_type_c = OPI;
66         end
67         opcode_LUI: begin
68             we_c = 1;
69             lui_c = 1;
70             inst_type_c = AUIPC;
71         end
72         opcode_AUIPC: begin
73             alusel1_c = 1;
74             alusel2_c = 1;
75             we_c = 1;
76             inst_type_c = AUIPC;
77         end
78         opcode_EBREAK: begin
79             ebreak_c = 1;
80         end
81         default: ;
82     endcase
83 end
84 endmodule

```

Listing 4.19 – Módulo control_unit.sv

É importante notar que para a instrução `jalr`, a unidade de controle define `inst_type_c = OPI`. Isso ocorre porque o formato da instrução `jalr` é idêntico ao tipo I em termos de campos, e a ALU deve tratá-lo como uma operação com imediato (soma de `rs1` com `imm`). O sinal `pcsource_c` orienta a unidade de branch a usar o valor de `rs1` como base para o salto.

4.12.3 Testbench da Unidade de Controle

O testbench `cu_tb.sv` instancia a unidade de controle e aplica um conjunto de instruções com opcodes representativos de cada tipo. Para cada instrução, os sinais de controle são observados para verificar se a decodificação está correta.

```

1  import data_structures::*;
2
3  module cu_tb ();
4      instruction_t inst_i;
5      inst_type_t   inst_type_c;
6      logic memwrite_c, memread_c, alusel1_c, alusel2_c,
7             we_c, memtoreg_c, lui_c, branch_c, jump_c, pcsource_c;
8
9      control_unit dut (.*);
10
11     initial begin
12         // Teste: Tipo-R (OP)
13         inst_i = 32'h00000033; #10;
14         // Teste: Tipo-I (OP_IMM)
15         inst_i = 32'h00000013; #10;
16         // Teste: LOAD
17         inst_i = 32'h00000003; #10;
18         // Teste: STORE
19         inst_i = 32'h00000023; #10;
20         // Teste: BRANCH
21         inst_i = 32'h00000063; #10;
22         // Teste: JAL
23         inst_i = 32'h0000006F; #10;
24         // Teste: JALR
25         inst_i = 32'h00000067; #10;
26         // Teste: LUI
27         inst_i = 32'h00000037; #10;
28         // Teste: AUIPC
29         inst_i = 32'h00000017; #10;
30         // Teste: EBREAK
31         inst_i = 32'h00100073; #20;
32         $stop;
33     end
34 endmodule

```

Listing 4.20 – Testbench `cu_tb.sv`

4.12.4 Resultados da Simulação

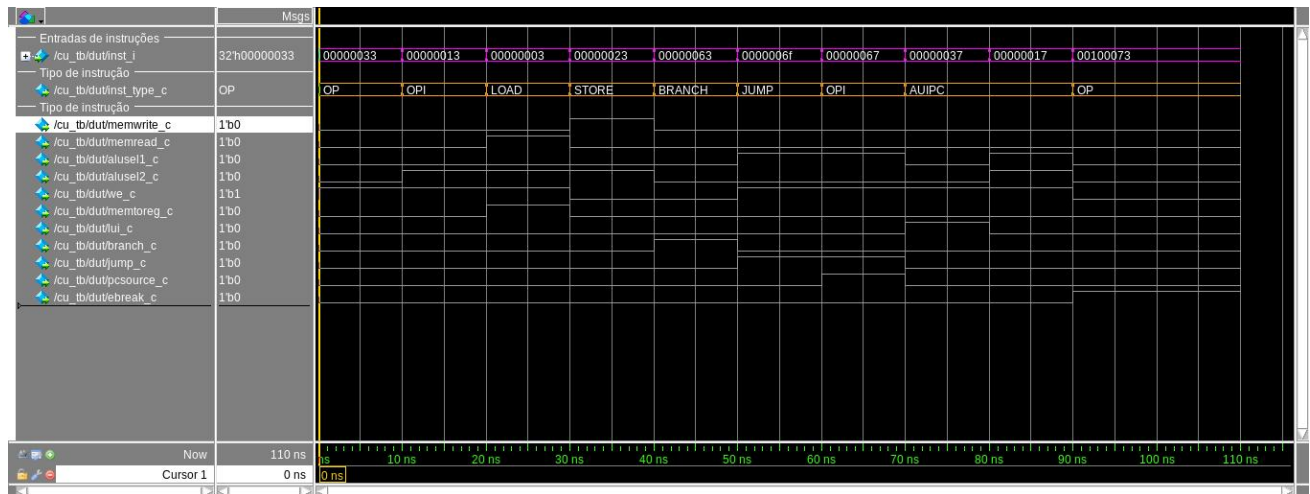


Figura 31 – Waveform da simulação da unidade de controle.

A Figura 31 apresenta os resultados da simulação para os dez opcodes testados. A análise é feita para cada intervalo de tempo:

- **OP (tipo R):** `inst_i = 0x00000033`. Apenas `we_c = 1`. `inst_type_c = OP`. Os demais sinais permanecem em 0.
- **OP_IMM:** `inst_i = 0x00000013`. `alusel2_c = 1`, `we_c = 1`. `inst_type_c = OPI`.
- **LOAD:** `inst_i = 0x00000003`. `memread_c = 1`, `alusel2_c = 1`, `we_c = 1`, `memtoreg_c = 1`. `inst_type_c = LOAD`.
- **STORE:** `inst_i = 0x00000023`. `memwrite_c = 1`, `alusel2_c = 1`. `inst_type_c = STORE`.
- **BRANCH:** `inst_i = 0x00000063`. `branch_c = 1`. `inst_type_c = BRANCH`.

- **JAL:** `inst_i = 0x0000006F`. `alusel1_c = 1`, `we_c = 1`, `jump_c = 1`. `inst_type_c = JUMP`.
- **JALR:** `inst_i = 0x00000067`. `alusel1_c = 1`, `we_c = 1`, `jump_c = 1`, `pcsource_c = 1`. `inst_type_c = OPI`.
- **LUI:** `inst_i = 0x00000037`. `we_c = 1`, `lui_c = 1`. `inst_type_c = AUIPC`.
- **AUIPC:** `inst_i = 0x00000017`. `alusel1_c = 1`, `alusel2_c = 1`, `we_c = 1`. `inst_type_c = AUIPC`.
- **EBREAK:** `inst_i = 0x00100073`. `ebreak_c = 1`. Todos os demais sinais permanecem em 0.

Todos os sinais de controle gerados estão de acordo com o esperado para cada tipo de instrução, confirmando o correto funcionamento da unidade de controle. A decodificação combinacional é instantânea, sem latência adicional, o que é fundamental para o modelo de execução monociclo.

Conclusão: A unidade de controle foi implementada e validada, gerando todos os sinais necessários para o funcionamento do processador. Ela será integrada ao caminho de dados completo, conectando-se à memória de instruções, à ALU, ao banco de registradores, à memória de dados e à unidade de branch.

4.13 Programas de Teste

Para validar o funcionamento do processador em diferentes cenários, foram desenvolvidos dois programas de teste em linguagem assembly para a arquitetura RV32IM: um para cálculo da sequência de Fibonacci e outro para cálculo do fatorial de um número. Ambos os programas foram escritos para serem executados tanto em simulação (ModelSim) quanto em bancada (FPGA), explorando o conjunto completo de instruções implementado no processador.

4.13.1 Programa Fibonacci

O programa `fibasmhalt.S` implementa o cálculo do n -ésimo termo da sequência de Fibonacci. O valor de entrada (índice) é lido de uma região de memória mapeada para E/S (endereço de entrada), e o resultado é exibido em quatro displays de sete segmentos, também mapeados em memória (endereço de saída). A sequência de execução é a seguinte:

1. O programa aguarda em um loop (`start_loop`) até que o bit mais significativo do sinal de confirmação (`x5`) seja ativado, indicando que o valor de entrada está disponível.

2. O valor de entrada (índice) é lido e armazenado em `x30`.
3. Se o índice for zero, o programa desvia para `fib_0` e atribui 0 ao resultado.
4. Caso contrário, a sequência de Fibonacci é calculada iterativamente no loop `fib_loop`, utilizando os registradores `x5`, `x6`, `x7` e `x8`.
5. O resultado é armazenado em `x29` e, em seguida, convertido para exibição nos displays através da sub-rotina `table`, que mapeia cada dígito decimal para o padrão de segmentos correspondente.
6. O valor final é escrito no endereço de saída (E/S) e o programa é encerrado com a instrução `ebreak`.

4.13.2 Programa Fatorial

O programa `fatasmhalt.S` segue a mesma estrutura de E/S do programa Fibonacci, porém calcula o fatorial do número de entrada. A sequência de execução é análoga:

1. Aguarda o sinal de confirmação para ler o valor de entrada.
2. Se o valor de entrada for zero, o resultado é 1 (caso `fat_0`).
3. Caso contrário, o fatorial é calculado iterativamente no loop `fat_loop`, utilizando a instrução de multiplicação (`mul`) da extensão M.
4. O resultado é exibido nos displays utilizando a mesma sub-rotina `table`.
5. O valor final é escrito no endereço de saída e o programa é encerrado com `ebreak`.

Ambos os programas utilizam a região de E/S mapeada nos endereços superiores da memória de dados, permitindo a interação com periféricos (chaves e displays) em bancada.

4.13.3 Processo de Compilação

Os programas em assembly foram compilados utilizando o conjunto de ferramentas GNU para RISC-V (`riscv64-elf-gcc`) ([Free Software Foundation, 2025](#)), disponível para sistemas Arch Linux através do repositório oficial `extra` ou via AUR (Arch User Repository) ([Arch Linux, 2025](#)) para versões específicas. O processo de compilação e conversão para o formato hexadecimal (compatível com a memória de instruções do processador) foi automatizado por um script escrito na linguagem Fish ([fish-shell, 2025](#)).

O script `macro.fish` realiza as seguintes etapas:

1. Solicita ao usuário o arquivo de origem (assembly) e o arquivo de destino (hexadecimal).
2. Compila o arquivo assembly utilizando o comando:

```
1 riscv64-elf-gcc -march=rv32im -mabi=ilp32 -nostartfiles -T link.ld $origem -o
   output.elf
```

Onde:

- `-march=rv32im` especifica a arquitetura RV32IM (base inteira com extensão de multiplicação/divisão).
 - `-mabi=ilp32` define a ABI (Application Binary Interface) para inteiros de 32 bits, ponteiros longos e inteiros longos.
 - `-nostartfiles` impede a vinculação dos arquivos de inicialização padrão (startup files).
 - `-T link.ld` utiliza o script de ligação personalizado `link.ld`, que define a organização da memória (instruções e dados).
3. Extrai apenas a seção `.text` (código executável) do arquivo ELF gerado e a converte para um binário raw utilizando o `riscv64-elf-objcopy`:

```
1 riscv64-elf-objcopy -j .text -O binary output.elf inst.bin
```

4. Converte o binário raw para o formato hexadecimal (little-endian, 32 bits por linha) utilizando um script Python ([Python Software Foundation, 2025](#)):

```
1 python3 -c "import sys; data=open('inst.bin','rb').read(); [print(f'{int.from_bytes
   (data[i:i+4], \"little\"):08x}') for i in range(0, len(data), 4)]" >$destino
```

O comando Python lê o arquivo binário em blocos de 4 bytes, interpreta cada bloco como um inteiro de 32 bits no formato little-endian (utilizando `int.from_bytes()` com `byteorder='little'`) e imprime o valor em hexadecimal com 8 dígitos.

5. Remove os arquivos temporários (`output.elf` e `inst.bin`).

O script de ligação `link.ld` define duas regiões de memória: a memória de instruções com 1 KB (256 palavras de 32 bits) e a memória de dados com 4 KB (1024 palavras de 32 bits), refletindo as capacidades implementadas nos módulos `inst_memory` e `data_memory`, respectivamente. As seções `.text`, `.data` e `.bss` são alocadas conforme a arquitetura Harvard do processador.

```
1 OUTPUT_ARCH( "riscv" )
2 ENTRY(_start)
3
4 MEMORY
5 {
6     INST_MEM (rx) : ORIGIN = 0x00000000, LENGTH = 1K
```

```

7   DATA_MEM (rwx) : ORIGIN = 0x00000000, LENGTH = 4K
8 }
9
10 SECTIONS
11 {
12   .text : { *(.text._start) *(.text .text.*) } > INST_MEM
13   .data : { *(.data .data.*) } > DATA_MEM
14   .bss  : { *(.bss .bss.*) *(COMMON) } > DATA_MEM
15   _end = .;
16 }

```

Listing 4.21 – Script de ligação link.ld atualizado

4.14 Integração Final do Processador

A integração final do processador consiste na interconexão de todos os módulos desenvolvidos ao longo do projeto: memória de instruções, banco de registradores, unidade lógica e aritmética (ALU), memória de dados (com E/S mapeada), gerador de imediatos, unidade de branch e unidade de controle. O módulo topo, denominado *integ*, instancia e conecta esses componentes, formando o caminho de dados completo do processador monociclo RISC-V.

4.14.1 Implementação do Módulo Topo

O módulo *integ.sv* define a interface do processador com o mundo externo: clock, reset, sinais de entrada e saída de sistema (para E/S mapeada). Internamente, ele instancia todos os blocos e realiza as conexões necessárias.

```

1  import data_structures::*;
2
3  module integ (
4      input logic clk_i,
5      input logic RESET,
6      input data_t [1:0] systeminput,
7      output data_t [1:0] systemoutput
8  );
9      // Sinais internos
10     register_t rs1, rs2, pc, pc_next;
11     data_t wdata, rdata, imm, aluout;
12     logic [4:0] rdaddr, rs1addr, rs2addr;
13     funct3_t funct3;
14     funct7_t funct7;
15     instruction_t inst;
16     logic halt, ebreak_c, memwrite_c, memread_c,
17         alusel1_c, alusel2_c, we_c, memtoreg_c,
18         lui_c, branch_c, jump_c, pcsource_c;
19     inst_type_t inst_type_c;
20
21     // Instâncias dos módulos
22     inst_memory inst_mem ( .addr_i(pc), .inst_o(inst) );
23     regfile regfile ( .we_c(we_c), .clk_i(clk_i), .wdata_i(wdata),
24         .rdaddr_i(rdaddr), .rs1addr_i(rs1addr), .rs2addr_i(rs2addr),

```

```

25         .rs1_o(rs1), .rs2_o(rs2) );
26     alu alu ( .rs1_i(rs1), .rs2_i(rs2), .imm_i(imm), .pc_i(pc),
27             .funct3_i(funct3), .funct7_i(funct7), .inst_type_c(inst_type_c),
28             .alusel1_c(alusel1_c), .alusel2_c(alusel2_c), .aluout_o(aluout) );
29     data_memory data_mem ( .clk_i(clk_i), .memwrite_c(memwrite_c),
30                           .memread_c(memread_c), .funct3_i(funct3),
31                           .addr_i(aluout), .wdata_i(rs2), .rdata_o(rdata),
32                           .systeminput_i(systeminput), .systemoutput_o(systemoutput) );
33     immediate_generator imm_gen ( .inst_i(inst), .inst_type_c(inst_type_c), .imm_o(imm) );
34     branch_unit bu ( .rs1_i(rs1), .pc_i(pc), .aluout_i(aluout), .imm_i(imm),
35                    .jump_c(jump_c), .branch_c(branch_c), .pcsource_c(pcsource_c),
36                    .pc_next_o(pc_next) );
37     control_unit cu ( .inst_i(inst), .inst_type_c(inst_type_c),
38                     .memwrite_c(memwrite_c), .memread_c(memread_c),
39                     .alusel1_c(alusel1_c), .alusel2_c(alusel2_c), .we_c(we_c),
40                     .memtoreg_c(memtoreg_c), .lui_c(lui_c), .branch_c(branch_c),
41                     .jump_c(jump_c), .pcsource_c(pcsource_c), .ebreak_c(ebreak_c) );
42
43     // Lógica de halt (parada por ebreak)
44     always_comb begin : Halt
45         halt = 1'b0;
46         if (RESET && ebreak_c) halt = 1'b1;
47     end
48
49     // Atualização do PC
50     always_ff @(posedge clk_i) begin : pc_update
51         if (!RESET) pc <= 32'd0;
52         else if (!halt) pc <= pc_next;
53     end
54
55     // Mux para seleção do dado a ser escrito no registrador
56     always_comb begin : mux_wdata
57         casez ({memtoreg_c, lui_c})
58             2'b00: wdata = aluout;
59             2'b10: wdata = rdata;
60             2'b?1: wdata = imm;
61             default: wdata = 32'd0;
62         endcase
63     end
64 endmodule

```

Listing 4.22 – Módulo topo integ.sv

Observações sobre a integração:

- O PC é atualizado na borda de subida do clock (`posedge clk_i`) e é zerado quando `RESET` está ativo.
- O sinal `halt` interrompe a atualização do PC quando uma instrução `ebreak` é executada, permitindo que o processador pare de forma controlada.
- Os campos da instrução (`rdaddr`, `rs1addr`, `rs2addr`, `funct3`, `funct7`) são extraídos diretamente da instrução lida da memória.

- O multiplexador de `wdata` seleciona entre o resultado da ALU, o dado lido da memória (`rdata`) e o imediato (para a instrução `lui`), conforme os sinais `memtoreg_c` e `lui_c`.

4.14.2 Testbench de Integração

O testbench `tb.sv` instancia o módulo `integ`, gera o clock e o sinal de reset, e fornece os valores de entrada para a interface de E/S. Para este teste, o programa armazenado na memória de instruções é o `fib.hex` (cálculo da sequência de Fibonacci). O valor de entrada é fornecido via `systeminput[1]`, que codifica o índice desejado e o sinal de confirmação.

```

1  import data_structures::*;
2
3  module tb ();
4      logic clk_c, RESET;
5      data_t [1:0] systeminput;
6      data_t [1:0] systemoutput;
7
8      integ integ (
9          .clk_i(clk_c),
10         .RESET(RESET),
11         .systeminput(systeminput),
12         .systemoutput(systemoutput)
13     );
14
15     always #10 clk_c = !clk_c;
16
17     initial begin
18         RESET = 0;
19         clk_c = 0;
20         #15;
21         RESET = 1; // Libera o processador
22
23         // systeminput[1] codifica valor e confirmação:
24         // 0 terceiro byte (bits 23:16) contém o sinal de confirmação.
25         // Quando este byte é igual a 0x80, o programa considera os dados válidos.
26         // Os demais bytes contêm o valor de entrada (índice).
27         systeminput[1] = 32'h05800000; // confirmação = 0x80, valor = 0x05 00 00?
28
29         $display("Iniciando execução do programa Fibonacci...");
30         #2800; // Tempo suficiente para a execução
31         $display("Fim da simulação. Valor de saída (segmentos): %h", systemoutput);
32         $finish;
33     end
34 endmodule

```

Listing 4.23 – Testbench `tb.sv` para integração final

No testbench, `systeminput[1]` é configurado com o valor `32'h05800000`. A interpretação correta, conforme o programa assembly, é:

- O terceiro byte (bits 23:16) contém o sinal de confirmação. No programa, o valor `0x80` (bit 7 ativo) é utilizado para indicar que os dados estão prontos.

- Os demais bytes (bits 15:0) contêm o valor de entrada propriamente dito (índice para Fibonacci). No exemplo, o valor efetivo é 0x0005.

O programa armazenado na memória de instruções (**fib.hex**) lê esses valores, processa o cálculo e escreve o resultado no endereço de saída (E/S 1020), que se reflete em **systemoutput**. O valor escrito já corresponde à codificação dos segmentos dos displays de sete segmentos, ou seja, cada dígito decimal é convertido pela sub-rotina **table** para o padrão de segmentos apropriado. A tabela utilizada (para cátodo comum, com segmentos ativos em nível baixo) mapeia:

- **case_0** 192 (0xC0)
- **case_1** 249 (0xF9)
- **case_2** 164 (0xA4)
- **case_3** 176 (0xB0)
- **case_4** 153 (0x99)
- **case_5** 146 (0x92)
- **case_6** 130 (0x82)
- **case_7** 248 (0xF8)
- **case_8** 128 (0x80)
- **case_9** 144 (0x90)

Assim, para o resultado 0005 (Fibonacci para entrada 5), o valor escrito em **systemoutput** é a concatenação: 0xC0C0C092 (quatro bytes: 0, 0, 0, 5). Para o fatorial de 5 (0120), o valor seria 0xC0F9A4C0 (0, 1, 2, 0).

4.14.3 Resultados da Simulação Integrada

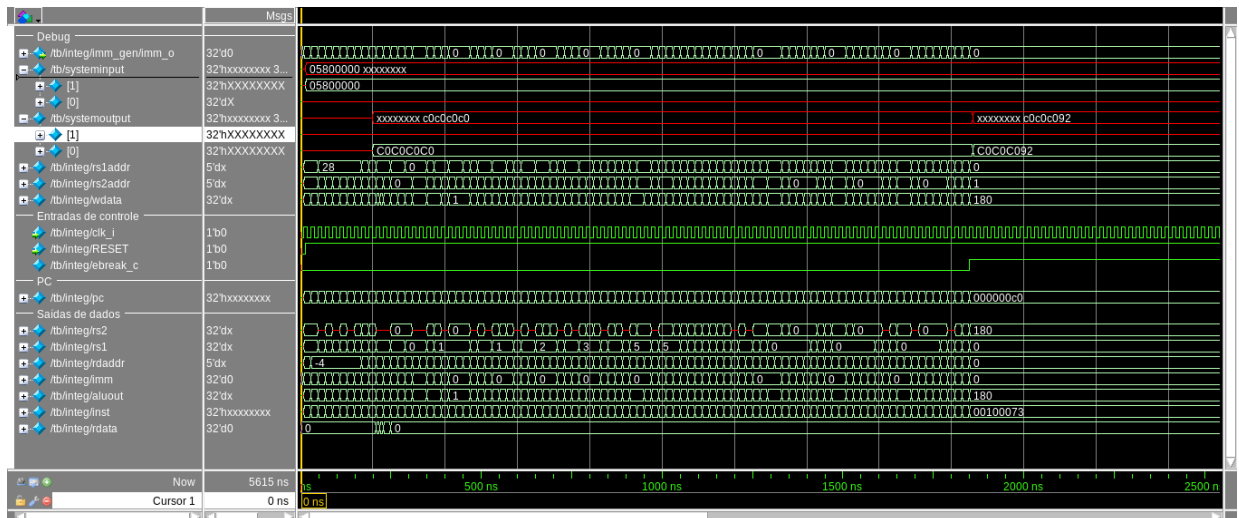


Figura 32 – Waveform da simulação integrada do processador (Fibonacci).

A Figura 32 apresenta uma visão geral da execução do programa Fibonacci (entrada 5). O waveform foi reduzido para mostrar todo o período de simulação (aproximadamente 2800 ns). Os principais eventos observados são:

- **Reset:** nos primeiros 15 ns, RESET está ativo, mantendo o PC zerado. Após a liberação, o processador inicia a execução a partir do endereço 0x00.
- **Execução do programa:** o PC avança sequencialmente, refletindo a busca e execução das instruções do programa `fib.hex`. O waveform mostra a variação contínua do PC e dos sinais de controle.
- **Leitura da entrada:** em determinado momento, o programa lê o valor de `systeminput[1]` no endereço de E/S correspondente (0xFFA). O valor `0x05800000` é interpretado como: byte de confirmação = `0x80` (ativo), e valor = `0x0005`. O programa aguarda até que o byte de confirmação seja igual a `0x80`, então prossegue com o cálculo.
- **Processamento:** a ALU executa as operações aritméticas (adições, deslocamentos, etc.) conforme a sequência de instruções. Os valores de `aluout`, `rdata` e `imm` variam ao longo do tempo.
- **Resultado final:** ao final da execução, o valor calculado (55 para Fibonacci de 5) é convertido para o padrão de segmentos dos displays pela sub-rotina `table`. O valor codificado `0xC0C0C092` (representando os dígitos 0, 0, 0, 5) é escrito no endereço de saída (E/S 1020), refletindo-se no sinal `systemoutput`. O waveform mostra o valor final em `systemoutput[0]` e `systemoutput[1]`.

- **Halt:** quando a instrução `ebreak` é executada, o sinal `ebreak_c` é ativado, e o PC para de ser atualizado, encerrando a execução de forma controlada.

A simulação completa do programa demonstra que todos os blocos do processador estão corretamente integrados e que o sistema é capaz de executar programas complexos, interagindo com a memória de dados e com a interface de E/S. O valor final obtido (`0xC0C0C092`) foi validado manualmente, confirmando o funcionamento do processador RISC-V monociclo.

4.14.3.1 Simulação do programa Fatorial

De forma análoga, o programa de cálculo do fatorial (`fat.hex`) foi executado com o mesmo testbench, alterando apenas o conteúdo da memória de instruções para `fat.hex` e ajustando o valor de entrada. Para a entrada 5, o resultado esperado é 120, que é convertido para `0xC0F9A4C0` (dígitos 0, 1, 2, 0). A Figura 33 apresenta o waveform correspondente.

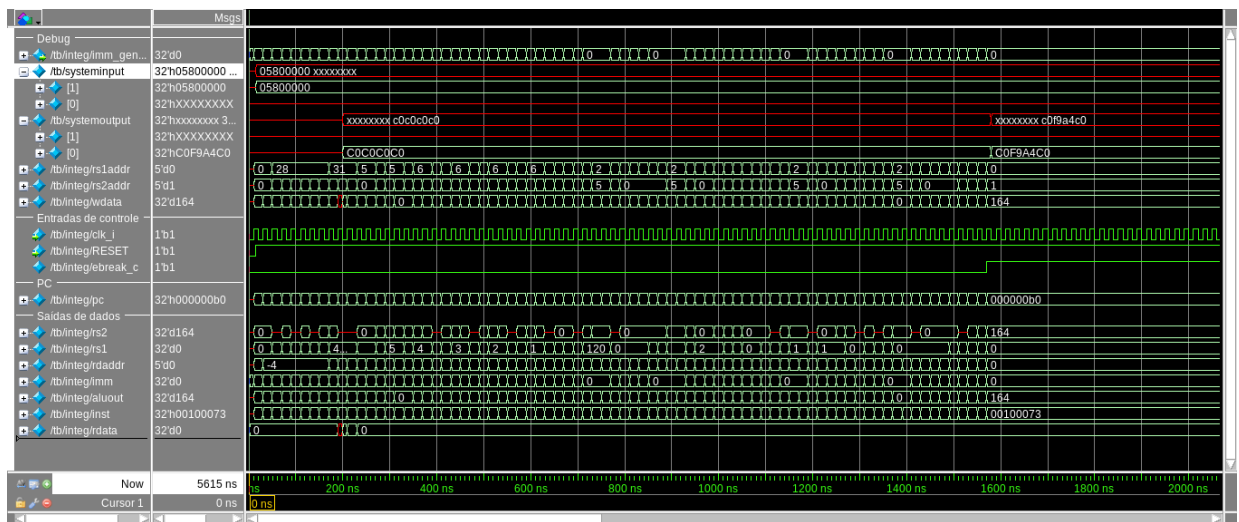


Figura 33 – Waveform da simulação integrada para o programa Fatorial.

Observa-se no waveform que o comportamento é análogo ao do Fibonacci: após o reset, o programa aguarda a confirmação, lê o valor de entrada, realiza o cálculo (utilizando a instrução `mul` da extensão M) e escreve o resultado codificado no endereço de saída. O valor final `0xC0F9A4C0` confirma o cálculo correto de $5! = 120$, demonstrando o pleno funcionamento da extensão de multiplicação.

Ambos os programas de teste comprovam a robustez do processador e a correta implementação de todas as instruções do conjunto RV32IM, incluindo a interação com a região de E/S mapeada em memória.

4.15 Implementação em FPGA e Testes em Bancada

Para viabilizar a execução do processador em hardware real, foi desenvolvido um módulo de topo específico para síntese em FPGA, denominado `rv32im`. Este módulo encapsula o processador (`integ`) e adiciona um divisor de clock, adequando a frequência de operação aos periféricos da bancada de testes (chaves e displays de sete segmentos).

4.15.1 Módulo de Topo para FPGA

O módulo `rv32im.sv` instancia o processador completo (`integ`) e inclui um contador que divide a frequência do clock de entrada (geralmente 50 MHz) para uma frequência mais baixa, compatível com a visualização dos displays e com a operação manual das chaves. A lógica de divisão implementada gera um clock interno (`clk_c`) que alterna a cada 50 ciclos do clock de entrada, resultando em um período de aproximadamente 2 μ s (para um clock de 50 MHz).

```

1  import data_structures::*;
2
3  module rv32im (
4      input logic clk_i,
5      input logic RESET,
6      input data_t [1:0] systeminput,
7      output data_t [1:0] systemoutput
8  );
9      int counter = 0;
10     logic clk_c = 0;
11
12     // Divisor de clock: gera um clock mais lento para a bancada
13     always_ff @(posedge clk_i) begin : Divisor
14         if (counter == 50) begin
15             counter <= 0;
16             clk_c    <= ~clk_c;
17         end else begin
18             counter <= counter + 1;
19         end
20     end
21
22     // Instância do processador
23     integ integ (
24         .clk_i(clk_c),
25         .RESET(RESET),
26         .systeminput(systeminput),
27         .systemoutput(systemoutput)
28     );
29 endmodule

```

Listing 4.24 – Módulo `rv32im.sv` topo para FPGA

O módulo `rv32im` é o arquivo de topo utilizado para a síntese no Quartus Prime, sendo responsável por conectar o processador aos pinos da FPGA. Os sinais `systeminput` e `systemoutput` são mapeados para as chaves e displays da bancada, respectivamente, permitindo a interação com o usuário.

4.15.2 Processo de Síntese e Programação

A síntese do processador foi realizada utilizando o Quartus Prime, com as seguintes etapas:

1. Importação de todos os arquivos SystemVerilog do projeto (`alu.sv`, `branch_unit.sv`, `control_unit.sv`, `data_memory.sv`, `data_structures.sv`, `immediate_generator.sv`, `inst_memory.sv`, `regfile.sv`, `integ.sv`, `rv32im.sv`).
2. Configuração do dispositivo FPGA (modelo utilizado na bancada de testes).
3. Atribuição dos pinos de entrada e saída (clock, reset, chaves e displays) conforme a placa de desenvolvimento.
4. Compilação e geração do arquivo de configuração (`.sof` ou `.pof`).
5. Programação da FPGA utilizando o programador do Quartus.

A memória de instruções é inicializada com o programa desejado (Fibonacci ou Fatorial) através do arquivo `fib.hex` ou `fat.hex`, que deve ser carregado na memória ROM durante a síntese ou via `$readmemh` (conforme implementado no módulo `inst_memory`).

4.15.3 Testes em Bancada

Até o momento da elaboração deste relatório, a apresentação prática no balcão ainda não foi realizada. No entanto, os testes iniciais em bancada foram conduzidos com sucesso, utilizando os programas Fibonacci e Fatorial. Os resultados obtidos em hardware foram consistentes com as simulações realizadas no ModelSim:

- **Programa Fibonacci (entrada 5):** os displays exibiram a sequência de segmentos correspondente ao valor 0005 (código `0xC0C0C092`), confirmando o cálculo correto do 5º termo da sequência.
- **Programa Fatorial (entrada 5):** os displays exibiram o valor 0120 (código `0xC0F9A4C0`), confirmando o cálculo correto de $5! = 120$.

Esses resultados preliminares indicam que o processador está funcionando conforme o esperado em hardware, validando a integração dos módulos e a correta implementação do conjunto de instruções RV32IM. A apresentação formal no balcão, com a demonstração completa dos programas e a interação com os periféricos, será realizada para o professor responsável na semana seguinte à entrega do relatório.

5 Considerações Finais

O trabalho desenvolvido atingiu todos os objetivos propostos para a implementação de um processador RISC-V monociclo completo. Com base nos arquivos codificados em SystemVerilog, nas simulações realizadas e nos testes em FPGA, os seguintes elementos foram completamente projetados, testados e documentados:

- **Unidade Lógica e Aritmética (ULA):** implementada com suporte a 24 operações, incluindo aritméticas básicas, lógicas, deslocamentos, comparações e a extensão de multiplicação e divisão (RV32M). O módulo foi validado por meio de testbenches exaustivos, e as formas de onda comprovam a correção de cada operação.
- **Banco de Registradores (Register File):** composto por 32 registradores de 32 bits, com o registrador x0 permanentemente ligado em zero. As simulações demonstraram a correta escrita síncrona e leitura combinacional.
- **Memória de Instruções (ROM):** com 256 palavras de 32 bits, inicializada a partir de arquivo hexadecimal e com leitura combinacional.
- **Memória de Dados (RAM):** implementada como memória síncrona de 1024 palavras (clock na borda de descida), com lógica de *byte enable* que permite acessos a byte, meia palavra e palavra, além de distinguir corretamente extensão de sinal (1b, 1h) de extensão de zero (1bu, 1hu), e com região de E/S mapeada nos quatro últimos endereços.
- **Gerador de Imediatos (Immediate Generator):** módulo combinacional que extrai e estende os imediatos dos formatos I, S, B, U e J, validado com testbench específico.
- **Unidade de Branch:** módulo combinacional que calcula o próximo PC para desvios condicionais, saltos incondicionais e saltos indiretos, validado com testbench próprio.
- **Unidade de Controle:** responsável pela decodificação das instruções e geração de todos os sinais de controle, incluindo o suporte à instrução de sistema `ebreak` para interrupção controlada da execução.
- **Contador de Programa (PC):** registrador síncrono de 32 bits com reset, implementado no módulo topo e atualizado a cada ciclo de clock.

- **Integração Final:** todos os módulos foram integrados no módulo topo `integ`, formando o caminho de dados completo do processador monociclo. A simulação integrada executou programas de teste (Fibonacci e Fatorial) com sucesso.
- **Implementação em FPGA:** o processador foi sintetizado para um kit FPGA utilizando o módulo `rv32im`, que adiciona um divisor de clock para operação em bancada. Os testes em hardware confirmaram os resultados obtidos em simulação, com os displays exibindo os valores corretos para os programas executados.

Todos os componentes foram simulados individualmente no ModelSim, e os resultados documentados por meio de capturas de tela (waveforms) atestam o funcionamento conforme a especificação RISC-V. A execução dos programas de teste em bancada produziu resultados consistentes com as simulações, demonstrando a correta implementação do processador em hardware.

A escolha da arquitetura RISC-V e da linguagem SystemVerilog mostrou-se adequada, permitindo uma descrição clara e modular dos componentes. As ferramentas de simulação (ModelSim) e de síntese (Quartus Prime) foram suficientes para a validação funcional e para a materialização do projeto em FPGA. O divisor de clock implementado no módulo `rv32im` mostrou-se eficaz para adequar a frequência de operação aos periféricos da bancada.

5.0.1 Dificuldades Encontradas

Durante o desenvolvimento do projeto, algumas dificuldades técnicas merecem destaque, pois exigiram atenção especial e aprofundamento no entendimento da linguagem SystemVerilog, da arquitetura RISC-V e das ferramentas de síntese e simulação.

A principal dificuldade esteve relacionada à correta manipulação de tipos com e sem sinal (*signed* vs. *unsigned*) dentro da Unidade Lógica e Aritmética (ULA). Em operações como deslocamento aritmético à direita (SRA) e comparações com sinal (SLT, SLTI), é fundamental garantir que os operandos sejam tratados como números com sinal. A linguagem SystemVerilog, por padrão, considera vetores (`logic [31:0]`) como sem sinal. A solução encontrada foi utilizar conversões explícitas com `$signed()` nos operandos, como exemplificado no trecho:

```
1 SRA: aluout_o = $signed(op_1) >>> op_2[4:0];
2 SLT: aluout_o[0] = ($signed(op_1) < $signed(op_2)) ? 1'b1 : 1'b0;
```

A ausência desses *casts* resultava em comportamentos incorretos, especialmente quando operandos negativos eram envolvidos. A dificuldade foi ainda maior nas operações de multiplicação e divisão da extensão M, onde era necessário tratar corretamente os produtos de 64 bits (MUL, MULH, MULHSU, MULHU) e as divisões com sinal (DIV, REM).

Outro ponto desafiador foi a implementação das instruções que utilizam a parte superior ou inferior de um produto de 64 bits (`MULH`, `MULHSU`, `MULHU`). Para essas operações, foi necessário armazenar o resultado completo de 64 bits em um registrador temporário (`tmp_reg`) e, em seguida, selecionar os 32 bits apropriados (`tmp_reg[63:32]` ou `tmp_reg[31:0]`). A lógica de extensão de sinal para os operandos em `MULHSU` (um operando com sinal e outro sem sinal) exigiu a criação de um operando estendido com `{1'b0, op_2}` para garantir a interpretação correta.

Além disso, a lógica de *byte enable* na memória de dados para acessos a byte e meia palavra (`lb`, `lbu`, `lh`, `lhu`, `sb`, `sh`) demandou cuidado no alinhamento de endereços e na geração correta dos sinais de habilitação (`be`). A simulação desses casos com endereços não alinhados foi essencial para validar a implementação.

Uma dificuldade específica da etapa de validação em hardware foi o *divisor de clock* implementado no módulo `rv32im`. O sinal gerado pelo contador (`clk_c`) não pôde ser simulado adequadamente no QuestaSim (ModelSim), pois a ferramenta de simulação não o interpreta como um sinal de clock real para efeitos de temporização e *timing analysis*. Isso exigiu que a validação do divisor de clock fosse realizada diretamente em bancada, onde o comportamento pôde ser observado nos displays e chaves. Para as simulações, o processador foi testado com o clock original (`clk_i`), sem o divisor, garantindo a correteude funcional independentemente da frequência de operação.

Por fim, a *síntese* do projeto no Quartus Prime apresentou tempos de compilação excessivamente longos, especialmente devido à complexidade do caminho de dados e à quantidade de módulos interconectados. Para contornar esse problema, foram utilizadas configurações especiais de compilação, como a redução do esforço de otimização (`Optimization Effort = Standard`) e a ativação de opções de compilação rápida (`Fast Compile`). Essas alterações reduziram significativamente o tempo de síntese, permitindo iterações mais rápidas durante os testes em bancada, sem comprometer a funcionalidade do processador.

Todas essas dificuldades foram superadas com estudo da documentação do SystemVerilog, simulações incrementais, e a utilização de testbenches dedicados para cada módulo, o que permitiu isolar e corrigir os problemas antes da integração final e da síntese em hardware.

Referências

Arch Linux. *riscv64-elf-gcc - Arch User Repository*. 2025. Acesso em: 12 jul. 2026. Disponível em: <<https://aur.archlinux.org/packages/riscv64-elf-gcc>>. Citado na página 82.

fish-shell. *fish shell documentation*. [S.l.], 2025. Acesso em: 12 jul. 2026. Disponível em: <<https://fishshell.com/docs/current/index.html>>. Citado na página 82.

Free Software Foundation. *GNU Compiler Collection - RISC-V Options*. 2025. Acesso em: 12 jul. 2026. Disponível em: <<https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html>>. Citado na página 82.

PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. 1. ed. [S.l.]: Morgan Kaufmann, 2018. Citado na página 11.

Python Software Foundation. *Python Standard Library - int.from_bytes*. [S.l.], 2025. Acesso em: 12 jul. 2026. Disponível em: <https://docs.python.org/3/library/stdtypes.html#int.from_bytes>. Citado na página 83.

RISC-V INTERNATIONAL. *RISC-V Instruction Set Manual, Volume I*. 2024. Acesso em: 04 abril 2026. Disponível em: <riscv.org>. Citado 3 vezes nas páginas 20, 21 e 27.

STALLINGS, W. *Arquitetura e Organização de Computadores*. 10. ed. São Paulo: Pearson, 2017. Citado na página 18.